TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

HUỲNH KIẾN DUY

ĐỀ TÀI

ỨNG DỤNG DỤNG ĐỘNG ĐẶT VÉ MÁY BAY

ĐỒ ÁN 2(ĐỒÁNCƠSỞKTPM)

NGÀNH KỸ THUẬT PHẦN MỀM

Mã số ngành: 7480201

Cần Thơ–2026

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

HUỲNH KIẾN DUY

225345

DH22KPM01

ĐỀ TÀI

ỨNG DỤNG DI ĐỘNG ĐẶT VÉ MÁY BAY

ĐỒ ÁN 2

NGÀNH KỸ THUẬT PHẦN MỀM

Mã số ngành: 7480201

Cần Thơ, ngày 17 tháng 2 năm 2025

GIẢNGVIÊNHƯỚNGDẪN

TRẦN VĂN THIỆN

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý thầy cô trong khoa Công nghệ Thông tin đã truyền đạt cho em nền tảng kiến thức về lập trình di động, cấu trúc dữ liệu, kỹ thuật phần mềm, cơ sở dữ liệu và phân tích thiết kế hệ thống. Những học phần này là nền tảng quan trọng để em có thể nghiên cứu và hoàn thiện đề tài ứng dụng đặt vé máy bay trên nền tảng Android.

Em xin gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn đã dành thời gian góp ý, định hướng cách tiếp cận đề tài, chỉ ra các nội dung cần hoàn thiện trong báo cáo và hỗ trợ em trong quá trình chuẩn hóa sản phẩm. Các góp ý về cách trình bày tài liệu, mô tả luồng nghiệp vụ và đánh giá hiện trạng hệ thống đã giúp em nhìn nhận đồ án dưới góc độ kỹ thuật và thực tiễn hơn.

Mặc dù đã nỗ lực hoàn thiện báo cáo theo hướng đầy đủ và bám sát mã nguồn, em nhận thấy sản phẩm vẫn còn những điểm có thể tiếp tục cải tiến như kiến trúc MVVM, cơ chế xác thực bằng token, tách lớp repository, tăng cường kiểm thử và tối ưu bảo mật mạng. Em rất mong nhận được thêm nhận xét của quý thầy cô để có thể tiếp tục nâng cao chất lượng ứng dụng và kỹ năng chuyên môn của bản thân.

Em xin chân thành cảm ơn!

Cần Thơ, ngày ..... tháng ..... năm 2026

Người thực hiện

(Ký, ghi rõ họ tên)

HUỲNH KIẾN DUY

LỜI CAM KẾT

Tôi xin cam kết rằng nội dung chuyên môn trong báo cáo này được biên soạn trung thực dựa trên việc phân tích mã nguồn của repo Android được cung cấp và tham chiếu cấu trúc từ báo cáo mẫu mà người dùng đã tải lên. Các sơ đồ, bảng biểu, hình minh họa và phần nhận xét kỹ thuật trong tài liệu đều được xây dựng mới nhằm phản ánh sát đặc điểm của dự án ứng dụng đặt vé máy bay trên Android.

Tôi chịu trách nhiệm về tính nhất quán của phần trình bày, đồng thời ý thức rằng các thông tin cá nhân ở trang bìa và trang xác nhận cần được cập nhật lại bởi chủ đề tài trước khi in, nộp hoặc đóng quyển chính thức.

Cần Thơ, ngày ..... tháng ..... năm 2026

Người thực hiện

(Ký, ghi rõ họ tên)

HUỲNH KIẾN DUY

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……

Cần Thơ, ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn	**Người thực hiện**
(Ký, ghi rõ họ tên)	(Ký, ghi rõ họ tên)
	TRẦN VĂN THIỆN

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Cần Thơ, ngày ..... tháng ..... năm 2026

Giảng viên phản biện	**Người thực hiện**
(Ký, ghi rõ họ tên)	(Ký, ghi rõ họ tên)
	TRẦN VĂN THIỆN

MỤC LỤC

CHƯƠNG 1. XÁC ĐỊNH BÀI TOÁN VÀ YÊU CẦU HỆ THỐNG 1
1.1. Bối cảnh và lý do chọn đề tài 1
1.2. Mục tiêu của đề tài 2
1.2.1. Mục tiêu tổng quát 2
1.2.2. Mục tiêu cụ thể 2
1.3. Phạm vi, đối tượng và phương pháp thực hiện 4
1.3.1. Phạm vi nghiên cứu 4
1.3.2. Đối tượng nghiên cứu 5
1.3.3. Phương pháp thực hiện 6
1.4. Các tác nhân và nhu cầu sử dụng 8
1.5. Yêu cầu chức năng 11
1.6. Yêu cầu phi chức năng 12
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT, CÔNG NGHỆ VÀ PHÂN TÍCH DỰ ÁN 13
2.1. Kiến trúc ứng dụng Android theo mô hình client-server 13
2.2. Android native Java và vòng đời Activity 13
2.3. Retrofit, Gson và mô hình REST API 14
2.4. SharedPreferences và quản lý phiên 14
2.5. Vật liệu giao diện AndroidX / Material và thông báo cục bộ 15
2.6. Phân tích cấu trúc dự án 16
2.7. Môi trường phát triển và vận hành 18
2.8. Về lựa chọn công nghệ 19
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 20
3.1. Kiến trúc tổng thể của ứng dụng 20
3.2. Phân rã module và cấu trúc package 21
3.3. Mô tả Use Case 23
3.3.1. Use Case tổng quát 23
3.3.2. Use Case đăng kí đăng nhập 24
3.3.3. của người dùng sau khi đăng nhập 24
3.3.1. Use Case rest api 25
3.4. Thiết kế dữ liệu phía client 27
3.5. Thiết kế lớp API 30
3.6. Thiết kế điều hướng màn hình 32
3.7. Thiết kế luồng xác thực, tạo tài khoản và khôi phục mật khẩu 34
3.8. Thiết kế luồng tìm kiếm, chọn ghế và đặt vé 36
3.9. Thiết kế lịch sử booking, hồ sơ và thông báo 39
3.10. Thiết kế lỗi, validate và bảo mật 40
CHƯƠNG 4. KẾT QUẢ TRIỂN KHAI THEO MÃ NGUỒN 42
4.1. Cấu trúc package và trách nhiệm từng nhóm lớp 42
4.2. Kết quả giao diện nhóm xác thực 42

4.2.1. MainActivity và Login 43
4.2.2. Signup, ForgotPasswordActivity và ResetPasswordActivity 44
4.3. Kết quả giao diện nhóm tìm chuyến bay và AI Planner 45
4.3.1. MainMenuActivity 46
4.3.2. SearchFlightActivity và FlightResultsActivity 46
4.3.3. AIPlannerActivity 47
4.4. Kết quả giao diện nhóm chọn ghế, tạo booking và thanh toán 48
4.4.1. ChooseSeatsActivity 49
4.4.2. BookingFormActivity, BookingDetailActivity và PayActivity 50
4.5. Kết quả giao diện nhóm lịch sử, hồ sơ và thông báo 51
4.5.1. BookingHistoryActivity 52
4.5.2. Profile và NotificationActivity 53
4.6. Kết quả triển khai khối API và DTO 54
4.7. Các luồng nghiệp vụ cốt lõi được hiện thực 55
CHƯƠNG 5. KẾT QUẢ TRIỂN KHAI, KIỂM THỬ VÀ HƯỚNG PHÁT TRIỂN 58
5.1. Kết quả đạt được 58
5.2. Kiểm thử theo kịch bản 58
5.3. Kiểm thử thực nghiệm và kết quả quan sát 59
5.4. Phân tích hiện trạng triển khai 60
5.4.1. Nội dung kỹ thuật cần tiếp tục hoàn thiện 60
5.5. Hướng phát triển 61
5.6. Mức độ hoàn thành so với mục tiêu ban đầu 61
CHƯƠNG 6. MÔ-ĐUN AI TRONG HỆ THỐNG 62
6.1. Vai trò của AI trong hệ thống 63
6.2. Cách AI hoạt động trong hệ thống 63
6.3. Phương pháp và xu hướng triển khai AI 64
6.4. Sơ đồ luồng dữ liệu của mô-đun AI 65
6.5. Sơ đồ hoạt động của mô-đun AI 65
6.6. Cách AI xử lý và đưa ra dữ liệu cho người dùng 67

TÀI LIỆU THAM KHẢO 68
PHỤ LỤC 69
PL.1. Danh mục file mã nguồn trọng yếu 69
PL.2. Ma trận truy vết yêu cầu – hiện thực 69
PL.3. Hướng dẫn cài đặt, triển khai và cấu hình chương trình 70
PL.4. Hướng dẫn sử dụng chương trình 71
PL.5. Vị trí đề xuất chèn ảnh chụp màn hình thực tế 71
PL.6. Gợi ý kịch bản demo khi bảo vệ đồ án 72
PL.7. Khuyến nghị chỉnh sửa trước khi nộp bản chính thức 73
PL.8. Bảng khóa SharedPreferences và nơi sử dụng 73
PL.9. Bảng mô tả DTO được dùng trực tiếp trong luồng nghiệp vụ 73

DANH SÁCH HÌNH

Hình 3.1. Kiến trúc tổng thể của ứng dụng đặt vé máy bay trên Android. 20
Hình 3.2. Sơ đồ module / package chính trong repo. 22
Hình 3.3. Use Case tổng quát của ứng dụng. 23
Hình 3.4. Use Case đăng kí đăng nhập.....24
Hình 3.5. Use Case của người dùng sau khi đăng nhập 25
Hình 3.6. Use Case rest api. 26
Hình 3.7. Thiết kế dữ liệu phía client: DTO, SharedPreferences và Intent extras. 29
Hình 3.8. Sơ đồ điều hướng giữa các Activity chính. 33
Hình 3.9. Sequence Diagram luồng đăng nhập và quên mật khẩu.....35
Hình 3.10. Activity tạo tài khoảng và đăng nhập 35
Hình 3.11. Actiuvity xác thực và khôi phục mật khẩu 36
Hình 3.12. Sequence Diagram luồng tìm kiếm, chọn ghế và đặt vé. 37
Hình 3.13. Activity Diagram luồng tìm chuyến bay đến hoàn tất booking. 39
Hình 3.14. State Diagram trạng thái booking và phiên người dùng.....40
Hình 4.1. Giao diện nhóm xác thực người dùng. 43
Hình 4.2. Giao diện nhóm đăng kí, quên mật khẩu.....44
Hình 4.3. Signup, ForgotPasswordActivity và ResetPasswordActivity 45
Hình 4.4. Giao diện màn hình chính và tìm kiếm chuyến bay. 46
Hình 4.5. MainMenuActivity 46
Hình 4.6. SearchFlightActivity và FlightResultsActivity 47
Hình 4.7. AIPlannerActivity.....48
Hình 4.8. Minh họa giao diện chọn ghế, tạo booking và thanh toán.....49
Hình 4.9. ChooseSeatsActivity.....50
Hình 4.10. BookingFormActivity, BookingDetailActivity và PayActivity.....51
Hình 4.11. Minh họa giao diện hồ sơ, lịch sử và thông báo.....52
Hình 4.12. BookingHistoryActivity 53
Hình 4.13. Profile và NotificationActivity 54
Hình 6.1. Sơ đồ luồng dữ liệu của mô-đun AI trong hệ thống.....65
Hình 6.2. Sơ đồ hoạt động của mô-đun AI trong ứng dụng 66

DANH SÁCH BẢNG

Bảng 1.1. Phân tích nhóm tác nhân và nhu cầu sử dụng 8
Bảng 1.2. Tổng hợp yêu cầu chức năng chính 11
Bảng 1.3. Tổng hợp yêu cầu phi chức năng 12
Bảng 2.1. Công nghệ và thư viện được sử dụng trong dự án 16
Bảng 2.2. Cấu trúc package / nhóm lớp chính 16
Bảng 2.3. Ánh xạ Activity – vai trò – nguồn dữ liệu 17
Bảng 2.4. Môi trường phát triển, vận hành và cấu hình triển khai 18
Bảng 3.1. Mô hình dữ liệu người dùng và lưu trữ cục bộ 30
Bảng 3.2. Danh mục endpoint thuộc AuthApiEndpoint 30
Bảng 3.3. Danh mục endpoint thuộc BookingApiEndpoint 31
Bảng 3.4. Danh mục endpoint thuộc FlightApiEndpoint 31
Bảng 3.5. Ma trận Intent extras trong ứng dụng 31
Bảng 4.1. Danh sách màn hình chính và trách nhiệm xử lý 42
Bảng 4.3. Ma trận chức năng – Activity – API liên quan 55
Bảng 5.1. Kế hoạch kiểm thử chức năng 58
Bảng 5.2. Kết quả test case tiêu biểu 59
Bảng 5.3. Nội dung kỹ thuật cần tiếp tục hoàn thiện 60
Bảng 6.1. Thành phần của mô-đun AI và vai trò xử lý 64
Bảng 6.2. Chuỗi xử lý dữ liệu của mô-đun AI 67
Bảng PL.1. Danh mục file mã nguồn trọng yếu 70
Bảng PL.2. Ma trận truy vết yêu cầu – hiện thực 70
Bảng PL.3. Yêu cầu môi trường cài đặt và cấu hình triển khai 71
Bảng PL.4. Vị trí đề xuất bổ sung ảnh minh chứng thực tế 72
Bảng PL.5. Khóa SharedPreferences và nơi sử dụng 74
Bảng PL.6. DTO được dùng trực tiếp trong luồng nghiệp vụ 74

DANH MỤC TỪ VIẾT TẮT

Bảng M0.1. Danh mục từ viết tắt

Từ viết tắt	Diễn giải
API	Application Programming Interface – giao diện lập trình ứng dụng
DTO	Data Transfer Object – đối tượng truyền dữ liệu
UI	User Interface – giao diện người dùng
UX	User Experience – trải nghiệm người dùng
OTP	One-Time Password – mật khẩu dùng một lần
JSON	JavaScript Object Notation – định dạng trao đổi dữ liệu
REST	Representational State Transfer – kiến trúc web service
SDK	Software Development Kit – bộ công cụ phát triển
XML	eXtensible Markup Language
HTTP	HyperText Transfer Protocol
SP	SharedPreferences – cơ chế lưu key-value cục bộ trên Android
UML	Unified Modeling Language

CHƯƠNG 1.
XÁC ĐỊNH BÀI TOÁN VÀ YÊU CẦU HỆ THỐNG

1.1. Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và xu hướng chuyển đổi số, ngành hàng không đã từng bước thay đổi phương thức cung cấp dịch vụ từ mô hình truyền thống sang mô hình trực tuyến. Nếu trước đây khách hàng phải đến trực tiếp quầy giao dịch hoặc liên hệ qua đại lý để tra cứu thông tin chuyến bay, đặt vé và thực hiện các thủ tục liên quan, thì hiện nay phần lớn nhu cầu này đã được chuyển sang các nền tảng số như website và ứng dụng di động. Sự thay đổi này không chỉ giúp doanh nghiệp tối ưu quy trình vận hành mà còn nâng cao trải nghiệm người dùng thông qua khả năng tra cứu nhanh, đặt vé mọi lúc mọi nơi và quản lý hành trình một cách thuận tiện.

Bên cạnh đó, điện thoại thông minh ngày càng trở thành thiết bị phổ biến trong đời sống hằng ngày. Người dùng có xu hướng sử dụng ứng dụng di động để thực hiện các tác vụ cá nhân như mua sắm, thanh toán, đặt xe, đặt phòng khách sạn và đặt vé máy bay. Trong bối cảnh đó, việc xây dựng một ứng dụng đặt vé máy bay trên nền tảng Android là một hướng tiếp cận phù hợp với xu thế công nghệ hiện nay, đồng thời đáp ứng được nhu cầu thực tế của người dùng về tính tiện lợi, nhanh chóng và trực quan.

Lý do đầu tiên để lựa chọn đề tài này là vì đây là một bài toán có tính thực tế cao. Nhu cầu đặt vé máy bay hiện nay ngày càng phổ biến do sự phát triển của du lịch, công tác, giao thương và nhu cầu di chuyển giữa các tỉnh thành, quốc gia. Một ứng dụng hỗ trợ người dùng tìm kiếm và đặt vé nhanh chóng sẽ mang lại giá trị sử dụng rõ ràng, gần gũi với đời sống thực tế và có khả năng mở rộng thành sản phẩm hoàn chỉnh trong tương lai.

Lý do thứ hai là đề tài có phạm vi nghiệp vụ đủ rộng để thể hiện được năng lực phân tích, thiết kế và phát triển phần mềm. So với các đề tài chỉ dừng ở mức quản lý dữ liệu đơn giản, hệ thống đặt vé máy bay bao gồm nhiều chức năng liên kết với nhau, tạo thành một quy trình xử lý tương đối hoàn chỉnh. Trong dự án này, người dùng có thể thực hiện các thao tác từ khởi tạo tài khoản, đăng nhập hệ thống, tìm kiếm chuyến bay, xem kết quả, chọn ghế, nhập thông tin hành khách, xác nhận đặt vé, xem lịch sử

giao dịch, theo dõi chi tiết đặt vé cho đến cập nhật thông tin hồ sơ cá nhân. Điều này giúp đề tài có chiều sâu cả về mặt chức năng lẫn kỹ thuật triển khai.

Ngoài ra, đề tài còn cho phép tiếp cận một số tình huống nghiệp vụ gần với hệ thống thực tế, chẳng hạn như kiểm tra thông tin đăng nhập, xác thực người dùng, quên mật khẩu và đặt lại mật khẩu, phân loại trạng thái đặt vé, xem chi tiết vé, cũng như hỗ trợ quản lý thông tin người dùng sau khi đăng nhập. Đây là những chức năng thường xuất hiện trong các hệ thống thương mại điện tử và dịch vụ trực tuyến hiện đại, vì vậy việc triển khai đề tài giúp nâng cao khả năng xây dựng ứng dụng theo hướng thực tiễn hơn thay vì chỉ mang tính mô phỏng.

1.2. Mục tiêu của đề tài

1.2.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng một ứng dụng đặt vé máy bay trên nền tảng Android nhằm hỗ trợ người dùng thực hiện các thao tác cơ bản trong quy trình đặt vé một cách thuận tiện, nhanh chóng và trực quan. Ứng dụng hướng tới việc số hóa các nghiệp vụ chính như đăng ký tài khoản, đăng nhập hệ thống, tìm kiếm chuyến bay, lựa chọn ghế ngồi, nhập thông tin hành khách, xác nhận đặt vé, theo dõi lịch sử đặt chỗ và quản lý hồ sơ cá nhân.

Thông qua đề tài, có thể vận dụng các kiến thức đã học về phát triển ứng dụng di động, thiết kế giao diện người dùng, xử lý dữ liệu, giao tiếp với hệ thống backend qua API và tổ chức luồng nghiệp vụ trong một hệ thống phần mềm có tính thực tiễn. Bên cạnh đó, đề tài còn hướng đến việc tạo ra một sản phẩm có cấu trúc chức năng tương đối hoàn chỉnh, phản ánh được quy trình đặt vé máy bay trong thực tế ở mức phù hợp với phạm vi đồ án.

1.2.2. Mục tiêu cụ thể

Xây dựng chức năng quản lý tài khoản người dùng. Hệ thống cần cho phép người dùng đăng ký tài khoản mới, đăng nhập vào ứng dụng, lưu trạng thái đăng nhập và sử dụng thông tin tài khoản trong toàn bộ quá trình sử dụng. Ngoài ra, hệ thống cũng cần hỗ trợ các chức năng liên quan đến bảo mật tài khoản như quên mật khẩu, xác thực OTP và đặt lại mật khẩu, nhằm bảo đảm tính liên tục khi người dùng gặp sự cố đăng nhập.

Xây dựng chức năng tìm kiếm chuyến bay. Ứng dụng cần hỗ trợ người dùng nhập các thông tin cơ bản như sân bay khởi hành, sân bay đến, ngày đi, ngày về, số lượng hành khách và hạng ghế để thực hiện tìm kiếm chuyến bay phù hợp. Kết quả tìm kiếm phải được hiển thị rõ ràng, trực quan và hỗ trợ người dùng tiếp tục các bước đặt vé tiếp theo.

Xây dựng chức năng hiển thị kết quả chuyến bay và lựa chọn chuyến phù hợp. Sau khi thực hiện tìm kiếm, người dùng cần được cung cấp danh sách các chuyến bay thỏa mãn điều kiện đầu vào. Hệ thống phải hỗ trợ việc xem thông tin cơ bản của chuyến bay như mã chuyến bay, thời gian khởi hành, thời gian đến, tuyến bay và các thông tin liên quan khác để người dùng đưa ra quyết định lựa chọn.

Xây dựng chức năng lựa chọn ghế ngồi. Đây là một mục tiêu quan trọng của đề tài. Hệ thống cần mô phỏng sơ đồ ghế trên chuyến bay, cho phép người dùng quan sát trạng thái ghế, lựa chọn một hoặc nhiều ghế phù hợp, đồng thời nhập thông tin hành khách tương ứng với từng ghế đã chọn. Chức năng này giúp quy trình đặt vé trở nên gần với thực tế hơn và làm tăng giá trị ứng dụng của hệ thống.

Xây dựng chức năng đặt vé và xác nhận đặt chỗ. Sau khi người dùng hoàn tất việc chọn ghế và nhập thông tin hành khách, hệ thống cần hỗ trợ tạo đơn đặt vé, gửi dữ liệu đến backend và nhận phản hồi xác nhận đặt vé thành công. Đồng thời, ứng dụng cần hiển thị mã đặt chỗ, thông tin tóm tắt đơn đặt vé và phản hồi phù hợp khi có lỗi xảy ra trong quá trình xử lý.

Xây dựng chức năng quản lý lịch sử đặt vé. Người dùng cần có khả năng xem lại danh sách các vé đã đặt trước đó, tra cứu theo mã đặt chỗ hoặc thông tin liên quan, đồng thời theo dõi trạng thái của từng đơn đặt vé. Đây là chức năng giúp tăng tính tiện ích và hỗ trợ người dùng quản lý hành trình cá nhân hiệu quả hơn.

Xây dựng chức năng xem chi tiết đặt vé. Bên cạnh việc xem danh sách lịch sử, hệ thống cần cho phép người dùng truy cập vào từng đơn đặt vé cụ thể để xem thông tin chi tiết như mã đặt chỗ, hành trình, thời gian bay, tổng tiền, trạng thái đặt vé và các dữ liệu liên quan khác. Mục tiêu này giúp hoàn thiện luồng nghiệp vụ của hệ thống.

Xây dựng chức năng quản lý hồ sơ cá nhân. Ứng dụng cần hỗ trợ người dùng xem thông tin cá nhân đã đăng ký, đồng thời cho phép cập nhật một số nội dung cần thiết như họ tên, email, số điện thoại và các thông tin cá nhân khác. Chức năng này giúp hệ thống không chỉ phục vụ việc đặt vé mà còn hỗ trợ người dùng quản lý tài khoản lâu dài.

Xây dựng chức năng thông báo trong ứng dụng. Hệ thống hướng tới việc hiển thị các thông báo liên quan đến hoạt động của người dùng, chẳng hạn như thông báo đặt vé thành công hoặc các thông báo nội bộ khác. Mục tiêu này góp phần nâng cao trải nghiệm sử dụng và tăng tính tương tác giữa hệ thống với người dùng.

Hoàn thiện kỹ năng phát triển ứng dụng di động thông qua dự án thực tế. Bên cạnh mục tiêu về sản phẩm, đề tài còn hướng tới mục tiêu học tập. Trong quá trình thực hiện, nhóm cần rèn luyện và củng cố các kỹ năng như phân tích yêu cầu, thiết kế giao diện, tổ chức mã nguồn, xử lý điều hướng giữa các màn hình, kết nối API bằng Retrofit, quản lý dữ liệu bằng DTO, làm việc với SharedPreferences và xử lý các tình huống lỗi trong ứng dụng Android.

1.3. Phạm vi, đối tượng và phương pháp thực hiện

1.3.1. Phạm vi nghiên cứu

Phạm vi nghiên cứu của đề tài tập trung vào việc phân tích, thiết kế và xây dựng ứng dụng đặt vé máy bay trên nền tảng Android dành cho người dùng cuối. Nội dung nghiên cứu hướng đến các chức năng chính của một hệ thống đặt vé máy bay trên thiết bị di động, bao gồm đăng ký tài khoản, đăng nhập, quên mật khẩu, tìm kiếm chuyến bay, xem kết quả tìm kiếm, lựa chọn ghế ngồi, nhập thông tin hành khách, xác nhận đặt vé, theo dõi lịch sử đặt vé, xem chi tiết booking, quản lý hồ sơ cá nhân và nhận thông báo trong ứng dụng.

Về mặt kỹ thuật, đề tài tập trung nghiên cứu cách tổ chức ứng dụng Android theo hướng client-side, bao gồm thiết kế giao diện người dùng, điều hướng giữa các màn hình, quản lý trạng thái đăng nhập, lưu trữ dữ liệu cục bộ, trao đổi dữ liệu với máy chủ thông qua API và xử lý các luồng nghiệp vụ chính liên quan đến quá trình đặt vé. Trọng tâm của đề tài là bảo đảm người dùng có thể thực hiện trọn vẹn

quy trình từ tìm kiếm chuyến bay cho đến hoàn tất booking ngay trên ứng dụng di động.

Đề tài chỉ xem xét các chức năng ở mức người dùng sử dụng hệ thống, tức là phía ứng dụng Android phục vụ khách hàng. Các nội dung liên quan đến backend, cơ sở dữ liệu, hạ tầng triển khai hay hệ thống quản trị chỉ được đề cập ở mức hỗ trợ cho việc làm rõ luồng hoạt động của ứng dụng, không phải là đối tượng nghiên cứu chính. Nói cách khác, đề tài không đi sâu vào việc phân tích chi tiết kiến trúc server, thiết kế cơ sở dữ liệu vật lý, tối ưu hiệu năng backend hay các nghiệp vụ quản trị nội bộ của hãng hàng không.

Ngoài ra, phạm vi nghiên cứu của đề tài cũng được giới hạn trong mức độ phù hợp với một đồ án phát triển ứng dụng di động. Vì vậy, đề tài chưa đặt trọng tâm vào các chức năng mở rộng như thanh toán trực tuyến qua cổng thanh toán thực tế, check-in điện tử hoàn chỉnh, tích điểm khách hàng thân thiết, tích hợp bản đồ thời gian thực hoặc các cơ chế bảo mật và vận hành ở mức hệ thống thương mại lớn. Những nội dung này có thể được xem là hướng phát triển tiếp theo của đề tài trong tương lai.

1.3.2. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là người dùng cuối của ứng dụng đặt vé máy bay trên nền tảng Android và các thành phần phần mềm trực tiếp tham gia vào quá trình vận hành của ứng dụng ở phía client.

Về mặt nghiệp vụ, đối tượng nghiên cứu bao gồm ba nhóm chính. Nhóm thứ nhất là khách chưa đăng nhập, tức những người mới tiếp cận ứng dụng và sử dụng các chức năng ban đầu như xem màn hình chào, đăng ký tài khoản, đăng nhập, quên mật khẩu và xác thực để thiết lập quyền truy cập vào hệ thống. Nhóm thứ hai là người dùng đã đăng nhập, là những người có thể sử dụng đầy đủ các chức năng chính của ứng dụng như tìm kiếm chuyến bay, xem kết quả tìm kiếm, chọn ghế ngồi, nhập thông tin hành khách, đặt vé, xem lịch sử đặt chỗ, xem chi tiết vé, quản lý hồ sơ cá nhân và nhận thông báo từ hệ thống. Nhóm thứ ba là quá trình tương tác giữa ứng dụng Android với backend thông qua REST API, thể hiện qua việc

gửi yêu cầu, nhận phản hồi và xử lý dữ liệu phục vụ cho toàn bộ luồng nghiệp vụ của người dùng.

Về mặt kỹ thuật, đối tượng nghiên cứu còn bao gồm các nhóm lớp và thành phần chính trong mã nguồn client Android. Trước hết là nhóm Activity, đóng vai trò tổ chức giao diện, xử lý sự kiện người dùng và điều hướng giữa các màn hình chức năng. Tiếp theo là nhóm API provider và endpoint interface, chịu trách nhiệm cấu hình kết nối mạng, định nghĩa các API được sử dụng và thực hiện giao tiếp với backend. Bên cạnh đó, báo cáo cũng nghiên cứu nhóm DTO/model, là các lớp dữ liệu dùng để trao đổi thông tin giữa client và server, cũng như các đối tượng nội bộ phục vụ cho việc hiển thị và xử lý nghiệp vụ trong ứng dụng. Ngoài ra, đối tượng nghiên cứu còn bao gồm fragment xác thực OTP và các thành phần hỗ trợ khác có liên quan trực tiếp đến luồng xác thực và khôi phục tài khoản người dùng.

1.3.3. Phương pháp thực hiện

Để hoàn thành đề tài xây dựng ứng dụng đặt vé máy bay trên nền tảng Android, sẽ triển khai theo phương pháp phát triển phần mềm theo từng giai đoạn, từ phân tích yêu cầu, thiết kế hệ thống, xây dựng giao diện, lập trình chức năng cho đến kiểm thử và hoàn thiện sản phẩm. Quá trình thực hiện được tiến hành theo các bước chính sau:

Thứ nhất, khảo sát và phân tích yêu cầu hệ thống.

Nhóm tiến hành xác định bài toán thực tế của hệ thống đặt vé máy bay, từ đó phân tích các chức năng cần thiết cho phía người dùng như đăng ký, đăng nhập, quên mật khẩu, tìm kiếm chuyến bay, xem kết quả tìm kiếm, chọn ghế, nhập thông tin hành khách, đặt vé, xem lịch sử đặt vé, xem chi tiết vé và quản lý hồ sơ cá nhân. Ở giai đoạn này, nhóm tập trung làm rõ các yêu cầu chức năng và luồng xử lý chính để làm cơ sở cho việc thiết kế và triển khai ứng dụng.

Thứ hai, thiết kế tổng thể hệ thống.

Sau khi xác định được yêu cầu, nhóm tiến hành thiết kế tổng thể ứng dụng theo mô hình client–server, trong đó ứng dụng Android đóng vai trò phía client và backend cung cấp dữ liệu thông qua REST API. Ở bước này, nhóm xác định các màn hình chính của ứng dụng, mối liên hệ giữa các màn hình, dữ liệu cần trao đổi

với máy chủ và các đối tượng dữ liệu sử dụng trong hệ thống. Đồng thời, nhóm xây dựng các sơ đồ như use case, activity diagram, sequence diagram và sơ đồ điều hướng để mô tả trực quan cách hệ thống hoạt động.

Thứ ba, thiết kế giao diện người dùng.

Nhóm thực hiện thiết kế giao diện cho các màn hình chính của ứng dụng theo hướng trực quan, dễ sử dụng và phù hợp với đặc điểm của ứng dụng di động. Các màn hình được bố trí theo đúng luồng thao tác của người dùng, từ màn hình chào, đăng nhập, đăng ký cho đến các màn hình tìm kiếm chuyến bay, chọn ghế, đặt vé, lịch sử đặt vé và hồ sơ cá nhân. Việc thiết kế giao diện được thực hiện trước khi lập trình nhằm giúp thống nhất bố cục và định hướng xử lý cho từng chức năng.

Thứ tư, xây dựng ứng dụng Android.

Sau giai đoạn thiết kế, nhóm tiến hành lập trình ứng dụng trên nền tảng Android bằng ngôn ngữ Java. Mỗi chức năng được hiện thực hóa thông qua các Activity, Fragment, lớp DTO/model, lớp gọi API và các thành phần hỗ trợ khác. Trong quá trình này, nhóm tập trung triển khai các chức năng cốt lõi của đề tài như:

Quản lý tài khoản người dùng;

Tìm kiếm chuyến bay;

Hiển thị danh sách kết quả;

Chọn ghế ngồi;

Nhập thông tin hành khách;

Tạo đơn đặt vé;

Xem lịch sử đặt vé;

Xem chi tiết đặt chỗ;

Quản lý hồ sơ cá nhân;

Hiển thị thông báo trong ứng dụng.

Thứ năm, tích hợp API và xử lý dữ liệu.

Ứng dụng được kết nối với backend thông qua các REST API để lấy và gửi dữ liệu. Nhóm sử dụng Retrofit để thực hiện giao tiếp mạng và Gson để chuyển đổi dữ liệu JSON giữa client và server. Các thông tin cần thiết như dữ liệu người dùng đăng nhập, thông tin chuyến bay, dữ liệu ghế ngồi, lịch sử đặt vé và hồ sơ cá nhân được trao đổi thông qua các endpoint tương ứng. Bên cạnh đó, ứng dụng cũng sử dụng SharedPreferences để lưu một số dữ liệu cục bộ như trạng thái đăng nhập và thông tin cơ bản của người dùng.

Thứ sáu, kiểm thử và hiệu chỉnh hệ thống.

Sau khi hoàn thiện các chức năng chính, nhóm tiến hành kiểm thử từng màn hình và từng luồng nghiệp vụ nhằm phát hiện lỗi và điều chỉnh những điểm chưa hợp lý. Quá trình kiểm thử tập trung vào việc kiểm tra tính đúng đắn của dữ liệu nhập vào, khả năng điều hướng giữa các màn hình, phản hồi khi gọi API thành công hoặc thất bại, cũng như tính ổn định của các chức năng chính như đăng nhập, tìm kiếm chuyến bay, chọn ghế và đặt vé. Những lỗi phát sinh trong quá trình thử nghiệm được chỉnh sửa dần để nâng cao độ hoàn thiện của sản phẩm.

Thứ bảy, đánh giá và hoàn thiện đề tài.

Sau giai đoạn kiểm thử, nhóm tiến hành đánh giá kết quả đạt được so với mục tiêu ban đầu của đề tài. Trên cơ sở đó, nhóm hoàn thiện lại giao diện, tối ưu một số luồng xử lý, chuẩn hóa cấu trúc chức năng và tổng hợp kết quả để xây dựng báo cáo đồ án. Đồng thời, nhóm cũng xác định những điểm còn hạn chế của hệ thống và đề xuất hướng phát triển mở rộng trong tương lai.

Đề tài được thực hiện theo quy trình từ phân tích yêu cầu → thiết kế hệ thống → thiết kế giao diện → lập trình chức năng → tích hợp API → kiểm thử và hoàn thiện. Phương pháp này giúp nhóm triển khai đề tài một cách có hệ thống, bảo đảm ứng dụng được xây dựng đúng hướng, bám sát mục tiêu đặt ra và phù hợp với phạm vi của một đồ án phát triển ứng dụng di động.

Bảng 1.1. Phân tích nhóm tác nhân và nhu cầu sử dụng

Nhóm tác nhân	Đặc điểm	Nhu cầu trọng tâm	Giá trị mà ứng dụng cung cấp
Khách chưa đăng nhập	Mới mở ứng dụng, chưa có phiên người dùng	Xem màn hình chào, đi vào đăng nhập hoặc đăng ký,	Tạo điểm vào rõ ràng cho chu trình xác thực và

Nhóm tác nhân	Đặc điểm	Nhu cầu trọng tâm	Giá trị mà ứng dụng cung cấp
		khôi phục mật khẩu khi quên thông tin đăng nhập	chuyển đổi sang người dùng hợp lệ.
Người dùng đã đăng nhập	Đã có user_id và cờ is_logged_in trong SharedPreferences	Tìm chuyến bay, chọn ghế, tạo booking, xem lịch sử, xem hồ sơ, nhận thông báo	Đáp ứng luồng nghiệp vụ đặt vé trọn vẹn trên thiết bị di động.
Backend REST API	Đóng vai trò cung cấp dữ liệu và xử lý nghiệp vụ phía server	Xác thực, tìm kiếm chuyến bay, quản lý seat map, tạo booking, trả lịch sử, hồ sơ, OTP	Tạo nguồn dữ liệu thống nhất cho client Android và đảm bảo tính nhất quán nghiệp vụ.

1.4. Các tác nhân và nhu cầu sử dụng

1.4.1. Khách chưa đăng nhập

Khách chưa đăng nhập là người dùng mới hoặc người dùng chưa xác thực tài khoản trong hệ thống. Đây là nhóm tác nhân đầu tiên tiếp cận ứng dụng và thường chỉ được phép sử dụng các chức năng cơ bản liên quan đến truy cập ban đầu.

Nhu cầu sử dụng của nhóm này bao gồm:

Xem màn hình chào và tiếp cận giao diện ban đầu của ứng dụng;

Đăng ký tài khoản mới để tham gia hệ thống;

Đăng nhập bằng tài khoản đã có;

Sử dụng chức năng quên mật khẩu khi không thể đăng nhập;

Xác thực otp và đặt lại mật khẩu khi cần khôi phục tài khoản.

Nhóm tác nhân này có vai trò quan trọng vì đây là điểm bắt đầu của toàn bộ quá trình sử dụng hệ thống. Ứng dụng cần bảo đảm các thao tác đăng ký, đăng nhập và khôi phục mật khẩu được thực hiện rõ ràng, dễ hiểu và thuận tiện.

1.4.2. Người dùng đã đăng nhập

Người dùng đã đăng nhập là tác nhân chính của hệ thống. Đây là nhóm người dùng đã được xác thực và có quyền truy cập vào hầu hết các chức năng nghiệp vụ của ứng dụng đặt vé máy bay.

Nhu cầu sử dụng của nhóm này bao gồm:

Tìm kiếm chuyến bay theo sân bay đi, sân bay đến, ngày đi, ngày về, số lượng hành khách và hạng ghế;

Xem danh sách kết quả chuyến bay phù hợp;

Lựa chọn một chuyến bay mong muốn;

Xem và chọn ghế ngồi trên sơ đồ ghế;

Nhập thông tin hành khách tương ứng với từng ghế đã chọn;

Xác nhận đặt vé và nhận thông báo khi đặt vé thành công;

Xem lịch sử đặt vé;

Xem chi tiết từng đơn đặt chỗ;

Quản lý thông tin hồ sơ cá nhân;

Cập nhật thông tin cá nhân hoặc thay đổi một số thông tin tài khoản;

Đăng xuất khỏi hệ thống khi kết thúc phiên làm việc.

Đây là tác nhân trung tâm của toàn bộ ứng dụng, do đó phần lớn các màn hình và luồng xử lý của hệ thống đều được xây dựng xoay quanh nhu cầu của nhóm người dùng này.

1.4.3. Hệ thống backend REST API

Mặc dù không phải là người dùng theo nghĩa truyền thống, backend REST API vẫn là một tác nhân quan trọng trong quá trình vận hành của hệ thống, vì ứng dụng Android không tự xử lý toàn bộ dữ liệu mà phải tương tác với máy chủ để nhận và gửi thông tin.

Nhu cầu và vai trò của tác nhân này trong hệ thống bao gồm:

Tiếp nhận yêu cầu đăng ký, đăng nhập, quên mật khẩu, xác thực OTP và đặt lại mật khẩu;

Cung cấp dữ liệu hồ sơ người dùng;

Cung cấp danh sách sân bay và dữ liệu chuyến bay phục vụ tìm kiếm;

Cung cấp bản đồ ghế và trạng thái ghế;

Tiếp nhận dữ liệu đặt vé từ client;

Trả về kết quả xác nhận đặt vé;

Cung cấp lịch sử đặt vé, chi tiết đặt vé và dữ liệu thông báo;

Phản hồi các trạng thái lỗi hoặc kết quả xử lý để ứng dụng Android hiển thị cho người dùng.

Có thể xem backend là tác nhân hỗ trợ trực tiếp cho mọi chức năng chính của ứng dụng. Ứng dụng Android đóng vai trò giao diện và xử lý tương tác người dùng, trong khi backend đảm nhận phần dữ liệu và nghiệp vụ phía máy chủ.

1.4.4. Nhóm tác nhân quản trị ở mức suy ra từ API

Từ các interface API trong hệ thống có thể nhận thấy backend còn hỗ trợ một số chức năng liên quan đến quản trị như quản lý chuyến bay, quản lý booking và cập nhật trạng thái đơn đặt vé. Tuy nhiên, trong phạm vi repo Android hiện tại, chưa thấy triển khai giao diện client riêng cho tác nhân quản trị. Vì vậy, nhóm này chỉ được xem là tác nhân tồn tại ở mức hệ thống tổng thể, chưa phải là tác nhân trực tiếp của ứng dụng Android phía người dùng.

Nhu cầu của nhóm này, nếu xét ở mức tổng thể hệ thống, có thể bao gồm:

Quản lý chuyến bay;

Quản lý danh sách booking;

Cập nhật trạng thái đơn đặt vé;

Xử lý các thao tác quản trị khác liên quan đến dữ liệu vận hành.

Do không có màn hình quản trị tương ứng trong client Android, phần này chỉ được ghi nhận như một tác nhân phụ trợ được suy ra từ thiết kế API, không phải là trọng tâm của đề tài.

1.5. Yêu cầu chức năng

Bảng 1.2. Tổng hợp yêu cầu chức năng chính

Mã	Tác nhân	Yêu cầu	Hiện thực trong dự án
F1	Khách chưa đăng nhập	Mở màn hình chào và chuyển sang Login	MainActivity

Mã	Tác nhân	Yêu cầu	Hiện thực trong dự án
F2	Khách chưa đăng nhập	Đăng nhập bằng email/username và mật khẩu	Login
F3	Khách chưa đăng nhập	Đăng ký tài khoản mới, kiểm tra dữ liệu đầu vào	Signup
F4	Khách chưa đăng nhập	Gửi OTP quên mật khẩu và đặt lại mật khẩu	ForgotPasswordActivity, OtpVerificationDialogFragment, ResetPasswordActivity
F5	Người dùng	Hiển thị menu trung tâm sau đăng nhập	MainMenuActivity
F6	Người dùng	Tải danh sách sân bay và nhập tiêu chí tìm kiếm chuyến bay	SearchFlightActivity
F7	Người dùng	Gửi truy vấn tìm kiếm nâng cao tới backend	SearchFlightActivity
F8	Người dùng	Hiển thị danh sách chuyến bay và chọn một chuyến	FlightResultsActivity
F9	Người dùng	Tải seat map, chọn ghế, nhập thông tin hành khách cho từng ghế	ChooseSeatsActivity
F10	Người dùng	Tạo booking và nhận bookingReference	BookingFormActivity
F11	Người dùng	Nhận thông báo cục bộ khi booking thành công	BookingFormActivity
F12	Người dùng	Xem lịch sử booking, lọc và mở chi tiết booking	BookingHistoryActivity
F13	Người dùng	Mở màn hình thanh toán / đặt lại từ lịch sử booking	PayActivity qua BookingHistoryActivity
F14	Người dùng	Xem hồ sơ cá nhân, cập nhật thông tin và đổi mật khẩu	Profile, EditProfileActivity, ChangePasswordActivity
F15	Người dùng	Xem badge thông báo chưa đọc và điều hướng đến màn thông báo	MainMenuActivity, NotificationActivity
F16	Người dùng	Tương tác với AI Planner để hỗ trợ lập kế hoạch hoặc tư vấn	AIPlannerActivity

1.6. Yêu cầu phi chức năng

Bảng 1.3. Tổng hợp yêu cầu phi chức năng

Mã	Thuộc tính	Mô tả	Biểu hiện trong repo
NF1	Tính dễ dùng	Giao diện cần đủ đơn giản để người dùng hoàn thành luồng đặt vé trên điện thoại	Các màn hình sử dụng bố cục ít trường, phân từng bước rõ ràng.
NF2	Tính phản hồi	Thao tác mạng phải có phản hồi thành công/thất bại rõ ràng	Repo dùng Toast, đổi trạng thái nút, ProgressBar ở một số màn hình.
NF3	Tính nhất quán dữ liệu	Phiên người dùng và dữ liệu điều hướng phải đồng bộ	Ứng dụng lưu user_prefs và truyền Intent extras giữa các Activity.
NF4	Tính bảo mật	Không lưu mật khẩu thô; bảo vệ thông tin đăng nhập và dữ liệu mạng	Client không lưu mật khẩu nhưng vẫn cần cải thiện base URL và HTTPS.
NF5	Khả năng bảo trì	Mã nguồn cần đủ rõ để đọc, sửa, mở rộng	Tên lớp khá trực quan, song kiến trúc Activity-centric làm giảm khả năng mở rộng.
NF6	Tính tương thích	Ứng dụng phải hoạt động được trên Android hiện đại	Repo sử dụng AndroidX/AppCompat/Material, phù hợp định hướng thiết bị hiện nay.
NF7	Khả năng phục hồi lỗi	Màn hình phải xử lý được lỗi API, dữ liệu rỗng, phiên hết hạn	Nhiều Activity đã có nhánh handleErrorResponse và redirectToLogin.

CHƯƠNG 2.

CƠ SỞ LÝ THUYẾT, CÔNG NGHỆ VÀ PHÂN TÍCH DỰ ÁN

2.1. Kiến trúc ứng dụng Android theo mô hình client-server

Ứng dụng có thể được xem là một client Android hoạt động phía trước một hệ thống REST API. Mọi dữ liệu nghiệp vụ quan trọng – như xác thực tài khoản, danh sách sân bay, kết quả tìm chuyến bay, seat map, booking history hay hồ sơ người dùng – đều được lấy từ server thông qua các interface Retrofit. Cách tiếp cận này phù hợp với mô hình phân lớp phổ biến trong ứng dụng di động hiện nay: client tập trung vào giao diện và điều hướng, còn backend phụ trách nghiệp vụ và dữ liệu.

So với ứng dụng desktop đơn lẻ, kiến trúc client-server mang lại lợi ích rõ ràng về tính đồng bộ. Khi dữ liệu chuyến bay, trạng thái ghế hoặc trạng thái booking được cập nhật ở phía server, toàn bộ thiết bị sử dụng cùng API có thể nhìn thấy cùng một kết quả. Điều này đặc biệt quan trọng với bài toán đặt vé máy bay, nơi tính nhất quán số ghế và trạng thái booking cần được đảm bảo.

Tuy nhiên, kiến trúc client-server cũng làm nảy sinh những đòi hỏi nhất định đối với client, chẳng hạn như xử lý lỗi mạng, xử lý phiên hết hạn, serialize/deserialize dữ liệu và quản lý trạng thái điều hướng khi thông tin đến từ server có thể thay đổi theo thời gian thực. Những khía cạnh này đều hiện diện rõ trong các Activity đã phân tích của repo.

2.2. Android native Java và vòng đời Activity

Ứng dụng android sử dụng Java làm ngôn ngữ lập trình chính cho ứng dụng Android. Việc chọn Java giúp dự án phù hợp với kiến thức nền tảng của phần lớn sinh viên, đồng thời cho phép tận dụng hệ sinh thái AndroidX/AppCompat đã ổn định trong nhiều năm. Các lớp như MainActivity, Login, Signup, SearchFlightActivity, ChooseSeatsActivity hay Profile đều kế thừa AppCompatActivity, thể hiện hướng tiếp cận tiêu chuẩn khi xây dựng giao diện Android truyền thống.

Một điểm quan trọng trong phát triển Android là nắm được vòng đời Activity. Khi người dùng mở màn hình, onCreate() được gọi để setContentView, binding view, khởi tạo API service, gán event handler và khôi phục dữ liệu cần thiết. Khi người dùng quay

lại ứng dụng hoặc chuyển từ màn hình con về màn hình cha, onResume() thường được dùng để nạp lại trạng thái như kiểm tra cờ đăng nhập, tải badge thông báo hoặc gọi lại API hồ sơ. Repo thể hiện tương đối rõ mô thức này ở MainMenuActivity, FlightResultsActivity, ChooseSeatsActivity, BookingHistoryActivity và Profile. [1]

Nếu không hiểu đúng lifecycle, ứng dụng có thể gặp các lỗi như mất dữ liệu tạm, gọi API lặp vô ích, hoặc mở sai màn hình khi phiên đã hết hạn. Chính vì vậy, báo cáo này xem lifecycle là một nền tảng lý thuyết quan trọng để giải thích cách ứng dụng đang vận hành chứ không chỉ đơn thuần là khái niệm của Android framework.

2.3. Retrofit, Gson và mô hình REST API

Retrofit là thư viện được sử dụng để định nghĩa và gọi các endpoint REST. Trong repo, lớp RetrofitClient đóng vai trò tạo instance Retrofit, còn ApiServiceProvider đóng vai trò factory cung cấp từng interface như AuthApiEndpoint, BookingApiEndpoint, FlightApiEndpoint và các interface khác. Cách tổ chức này giúp phần Activity không cần trực tiếp biết cách tạo Retrofit mà chỉ cần gọi phương thức tương ứng từ provider. [2]

Gson được dùng để ánh xạ JSON thành object Java và ngược lại. Điều này đặc biệt hữu ích trong các luồng phức tạp như tìm kiếm chuyến bay, nơi SearchFlightActivity nhận về FlightSearchResultDto chứa outboundFlights và returnFlights, sau đó serialize thành chuỗi JSON để truyền qua Intent sang FlightResultsActivity. Việc serialize giúp giảm phụ thuộc vào Parcelable trong giai đoạn đồ án, đồng thời làm cho luồng truyền dữ liệu giữa Activity tương đối đơn giản. [3]

REST API trong dự án được phân thành ba nhóm lớn: nhóm xác thực – hồ sơ, nhóm booking và nhóm chuyến bay/seat map. Cách phân chia này cho thấy mỗi interface đảm nhiệm một miền nghiệp vụ riêng, từ đó thuận tiện cho việc mô tả contract API và đối chiếu chức năng trong quá trình phân tích mã nguồn [2][8].

2.4. SharedPreferences và quản lý phiên

SharedPreferences là cơ chế lưu trữ key-value cục bộ nhẹ của Android. Trong repo, SharedPreferences được dùng với tên user_prefs để lưu các thông tin quan trọng như user_id, username, user_name, user_email, user_phone, date_of_birth, gender,

total_bookings, is_logged_in và ở một vài thời điểm còn lưu cả flightId. Đây là lựa chọn thực tế, dễ cài đặt và phù hợp với quy mô của một đồ án sinh viên. [4]

SharedPreferences cung cấp cơ chế lưu trữ key-value đơn giản, truy cập nhanh và phù hợp với dữ liệu phiên có dung lượng nhỏ. Nhiều màn hình trong repo cùng đọc một tập khóa chung để duy trì thông tin người dùng giữa các bước điều hướng; đồng thời, cơ chế này cũng đặt ra yêu cầu phải chuẩn hóa tốt hơn việc đồng bộ phiên với backend trong các giai đoạn phát triển tiếp theo [4][8].

Về mặt thiết kế, SharedPreferences đóng vai trò như một trạng thái toàn cục tối giản của ứng dụng. Khi người dùng đăng nhập thành công, Login lưu dữ liệu vào user_prefs. Khi các Activity khác cần xác định danh tính hoặc trạng thái đăng nhập, chúng chỉ đọc từ user_prefs. Khi logout, MainMenuActivity và một số màn hình khác clear toàn bộ preferences. Cách tiếp cận này rất trực quan để mô tả trong báo cáo và cũng là một đặc trưng nổi bật của repo.

2.5. Vật liệu giao diện AndroidX / Material và thông báo cục bộ

Từ các import quan sát được trong mã nguồn, repo sử dụng AppCompat, AndroidX và Material Components như TextInputEditText để xây dựng giao diện nhập liệu. Điều này giúp các form đăng nhập, đăng ký, nhập thông tin hành khách hay đặt lại mật khẩu trở nên đồng bộ và thân thiện hơn với người dùng. Cách đặt tên view và bind view thủ công bằng findViewById cũng phản ánh phong cách Android truyền thống, dễ hiểu và phù hợp với người mới học. [5]

Ngoài giao diện nhập liệu, repo còn sử dụng RecyclerView để hiển thị danh sách chuyến bay ở FlightResultsActivity và danh sách booking ở BookingHistoryActivity. Cách tổ chức này phù hợp với đặc điểm dữ liệu dạng danh sách của ứng dụng và cho phép mở rộng số lượng phần tử hiển thị khi cần [6][8].

Một điểm đáng chú ý khác là cơ chế thông báo cục bộ. BookingFormActivity sau khi tạo booking thành công sẽ dựng NotificationChannel, NotificationCompat.Builder và PendingIntent để mở BookingDetailActivity khi người dùng chạm vào thông báo. Điều này tạo cảm giác hoàn chỉnh cho luồng nghiệp vụ: sau khi booking thành công, người dùng không chỉ nhận Toast mà còn có một dấu mốc hệ thống giúp truy cập lại chi tiết booking ở thời điểm sau. [7]

Bảng 2.1. Công nghệ và thư viện được sử dụng trong dự án

Công nghệ / cơ chế	Vai trò	Biểu hiện trong mã nguồn
Java / Android native	Ngôn ngữ và nền tảng UI chính	Toàn bộ Activity, xử lý sự kiện và điều hướng màn hình.
AppCompat / AndroidX	Khung Activity và tương thích thiết bị	AppCompatActivity, RecyclerView, BottomNavigationView, OnBackPressedDispatcher...
Material Components	Điều khiển giao diện nhập liệu	TextInputEditText và các thành phần Material.
Retrofit	Kết nối REST API	Được tạo bởi RetrofitClient và ApiServiceProvider.
Gson	Serialize / deserialize JSON	Dùng trong RetrofitClient và SearchFlightActivity.
SharedPreferences	Lưu trạng thái phiên cục bộ	Lưu user_prefs và flightId.
NotificationCompat	Thông báo nội bộ sau booking	Dùng ở BookingFormActivity.
Intent extras	Điều hướng và truyền dữ liệu giữa Activity	search_results_json, selectedSeatsList, bookingId, route...

2.6. Phân tích cấu trúc dự án

Dự án được xây dựng theo mô hình ứng dụng Android native, trong đó các chức năng được tổ chức chủ yếu theo từng Activity tương ứng với từng màn hình nghiệp vụ. Cấu trúc này giúp ứng dụng dễ triển khai và dễ theo dõi luồng xử lý từ đăng nhập, tìm chuyến bay, chọn ghế, đặt vé cho đến xem lịch sử và quản lý hồ sơ cá nhân.

Về mặt tổ chức, dự án có thể chia thành các nhóm chính gồm: nhóm giao diện và điều hướng màn hình, nhóm kết nối API, nhóm DTO/model và nhóm tài nguyên giao diện. Các màn hình như đăng nhập, đăng ký, tìm chuyến bay, chọn ghế, đặt vé, lịch sử booking, chi tiết booking và hồ sơ được tách riêng thành các lớp Activity, giúp phân chia chức năng khá rõ ràng. Bên cạnh đó, phần giao tiếp với backend được tách thành các lớp endpoint và provider, hỗ trợ gọi API và trao đổi dữ liệu giữa client với server.

Cấu trúc dự án phù hợp với phạm vi của một đồ án ứng dụng di động vì có sự phân chia tương đối rõ giữa giao diện, dữ liệu và kết nối mạng. Tuy nhiên, phần xử lý nghiệp vụ hiện vẫn tập trung nhiều trong các Activity, nên nếu mở rộng trong tương lai, dự án nên tách rõ hơn giữa giao diện, nghiệp vụ và dữ liệu để dễ bảo trì và nâng cấp.

Bảng 2.2. Cấu trúc package / nhóm lớp chính

Nhóm thành phần	Ví dụ file tiêu biểu	Mục đích
Activity xác thực	MainActivity, Login, Signup, ForgotPasswordActivity, ResetPasswordActivity	Cung cấp luồng tạo tài khoản, đăng nhập, khôi phục mật khẩu.
Activity nghiệp vụ đặt vé	SearchFlightActivity, FlightResultsActivity, ChooseSeatsActivity, BookingFormActivity	Tạo luồng tìm chuyến bay – chọn ghế – đặt vé.
Activity quản lý hậu giao dịch	BookingHistoryActivity, BookingDetailActivity, PayActivity	Cho phép xem lịch sử, chi tiết booking và thanh toán.
Activity tài khoản / tiện ích	MainMenuActivity, Profile, EditProfileActivity, ChangePasswordActivity, NotificationActivity, AIPlannerActivity	Trung tâm điều hướng, hồ sơ, thông báo và tính năng bổ trợ.
API layer	RetrofitClient, ApiServiceProvider, AuthApiEndpoint, BookingApiEndpoint, FlightApiEndpoint	Định nghĩa giao tiếp với backend.
DTO / model	UserProfileDto, AdvancedFlightSearchDto, SelectedSeatInfo, BookingResponseDto...	Đóng gói dữ liệu truyền/nhận.
Fragment hỗ trợ	OtpVerificationDialogFragment	Hỗ trợ nhập mã OTP trong luồng quên mật khẩu.

Bảng 2.3. Ánh xạ Activity – vai trò – nguồn dữ liệu

Activity	Nhóm vai trò	Nguồn dữ liệu chính	Trách nhiệm
MainActivity	Màn chào	Không dùng dữ liệu server	Khởi đầu ứng dụng, mở Login.
Login	Xác thực	AuthApiEndpoint.login, user_prefs	Validate tài khoản, lưu phiên.
Signup	Đăng ký	AuthApiEndpoint.register	Tạo tài khoản mới, kiểm tra dữ liệu đầu vào.
ForgotPasswordActivity	Quên mật khẩu	AuthApiEndpoint.forgotPassword	Gửi yêu cầu OTP và mở dialog xác thực.
ResetPasswordActivity	Đặt lại mật khẩu	AuthApiEndpoint.resetPassword	Đổi mật khẩu sau khi xác thực OTP.
MainMenuActivity	Trung tâm điều hướng	NotificationApiEndpoint.getUnreadCount, user_prefs	Điều hướng tới các màn nghiệp vụ chính.
SearchFlightActivity	Tìm kiếm chuyến bay	Airport API, Advanced Search API	Nhập tiêu chí tìm kiếm, gửi query tới server.
FlightResultsActivity	Kết quả chuyến bay	search_results_json	Hiển thị danh sách chuyến bay và mở chọn ghế.
ChooseSeatsActivity	Chọn ghế	FlightApiEndpoint.getSeatMap, user_prefs	Hiển thị seat map, nhập thông tin hành khách.
BookingFormActivity	Tạo booking	BookingApiEndpoint.createBooking	Tổng hợp ghế, gửi booking, phát thông báo.
BookingHistoryActivity	Lịch sử booking	BookingApiEndpoint.getBookingHistory	Tìm kiếm/lọc các booking đã phát sinh.

Activity	Nhóm vai trò	Nguồn dữ liệu chính	Trách nhiệm
Profile	Hồ sơ cá nhân	AuthApiEndpoint.getProfile	Hiển thị hồ sơ người dùng và thống kê cá nhân.

2.7. Môi trường phát triển và vận hành

Về môi trường phát triển, ứng dụng là một dự án Android Studio / Gradle. Phần code Java được đặt trong cấu trúc tiêu chuẩn app/src/main/java, tài nguyên giao diện trong app/src/main/res/layout và tệp cấu hình ứng dụng trong AndroidManifest.xml. Đây là cấu trúc tương thích với quy trình phát triển Android thông thường, thuận tiện cho việc build, debug và mở rộng. [8]

Về môi trường vận hành, RetrofitClient hiện chọn base URL dựa trên việc phát hiện emulator hay thiết bị thật. Khi chạy trên giả lập, ứng dụng ưu tiên địa chỉ 10.0.2.2; khi chạy trên máy thật, ứng dụng chuyển sang một địa chỉ IP LAN. Cách làm này giúp demo trong môi trường nội bộ thuận tiện, nhưng đồng thời cũng là một ràng buộc kỹ thuật vì ứng dụng chưa được cấu hình cho môi trường production hoặc chuyển đổi môi trường bằng file build config rõ ràng.

Để phục vụ việc biên dịch, kiểm thử và trình diễn ứng dụng, môi trường triển khai được mô tả theo hai nhóm: thông số có thể xác định từ cấu hình mã nguồn và thông số khuyến nghị cho quá trình cài đặt thực tế. Cách trình bày này giúp phân biệt rõ giữa giá trị được suy ra từ dự án với giá trị cần hoàn thiện thêm trong quá trình triển khai trên máy phát triển [8].

Bảng 2.4. Môi trường phát triển, vận hành và cấu hình triển khai

Nhóm thông tin	Giá trị / mô tả	Ghi chú
Môi trường phát triển	Android Studio mở dự án Android / Gradle	Sử dụng bản tương thích với Gradle Wrapper của dự án.
Ngôn ngữ và JDK	Java; khuyến nghị JDK 17	Phù hợp chuỗi build Android hiện nay.
Hệ thống build	Gradle và Android Gradle Plugin theo cấu hình dự án	Xác nhận lại trong gradle-wrapper.properties trước khi đóng gói bản nộp chính thức.
Android SDK tối thiểu	API 27 (Android 8.1)	Giá trị xác định từ cấu hình dự án theo phân tích mã nguồn.
Android SDK mục tiêu	API 35	Giá trị xác định từ cấu hình dự án theo phân tích mã nguồn.
Thiết bị kiểm thử	Android Emulator hoặc thiết bị thật	Thiết bị thật cần kết nối cùng mạng LAN với backend.
Backend REST API	Máy chủ cục bộ hoặc máy chủ trong cùng mạng nội bộ	Client Android lấy dữ liệu qua RetrofitClient.
Cấu hình base URL	Giả lập dùng 10.0.2.2; thiết bị thật dùng IP LAN của máy chủ	Cần cập nhật đúng địa chỉ trước khi chạy thử.

2.8. Về lựa chọn công nghệ

Nhìn tổng thể, stack công nghệ của dự án gồm Java, Activity, Retrofit, Gson, SharedPreferences và NotificationCompat. Tập hợp công nghệ này phản ánh cách tiếp cận quen thuộc trong các đồ án Android theo mô hình client–server, đồng thời đủ để hiện thực các luồng chức năng đã được mô tả ở các mục trước [2][3][4][7][8].

Nếu xét theo định hướng phát triển lâu dài, dự án có thể được nâng cấp sang Kotlin, ViewModel, LiveData hoặc StateFlow, kèm Repository và Room để cải thiện quản lý trạng thái. Dù vậy, việc bắt đầu bằng một kiến trúc đơn giản như repo hiện tại là phù hợp với mục tiêu học tập: người học có thể nắm chắc từng thành phần trước khi chuyển sang những kiến trúc cao hơn.

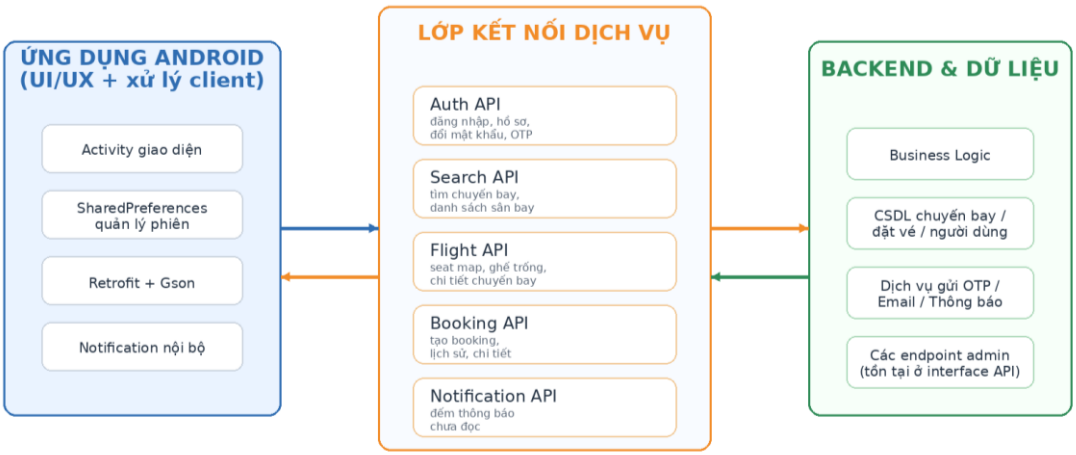
CHƯƠNG 3.
PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể của ứng dụng

Kiến trúc tổng thể của ứng dụng được mô tả theo ba lớp nhìn từ phía client: lớp giao diện người dùng, lớp kết nối dịch vụ và lớp backend dữ liệu. Lớp giao diện người dùng bao gồm toàn bộ Activity, fragment và adapter hiển thị cho người dùng. Lớp kết nối dịch vụ gồm RetrofitClient, ApiServiceProvider và các interface endpoint. Lớp backend dữ liệu không nằm trong repo nhưng được biểu diễn thông qua các contract API mà client đang sử dụng.

Điểm cốt lõi của kiến trúc này là sự tách biệt tương đối giữa giao diện và giao thức mạng. Dù chưa có Repository hoặc Service layer riêng phía client, repo vẫn cho thấy một quy ước đủ rõ: Activity chỉ lấy instance API từ ApiServiceProvider rồi gọi các phương thức phù hợp. Nhờ đó, tài liệu báo cáo có thể mô hình hóa lại hệ thống ở mức khái quát mà không cần phải dựng thêm các lớp không tồn tại.

Sơ đồ kiến trúc tổng thể dưới đây được thiết kế lại nhằm mô tả đúng quan hệ giữa các lớp hiện hữu trong dự án. Sơ đồ cho thấy luồng dữ liệu đi – về giữa Android app và backend, đồng thời nhấn mạnh vai trò của SharedPreferences như lớp lưu trạng thái phiên cục bộ và vai trò của Notification nội bộ như một thành phần trải nghiệm sau giao dịch.



Hình 3.1. Kiến trúc tổng thể của ứng dụng đặt vé máy bay trên Android.

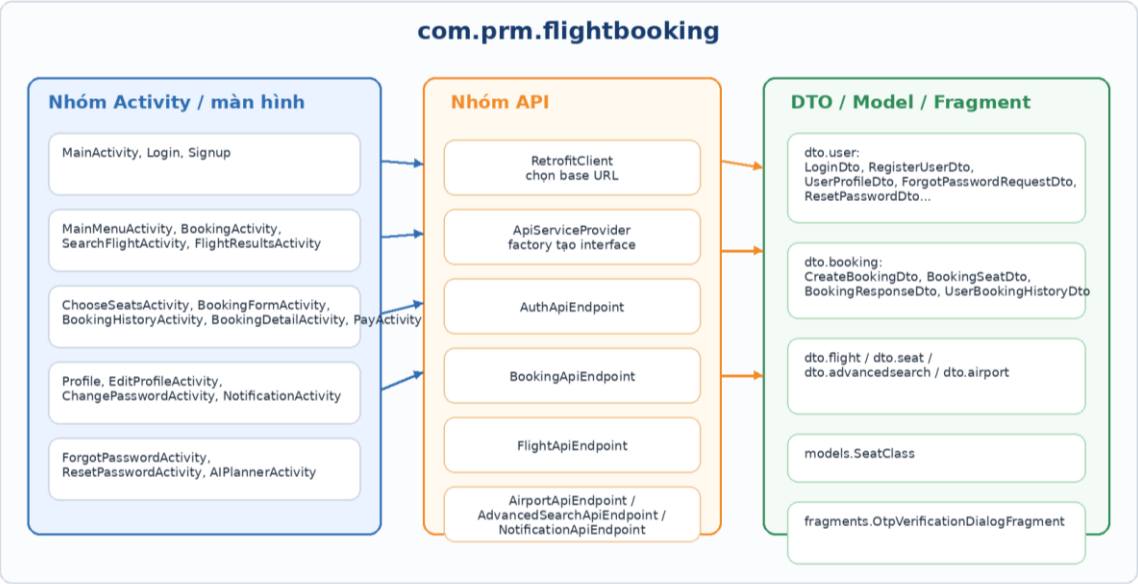
Sơ đồ trên đặc biệt hữu ích để giải thích vì sao báo cáo hiện tại không xây dựng một ERD cấp server như các báo cáo website truyền thống. Client Android không sở hữu cơ sở dữ liệu nghiệp vụ của hệ thống; những gì client biết là contract dữ liệu thông qua DTO. Vì vậy, việc mô tả kiến trúc theo hướng service contract phản ánh trung thực hơn thực tế dự án.

Ngoài ra, kiến trúc này còn cho thấy tiềm năng mở rộng: nếu backend thay đổi cách lưu dữ liệu nhưng vẫn giữ nguyên contract JSON, client hầu như không phải sửa quá nhiều. Ngược lại, nếu contract API thay đổi, toàn bộ tầng Activity và DTO liên quan sẽ bị ảnh hưởng. Đây là một trong những lý do cần tài liệu hóa kỹ lớp endpoint và mô hình dữ liệu ở những mục tiếp theo.

3.2. Phân rã module và cấu trúc package

Ở mức thiết kế chi tiết hơn, dự án có thể được phân rã thành bốn khối module chính. Khối thứ nhất là module giao diện nghiệp vụ, gồm các Activity phục vụ người dùng. Khối thứ hai là module API, giúp chuẩn hóa lời gọi mạng. Khối thứ ba là module DTO/model, dùng để chuyển đổi dữ liệu giữa JSON và object Java. Khối thứ tư là module phụ trợ như fragment OTP.

Cách phân rã này là cơ sở để viết tài liệu bảo trì. Khi một thành viên mới tham gia dự án, họ có thể nhanh chóng xác định vị trí cần đọc bằng cách chọn đúng nhóm module thay vì lần mò toàn bộ source tree. Trong báo cáo đồ án, việc nhóm theo module còn giúp trình bày mạch lạc hơn so với việc mô tả từng file rời rạc.



Hình 3.2. Sơ đồ module / package chính trong repo.

Cấu trúc module/package của dự án hiện được tổ chức chủ yếu theo hướng gom các thành phần trong cùng không gian tên com.prm.flightbooking. Thay vì tách riêng hoàn toàn theo các tầng như presentation, domain và data, dự án phân biệt vai trò của từng phần thông qua tên lớp và một số package con chuyên biệt. Cách tổ chức này thể hiện rõ đặc điểm của một ứng dụng Android được xây dựng theo hướng chức năng, trong đó mỗi màn hình hoặc nhóm nghiệp vụ được biểu diễn trực tiếp bởi các lớp tương ứng.

Bên trong cấu trúc đó, một số package con như api, dto, fragments và models được sử dụng để nhóm các thành phần có cùng vai trò. Package api chứa các lớp liên quan đến giao tiếp với backend, package dto chứa các đối tượng trao đổi dữ liệu, package fragments chứa các thành phần giao diện dạng fragment, còn package models biểu diễn một số đối tượng dữ liệu dùng trong ứng dụng. Nhờ cách đặt tên rõ ràng theo chức năng và nghiệp vụ, việc đọc mã nguồn và xác định trách nhiệm của từng lớp vẫn tương đối thuận tiện.

Cấu trúc package hiện tại cho thấy dự án được tổ chức theo hướng thực dụng, ưu tiên sự rõ ràng trong việc nhận diện chức năng hơn là phân tầng kiến trúc chặt chẽ. Đồng thời, việc đã hình thành các nhóm package con theo vai trò như api, dto,

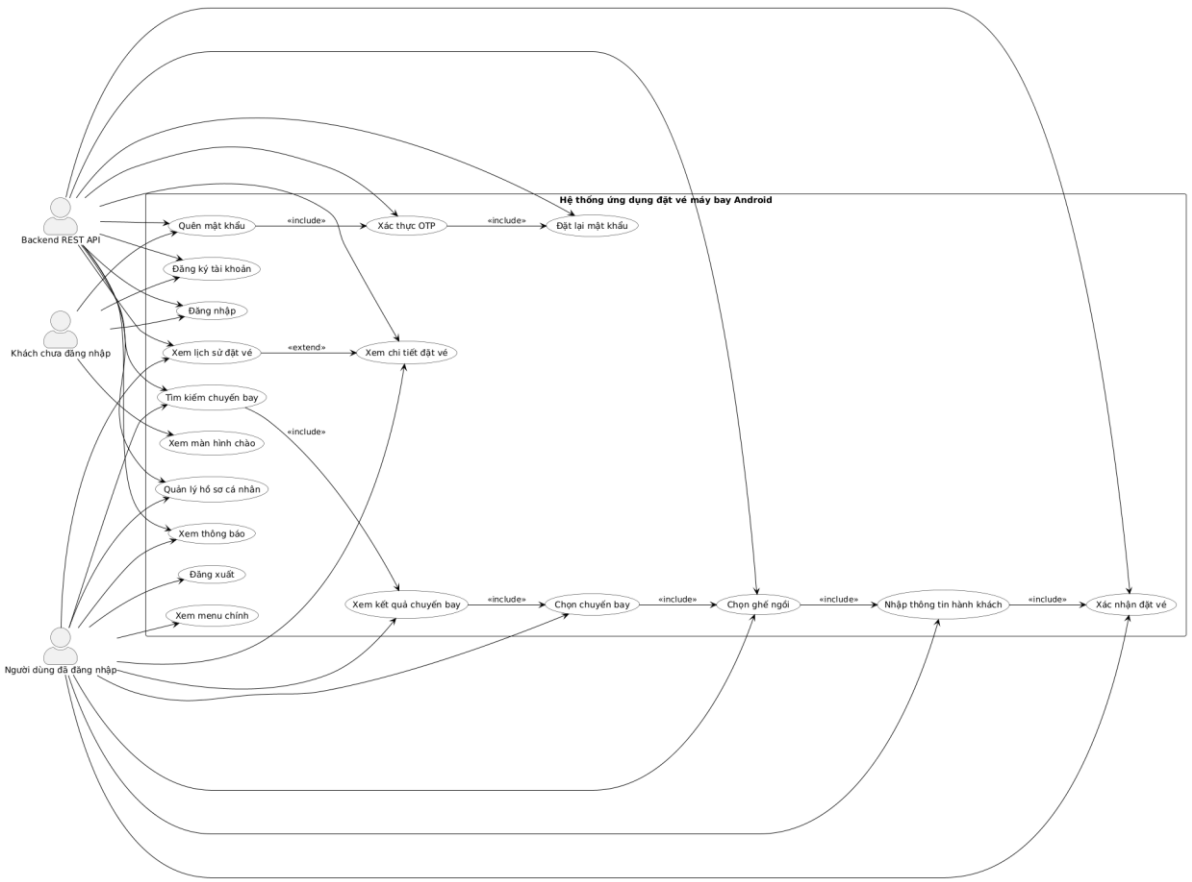
fragments, models cũng tạo ra một nền tảng tổ chức tương đối thống nhất cho toàn bộ mã nguồn.

3.3. Mô tả Use Case

Trong phạm vi client Android, Use Case được xác định theo những gì người dùng thực sự tương tác trên thiết bị. Điều này khác với các báo cáo mô tả toàn hệ thống hàng không, nơi Use Case có thể bao gồm cả quản trị chuyến bay, quản lý giá vé hoặc phê duyệt dữ liệu ở back-office. Ở repo hiện tại, trọng tâm là khách và hành khách cuối cùng.

3.3.1. Use Case tổng quát

Use Case tổng quát được nhóm theo ba mảng: xác thực – tài khoản, tìm kiếm – đặt vé, và hỗ trợ – theo dõi. Việc nhóm như vậy phản ánh đúng menu và đường đi thực tế từ MainMenuActivity, đồng thời thuận tiện cho kiểm thử chức năng về sau.

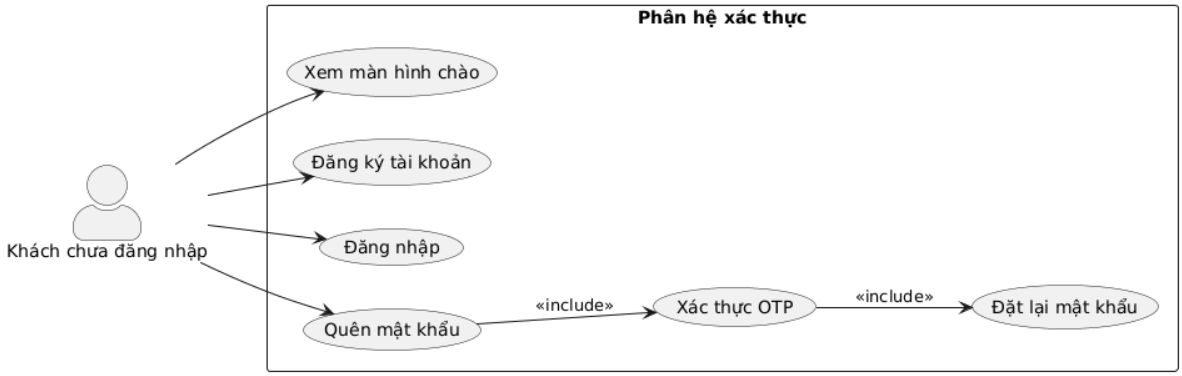


Hình 3.3. Use Case tổng quát của ứng dụng.

Ứng dụng hiện tại tối ưu cho luồng self-service. Người dùng tự thao tác hầu hết các bước mà không cần can thiệp thủ công từ nhân viên. Cơ chế này phù hợp với ứng

dụng mobile hiện đại và cũng là lý do vì sao các màn hình validate dữ liệu và hiển thị phản hồi đóng vai trò rất quan trọng.

3.3.2. Use Case đăng kí đăng nhập

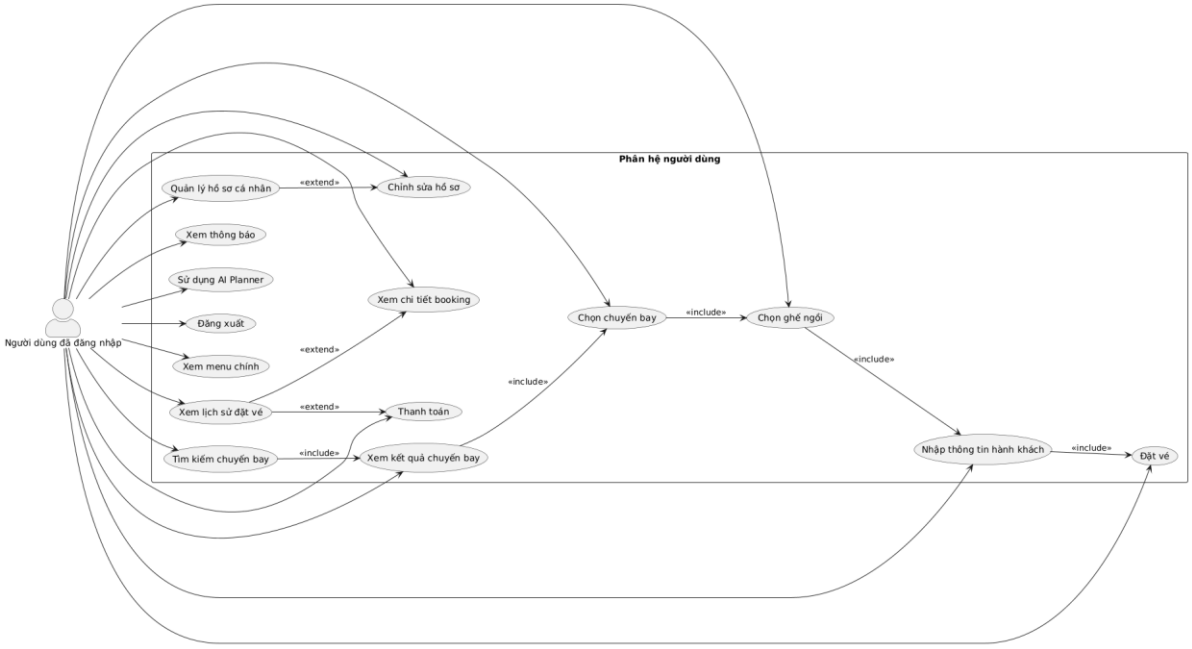


Hình 3.4. Use Case đăng kí đăng nhập.

Hình 3.4 mô tả các ca sử dụng thuộc phân hệ xác thực dành cho khách chưa đăng nhập. Trong giai đoạn đầu tiếp cận hệ thống, người dùng có thể thực hiện các chức năng cơ bản gồm xem màn hình chào, đăng ký tài khoản, đăng nhập và khôi phục mật khẩu. Đây là nhóm chức năng đóng vai trò khởi tạo quan hệ giữa người dùng và hệ thống, đồng thời thiết lập điều kiện truy cập vào các chức năng nghiệp vụ phía sau.

Trong sơ đồ, ca sử dụng Quên mật khẩu được mở rộng theo chuỗi xử lý gồm Xác thực OTP và Đặt lại mật khẩu thông qua quan hệ include. Điều này cho thấy quá trình khôi phục tài khoản không phải là một thao tác đơn lẻ mà được tổ chức thành nhiều bước liên tiếp nhằm bảo đảm tính an toàn và xác thực chính chủ. Nhìn tổng thể, sơ đồ phản ánh luồng xác thực được thiết kế tương đối đầy đủ, bao phủ cả trường hợp truy cập thông thường và trường hợp xử lý sự cố đăng nhập.

3.3.3. *của người dùng sau khi đăng nhập*

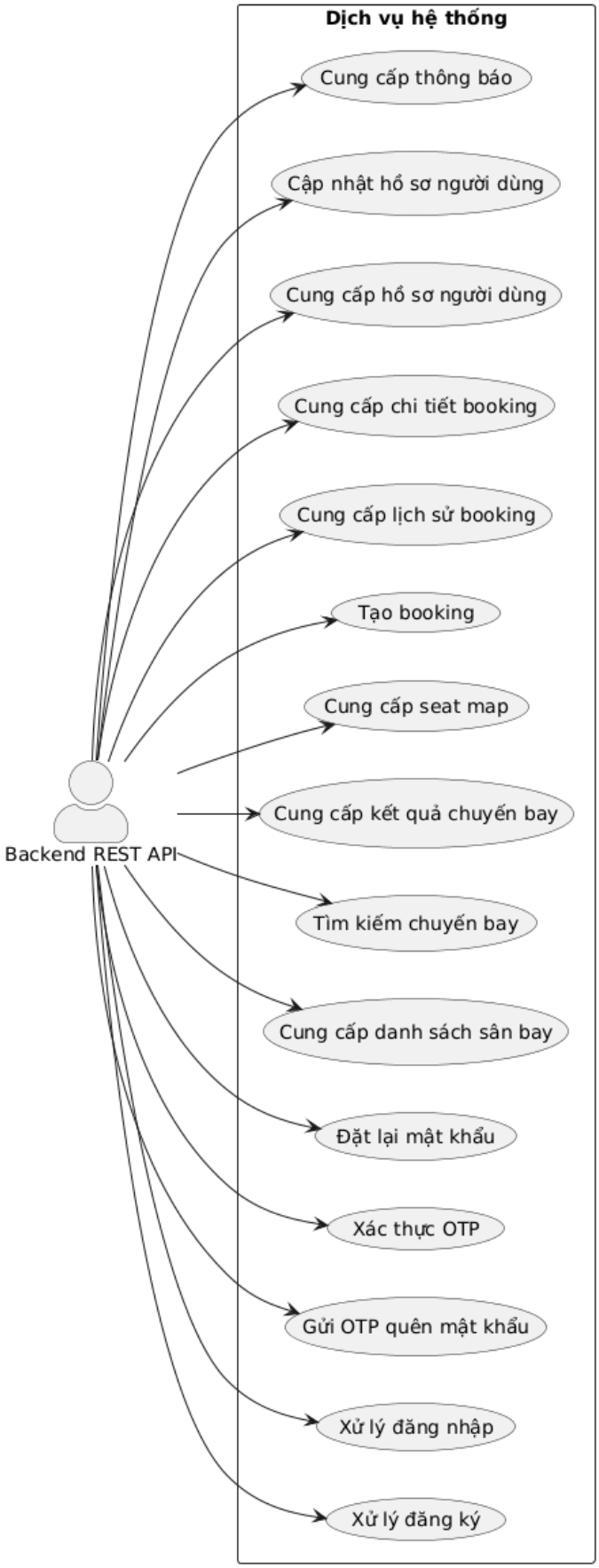


Hình 3.5. Use Case của người dùng sau khi đăng nhập

Hình 3.6 mô tả các ca sử dụng ở mức dịch vụ hệ thống, trong đó Backend REST API đóng vai trò là tác nhân trung tâm cung cấp dữ liệu và xử lý yêu cầu từ phía ứng dụng di động. Các ca sử dụng trong sơ đồ bao gồm xử lý đăng ký, xử lý đăng nhập, gửi OTP quên mật khẩu, xác thực OTP, đặt lại mật khẩu, cung cấp danh sách sân bay, tìm kiếm chuyến bay, cung cấp kết quả chuyến bay, cung cấp sơ đồ ghế, tạo booking, cung cấp lịch sử booking, cung cấp chi tiết booking, cung cấp hồ sơ người dùng, cập nhật hồ sơ người dùng và cung cấp thông báo.

Sơ đồ này cho thấy hệ thống backend không chỉ đảm nhiệm vai trò xác thực người dùng mà còn là tầng dịch vụ trung gian cho toàn bộ các nghiệp vụ chính của ứng dụng. Từ góc nhìn kiến trúc, các API đã được tổ chức theo hướng hỗ trợ đầy đủ cho vòng đời sử dụng của ứng dụng, bao gồm giai đoạn xác thực, tra cứu dữ liệu chuyến bay, phát sinh giao dịch đặt vé và quản lý dữ liệu người dùng sau giao dịch. Điều này phản ánh sự liên kết chặt chẽ giữa ứng dụng Android và backend thông qua mô hình REST API.

3.3.1. Use Case rest api



Hình 3.6. Use Case rest api.

Hình 3.5 mô tả các ca sử dụng thuộc **phân hệ người dùng** sau khi người dùng đã hoàn tất bước xác thực. Tác nhân trong sơ đồ là **người dùng đã đăng nhập**, có thể tương tác với hệ thống thông qua các chức năng như quản lý hồ sơ cá nhân, xem thông báo, sử dụng AI Planner, xem menu chính, xem lịch sử đặt vé, tìm kiếm chuyến bay và đăng xuất. Đây là nhóm chức năng phản ánh phần lớn các thao tác sử dụng thực tế của người dùng trong ứng dụng.

Bên trong sơ đồ, các nghiệp vụ chính được liên kết với nhau theo trình tự hợp lý. Từ chức năng **Tìm kiếm chuyến bay**, hệ thống dẫn đến **Xem kết quả chuyến bay**, tiếp theo là **Chọn chuyến bay**, **Chọn ghế ngồi**, **Nhập thông tin hành khách** và **Đặt vé**. Chuỗi này thể hiện luồng đặt vé là nghiệp vụ trung tâm của hệ thống. Bên cạnh đó, nhóm chức năng sau giao dịch cũng được mô tả thông qua các thao tác như **Xem lịch sử đặt vé**, **Xem chi tiết booking** và **Thanh toán**, cho thấy hệ thống không chỉ phục vụ thao tác tạo booking mà còn hỗ trợ tra cứu và xử lý giao dịch sau khi booking được hình thành.

Ngoài ra, sơ đồ còn thể hiện các chức năng hỗ trợ trải nghiệm người dùng như **Chỉnh sửa hồ sơ**, **Xem thông báo** và **Sử dụng AI Planner**. Điều này cho thấy ứng dụng đã được phát triển không chỉ theo hướng hoàn thành nghiệp vụ đặt vé cơ bản mà còn mở rộng sang các chức năng hỗ trợ quản lý tài khoản và tăng cường tương tác trong quá trình sử dụng.

3.4. Thiết kế dữ liệu phía client

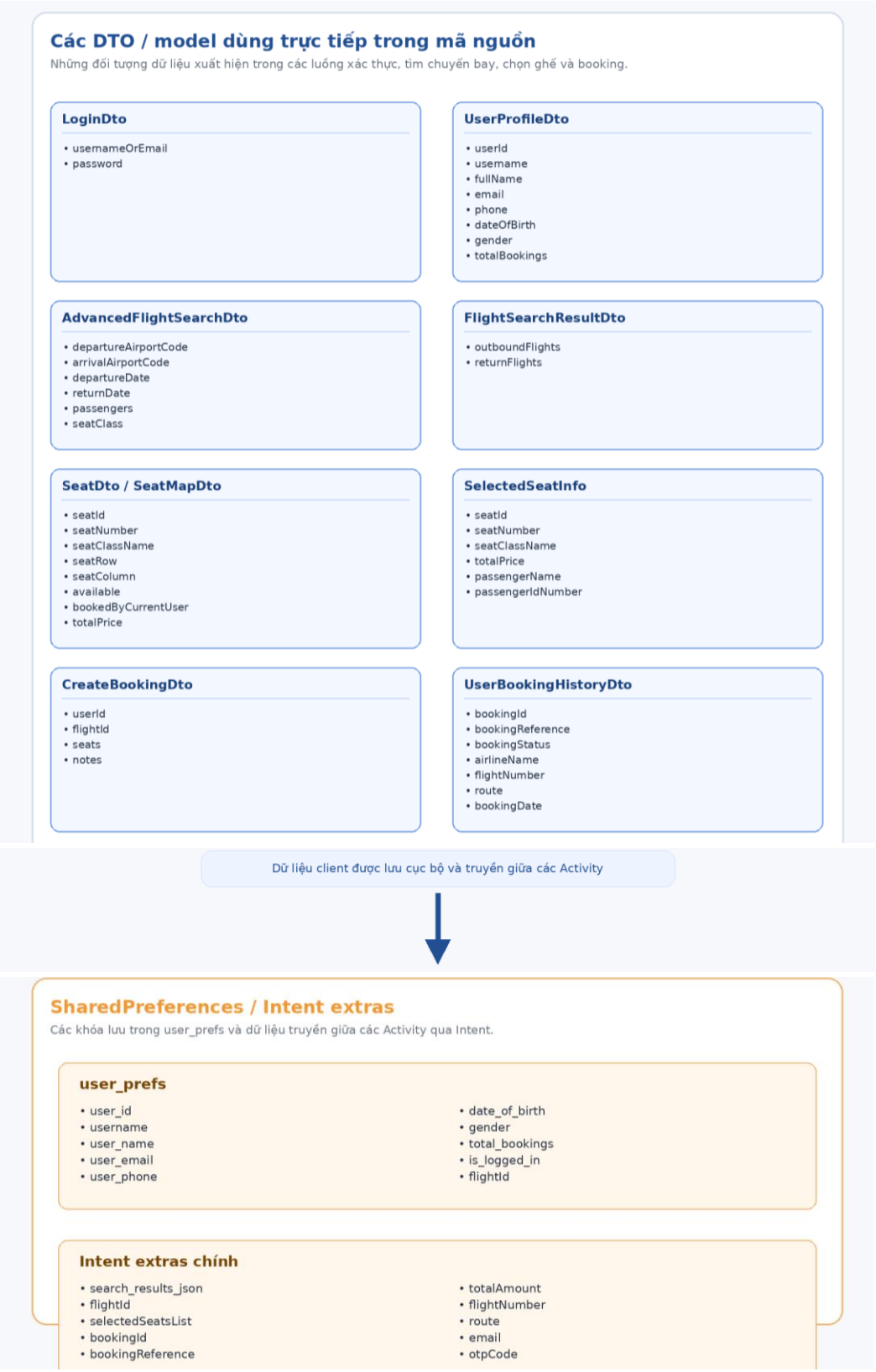
Dữ liệu phía client trong đề tài được thiết kế theo hướng phục vụ trực tiếp cho các chức năng của ứng dụng Android và hỗ trợ quá trình trao đổi dữ liệu với hệ thống backend. Khác với mô hình cơ sở dữ liệu quan hệ ở phía server, dữ liệu phía client chủ yếu được tổ chức dưới dạng các đối tượng dữ liệu tạm thời, dữ liệu trao đổi qua API và dữ liệu cục bộ cần lưu lại trong suốt quá trình sử dụng ứng dụng.

Về tổng thể, dữ liệu phía client có thể được chia thành ba nhóm chính. Nhóm thứ nhất là dữ liệu trao đổi với server, bao gồm các thông tin được gửi đi và nhận về trong quá trình đăng nhập, đăng ký, tìm kiếm chuyến bay, chọn ghế, đặt vé, xem lịch sử booking, lấy hồ sơ người dùng và nhận thông báo. Nhóm dữ liệu này giúp ứng dụng giao tiếp với backend để thực hiện các chức năng nghiệp vụ. Nhóm thứ hai là dữ liệu truyền giữa các màn hình, được sử dụng để chuyển tiếp thông tin trong luồng sử dụng, chẳng hạn như kết quả tìm kiếm chuyến bay, mã chuyến bay, danh sách ghế đã chọn hoặc mã booking. Nhóm thứ ba là dữ liệu lưu cục bộ, chủ yếu phục vụ việc duy trì trạng thái đăng nhập, lưu thông tin người dùng hiện tại và hỗ trợ các bước xử lý liên tiếp trong ứng dụng.

Thiết kế dữ liệu phía client được xây dựng theo nguyên tắc đơn giản, phù hợp với đặc điểm của ứng dụng di động. Ứng dụng không lưu trữ toàn bộ dữ liệu nghiệp vụ một cách lâu dài ở phía thiết bị, mà chủ yếu chỉ lưu lại những thông tin cần thiết để duy trì trải nghiệm người dùng, còn phần dữ liệu chính vẫn được lấy từ backend khi cần. Cách tổ chức này giúp giảm độ phức tạp trong quản lý dữ liệu trên thiết bị, đồng thời bảo đảm rằng thông tin hiển thị trong ứng dụng luôn bám sát dữ liệu từ hệ thống server.

Ngoài ra, thiết kế dữ liệu phía client còn gắn chặt với luồng điều hướng của ứng dụng. Ở mỗi bước của quá trình sử dụng, dữ liệu được tạo ra, xử lý và chuyển tiếp theo từng màn hình tương ứng, từ xác thực tài khoản đến tìm kiếm chuyến bay, chọn ghế, xác nhận đặt vé và theo dõi lịch sử giao dịch. Nhờ đó, dữ liệu phía client không chỉ đóng vai trò lưu trữ thông tin, mà còn là thành phần quan trọng giúp kết nối các bước trong toàn bộ quy trình nghiệp vụ của hệ thống.

Nhìn chung, thiết kế dữ liệu phía client trong đề tài được xây dựng theo hướng gọn nhẹ, linh hoạt và phù hợp với ứng dụng Android, trong đó tập trung vào việc hỗ trợ luồng xử lý của người dùng, duy trì trạng thái cần thiết trên thiết bị và đảm bảo khả năng tương tác hiệu quả với backend. Cách tiếp cận này phù hợp với phạm vi của một đề tài phát triển ứng dụng di động và đáp ứng tốt yêu cầu của hệ thống đặt vé máy bay ở phía người dùng cuối.



Hình 3.7. Thiết kế dữ liệu phía client: DTO, SharedPreferences và Intent extras.

Bảng 3.1. Mô hình dữ liệu người dùng và lưu trữ cục bộ

Thành phần	Trường dữ liệu tiêu biểu	Mục đích
UserProfileDto	userId, username, fullName, email, phone, dateOfBirth, gender, totalBookings, createdAt	Nhận từ API khi đăng nhập hoặc đọc hồ sơ người dùng.
SharedPreferences user_prefs	user_id, username, user_name, user_email, user_phone, date_of_birth, gender, total_bookings, is_logged_in, flightId	Lưu trạng thái phiên và một số ngữ cảnh điều hướng giữa các màn hình.
SelectedSeatInfo	seatId, seatNumber, seatClassName, totalPrice, passengerName, passengerIdNumber	Gom dữ liệu của từng ghế/hành khách để tạo booking.
Intent extras	search_results_json, selectedSeatsList, bookingId, bookingReference, totalAmount, route, email, otpCode	Truyền ngữ cảnh giữa các Activity mà không cần dùng storage lâu dài.

3.5. Thiết kế lớp API

Tầng API chia theo nghiệp vụ. AuthApiEndpoint chứa các phương thức liên quan đến đăng nhập, hồ sơ, đổi mật khẩu, quên mật khẩu và OTP. BookingApiEndpoint phụ trách tạo booking, lịch sử và chi tiết booking. FlightApiEndpoint phụ trách các thao tác liên quan đến chuyến bay và seat map. Các interface khác như AirportApiEndpoint, AdvancedSearchApiEndpoint và NotificationApiEndpoint hỗ trợ các nhu cầu chuyên biệt.

Thiết kế theo interface thuận tiện cho Retrofit và giúp báo cáo hóa cấu trúc API một cách trực tiếp. Từ các annotation @GET, @POST, @PUT, @DELETE trong interface, ta có thể lập bảng endpoint chính – một phần rất quan trọng để người đọc hiểu ứng dụng đang nói chuyện với backend như thế nào.

Bảng 3.2. Danh mục endpoint thuộc AuthApiEndpoint

Endpoint	Phương thức interface	Mục đích
POST Auth/register	register(RegisterUserDto)	Đăng ký tài khoản mới và nhận UserProfileDto.
POST Auth/login	login(LoginDto)	Xác thực người dùng, nhận hồ sơ để lưu phiên.
GET Auth/profile/{userId}	getProfile(userId)	Lấy hồ sơ cá nhân người dùng.
PUT Auth/profile/{userId}	updateProfile(userId, UpdateProfileDto)	Cập nhật hồ sơ cá nhân.
POST Auth/change-password/{userId}	changePassword(...)	Đổi mật khẩu khi đang đăng nhập.
POST Auth/delete-account/{userId}	deleteAccount(...)	Xóa tài khoản theo yêu cầu.
POST Auth/forgot-password	forgotPassword(ForgotPasswordRequestDto)	Gửi OTP về email để khôi phục mật khẩu.
POST Auth/verify-otp	verifyOtp(VerifyOtpDto)	Xác thực OTP.

Endpoint	Phương thức interface	Mục đích
POST Auth/reset-password	resetPassword(ResetPasswordDto)	Đặt lại mật khẩu mới sau xác thực OTP.

Bảng 3.3. Danh mục endpoint thuộc BookingApiEndpoint

Endpoint	Phương thức interface	Mục đích
POST Bookings	createBooking(CreateBookingDto)	Tạo booking mới và nhận bookingReference.
GET Bookings/user/{userId}	getUserBookings(userId)	Lấy danh sách booking theo user.
GET user/Bookings/{userId}/history	getBookingHistory(userId, page, pageSize)	Lấy lịch sử booking theo phân trang.
GET user/Bookings/{userId}/{bookingId}	getBookingDetail(userId, bookingId)	Lấy chi tiết một booking.
POST user/Bookings/{userId}/{bookingId}/cancel	cancelBookingUser(...)	Hủy booking phía người dùng.
GET admin/Bookings	getAllBookings(page, pageSize)	Khả năng quản trị từ backend, chưa thấy UI admin trên mobile client.
GET admin/Bookings/{bookingId}	getBookingById(bookingId)	Lấy chi tiết booking cho mục đích quản trị.
PUT admin/Bookings/{bookingId}/status	updateBookingStatus(...)	Đổi trạng thái booking ở tầng admin.
POST admin/Bookings/{bookingId}/cancel	cancelBookingAdmin(bookingId)	Hủy booking ở tầng admin.

Bảng 3.4. Danh mục endpoint thuộc FlightApiEndpoint

Endpoint	Phương thức interface	Mục đích
POST Flights/search	searchFlights(FlightSearchDto)	Tìm kiếm chuyến bay theo tiêu chí cơ bản.
GET Flights/{flightId}/seats	getSeatMap(flightId, userId)	Lấy bản đồ ghế cho một chuyến bay.
GET admin/Flights	getAllFlights(page, pageSize)	Danh sách chuyến bay phía admin.
GET admin/Flights/{flightId}	getFlightById(flightId)	Chi tiết chuyến bay phía admin.
POST admin/Flights	createFlight(CreateFlightDto)	Tạo chuyến bay mới.
PUT admin/Flights/{flightId}	updateFlight(...)	Cập nhật chuyến bay.
DELETE admin/Flights/{flightId}	deleteFlight(flightId)	Xóa chuyến bay.
POST admin/Flights/{flightId}/generate-seats	generateSeats(flightId)	Sinh danh sách ghế cho chuyến bay.

Bảng 3.5. Ma trận Intent extras trong ứng dụng

Điểm xuất phát → điểm đến	Intent extra	Ý nghĩa
SearchFlightActivity → FlightResultsActivity	search_results_json	Chuỗi JSON kết quả tìm kiếm, được Gson serialize từ FlightSearchResultDto.
FlightResultsActivity → ChooseSeatsActivity	flightId	Mã chuyến bay người dùng chọn.
ChooseSeatsActivity → BookingFormActivity	flightId, selectedSeatsList	Mã chuyến bay và danh sách ghế/hành khách.
ForgotPasswordActivity → ResetPasswordActivity	email, otpCode	Email và OTP đã xác thực.
BookingHistoryActivity → BookingDetailActivity	bookingId	Xem chi tiết booking từ lịch sử.

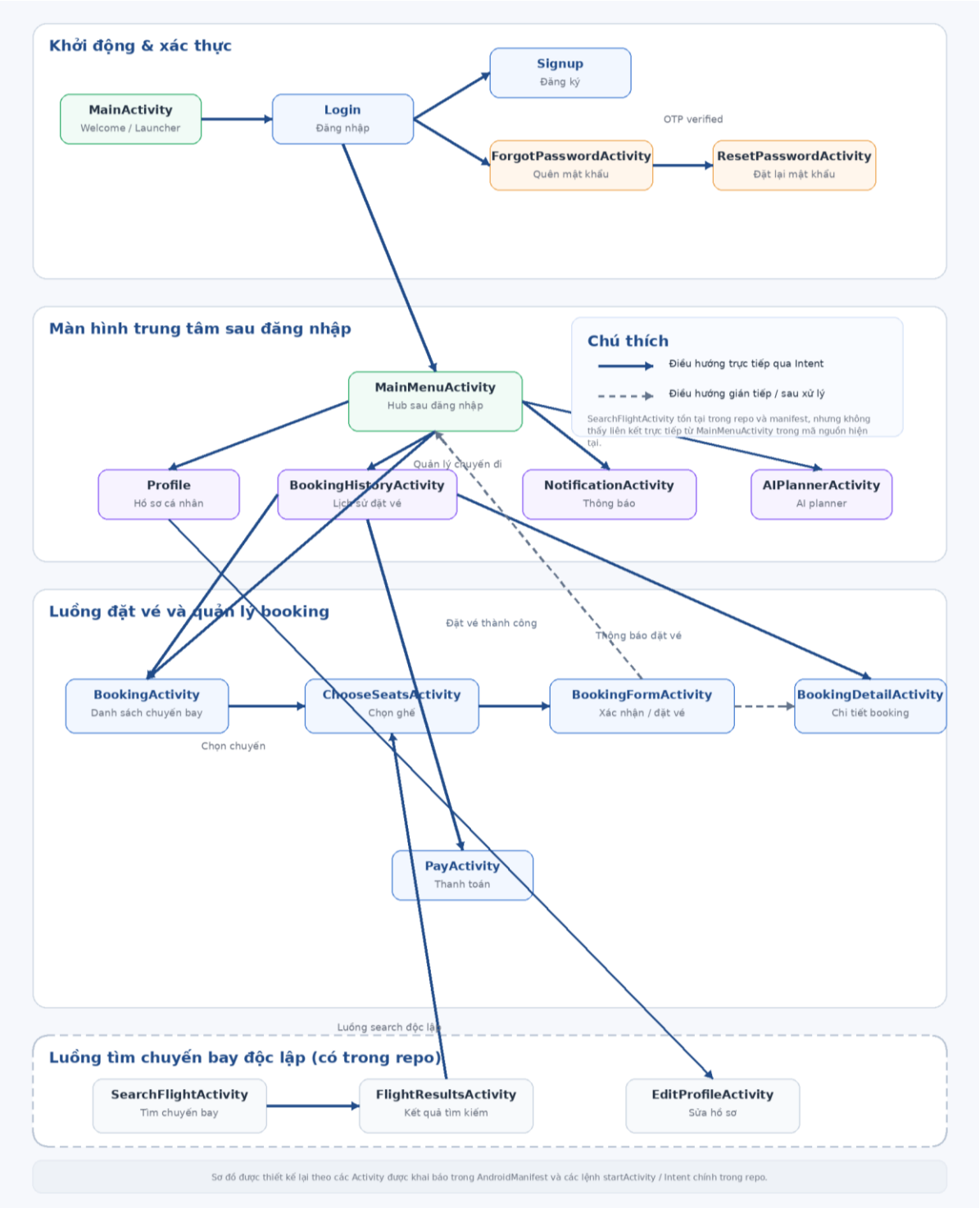
Điểm xuất phát → điểm đến	Intent extra	Ý nghĩa
BookingHistoryActivity → PayActivity	bookingId, bookingReference, totalAmount, flightNumber, route	Ngữ cảnh thanh toán hoặc xem lại thông tin chuyến bay.
BookingFormActivity → BookingDetailActivity (qua notification)	bookingId	Truy cập lại chi tiết booking từ thông báo hệ thống.

3.6. Thiết kế điều hướng màn hình

Thiết kế điều hướng màn hình của hệ thống được xây dựng theo hướng bám sát quy trình sử dụng thực tế của ứng dụng đặt vé máy bay. Người dùng đi qua nhiều bước liên tiếp như xác thực tài khoản, tìm kiếm chuyến bay, chọn ghế, đặt vé, xem lịch sử booking và quản lý hồ sơ cá nhân. Vì vậy, điều hướng màn hình đóng vai trò kết nối các chức năng riêng lẻ thành một luồng sử dụng thống nhất.

Trong hệ thống, mỗi chức năng chính thường tương ứng với một Activity riêng. Dữ liệu được truyền giữa các màn hình thông qua Intent extras, đồng thời một số thông tin cần duy trì như trạng thái đăng nhập hoặc thông tin người dùng được lưu bằng SharedPreferences. Cách tổ chức này giúp luồng xử lý của ứng dụng rõ ràng và thuận tiện cho việc theo dõi từng bước nghiệp vụ.

Thiết kế điều hướng màn hình của đề tài được tổ chức theo hướng tuần tự, rõ ràng và phù hợp với đặc điểm của ứng dụng Android. Mỗi màn hình đảm nhận một vai trò cụ thể, đồng thời liên kết với các màn hình khác để tạo thành quy trình sử dụng hoàn chỉnh cho người dùng.



Hình 3.8. Sơ đồ điều hướng giữa các Activity chính.

Trong sơ đồ, MainMenuActivity là nút trung tâm sau khi người dùng đăng nhập. Từ đây, ứng dụng cho phép đi tới các nhóm tính năng như booking, lịch sử, hồ sơ,

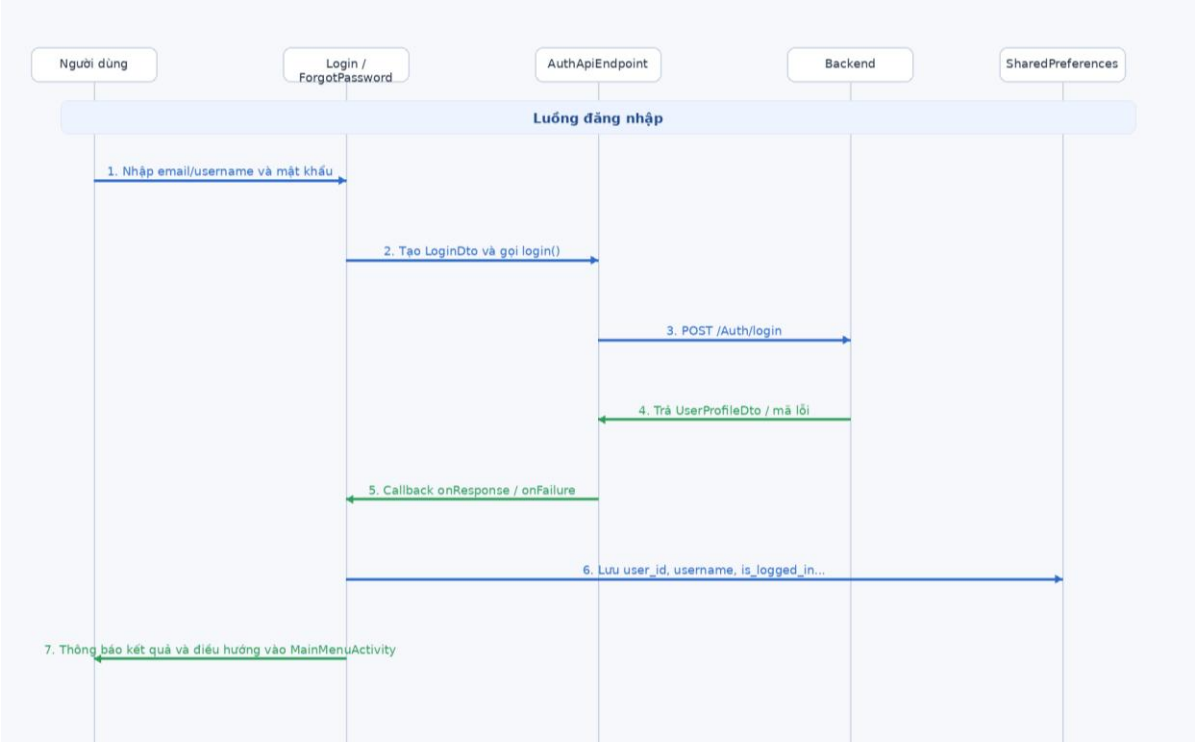
thông báo và AI Planner. Điều này phù hợp với quan sát từ mã nguồn khi MainMenuActivity chứa nhiều click handler và bottom navigation.

Đường điều hướng booking được xây dựng rõ nhất: SearchFlightActivity → FlightResultsActivity → ChooseSeatsActivity → BookingFormActivity. Đây là luồng nghiệp vụ cốt lõi và cũng là phần dễ thuyết trình nhất trong báo cáo hoặc lúc demo sản phẩm.

3.7. Thiết kế luồng xác thực, tạo tài khoản và khôi phục mật khẩu

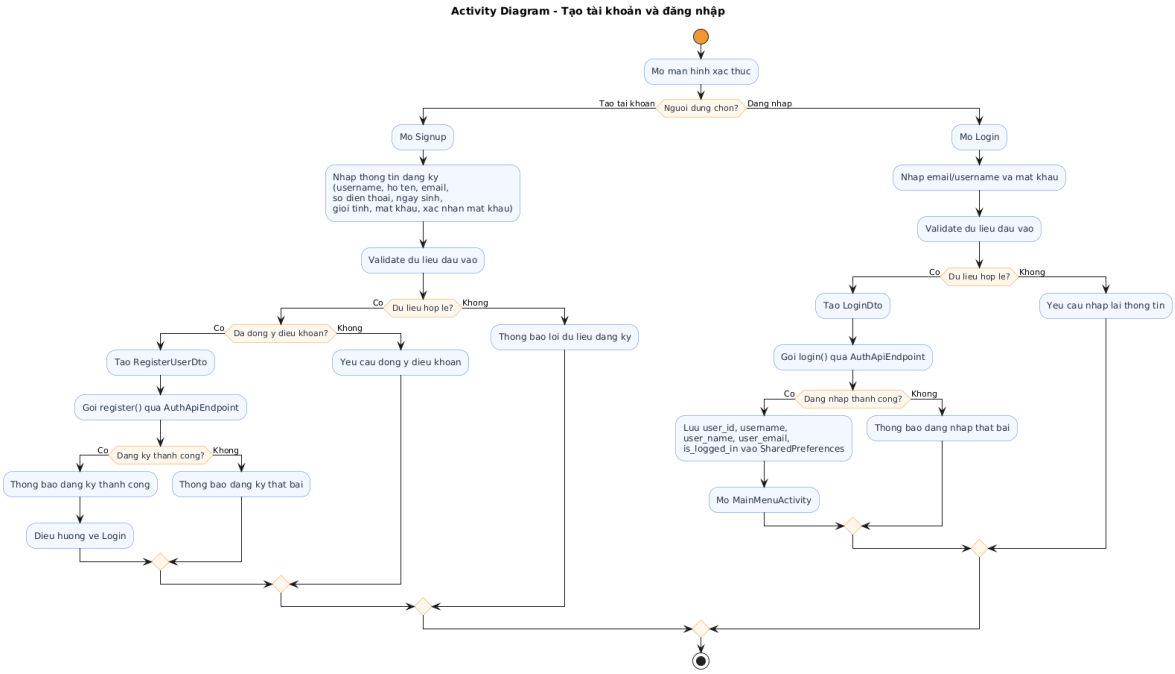
Luồng xác thực được xây dựng như điểm vào đầu tiên của hệ thống. Ở màn hình Signup, người dùng nhập thông tin tài khoản và thông tin cá nhân cơ bản; ứng dụng kiểm tra tính hợp lệ của dữ liệu đầu vào trước khi gửi yêu cầu đăng ký tới backend. Sau khi tạo tài khoản thành công, hệ thống điều hướng người dùng về màn hình Login để bắt đầu phiên làm việc [8].

Tại màn hình Login, người dùng nhập email hoặc tên đăng nhập cùng mật khẩu. Ứng dụng gọi API xác thực, nhận dữ liệu hồ sơ người dùng và lưu trạng thái phiên vào SharedPreferences để duy trì ngữ cảnh sử dụng giữa các màn hình. Sau bước này, người dùng được chuyển đến MainMenuActivity để truy cập các chức năng nghiệp vụ chính [4][8].



Hình 3.9. Sequence Diagram luồng đăng nhập và quên mật khẩu.

Sequence Diagram của phân hệ xác thực thể hiện mối liên hệ giữa Signup, Login, ForgotPasswordActivity, OtpVerificationDialogFragment và ResetPasswordActivity. Sơ đồ cho thấy dữ liệu đi qua ba chặng: tạo tài khoản mới, xác thực đăng nhập và khôi phục mật khẩu khi người dùng không còn khả năng truy cập tài khoản.



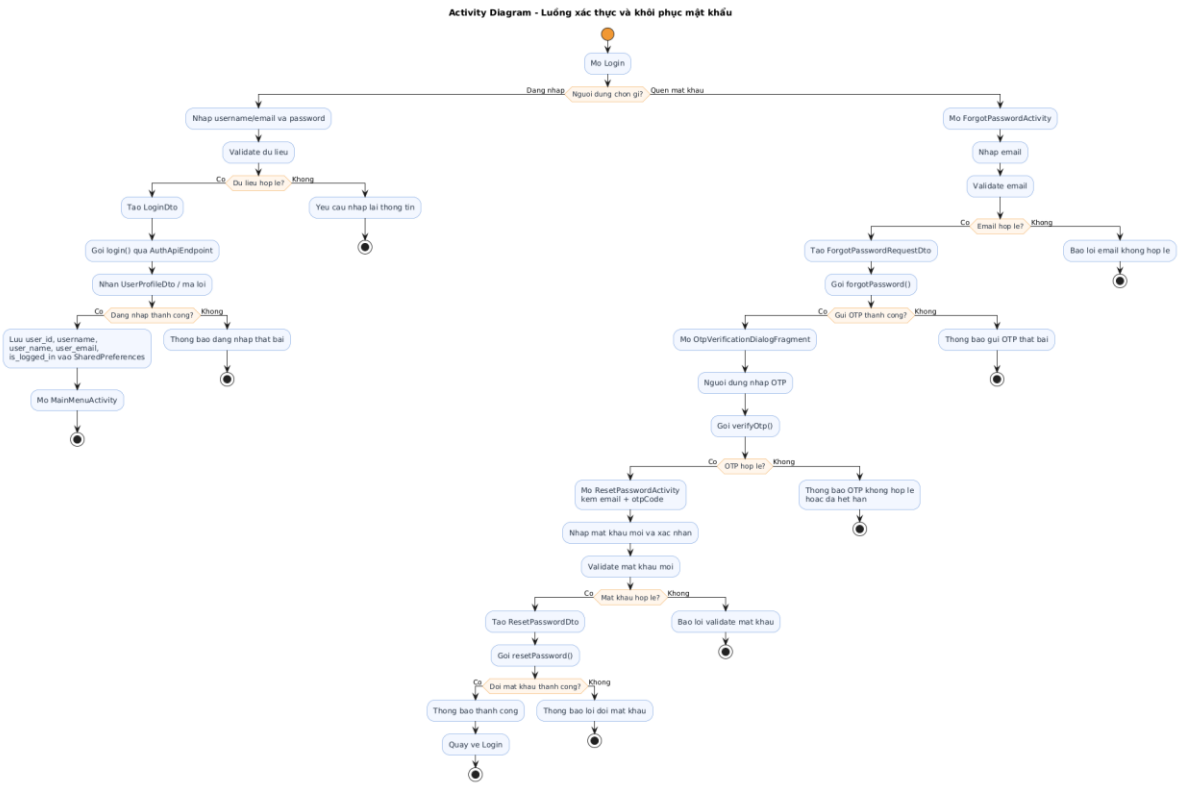
Hình 3.10. Activity tạo tài khoảng và đăng nhập

Trong nhánh khôi phục mật khẩu, ForgotPasswordActivity tiếp nhận địa chỉ email và gửi yêu cầu phát sinh OTP. OtpVerificationDialogFragment thực hiện bước xác thực mã dùng một lần; khi mã hợp lệ, ResetPasswordActivity tiếp tục nhận email và otpCode qua Intent để người dùng khai báo mật khẩu mới. Cấu trúc này giúp tách rõ từng bước của quy trình khôi phục tài khoản, đồng thời làm giảm mức độ phụ thuộc giữa các màn hình [8].

Logic lưu phiên được thực hiện ngay sau khi đăng nhập thành công và được tái sử dụng trong những tình huống cần khôi phục ngữ cảnh người dùng ở các màn hình phía sau. Đây là cơ chế giúp hệ thống duy trì thông tin định danh tối thiểu trong suốt quá trình tìm kiếm chuyến bay và tạo booking.

Sequence Diagram cũng cho thấy thao tác đăng nhập không dừng ở lời gọi API đơn lẻ mà còn gắn với ba bước xử lý bổ sung: xác minh dữ liệu phản hồi, lưu trạng thái phiên cục bộ và điều hướng người dùng sang màn hình trung tâm. Cách mô tả này phản ánh chính xác vai trò của lớp xác thực trong toàn bộ chuỗi nghiệp vụ của ứng dụng.

Thiết kế này nhằm tách biệt rõ từng bước của cơ chế phục hồi tài khoản. Việc đặt bước nhập OTP trong một fragment trung gian giúp quy trình khôi phục mật khẩu được tổ chức theo từng pha và tránh phải mở thêm một màn hình toàn phần chỉ cho thao tác xác thực mã dùng một lần.

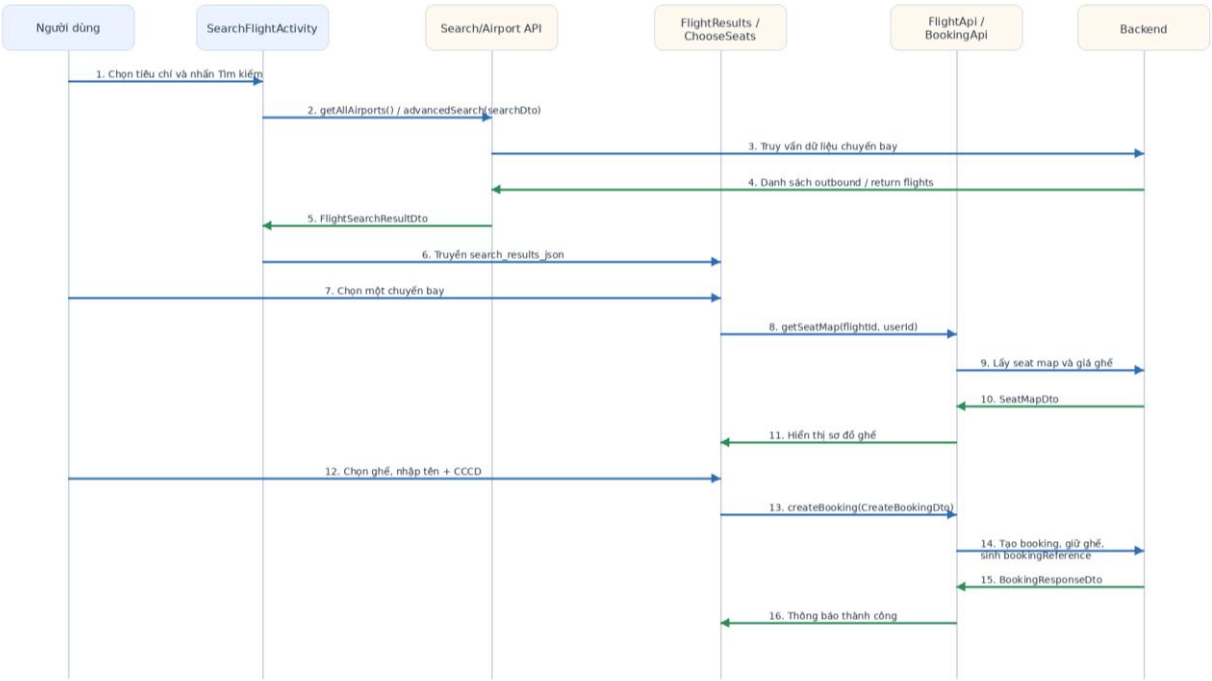


Hình 3.11. Actiuvity xác thực và khôi phục mật khẩu

3.8. Thiết kế luồng tìm kiếm, chọn ghế và đặt vé

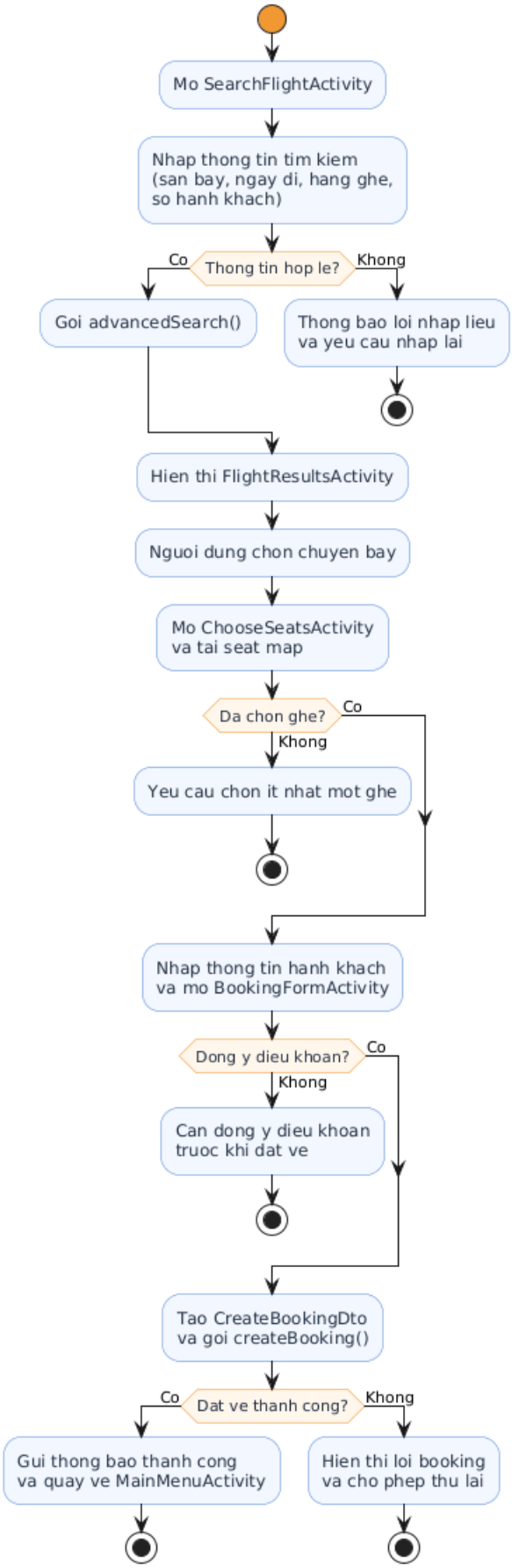
Đây là luồng nghiệp vụ quan trọng nhất và chiếm phần lớn giá trị của ứng dụng. Từ mã nguồn có thể thấy luồng được chia thành các bước nhỏ: nhập tiêu chí tìm kiếm, lấy danh sách chuyến bay, chọn chuyến bay, tải bản đồ ghế, thu thập thông tin hành khách theo ghế, tạo booking và phản hồi kết quả. Cách chia này có lợi cho cả giao diện lẫn kỹ thuật, vì mỗi màn hình chỉ phải giải quyết một nhiệm vụ cụ thể.

SearchFlightActivity là nơi người dùng nhập vào sân bay đi/đến, ngày đi/ngày về, số khách và hạng ghế. Sau khi validate, Activity tạo AdvancedFlightSearchDto và gọi searchApi.advancedSearch(). FlightResultsActivity tiếp nhận JSON kết quả, parse bằng Gson và đưa danh sách chuyến bay lên RecyclerView. Khi người dùng chọn một chuyến bay, ChooseSeatsActivity được mở để tải seat map từ FlightApiEndpoint. Sau cùng, BookingFormActivity tổng hợp dữ liệu và gửi CreateBookingDto tới BookingApiEndpoint.



Hình 3.12. Sequence Diagram luồng tìm kiếm, chọn ghế và đặt vé.

Activity Diagram - Luồng đặt vé (bản đơn giản)



Hình 3.13. Activity Diagram luồng tìm chuyến bay đến hoàn tất booking.

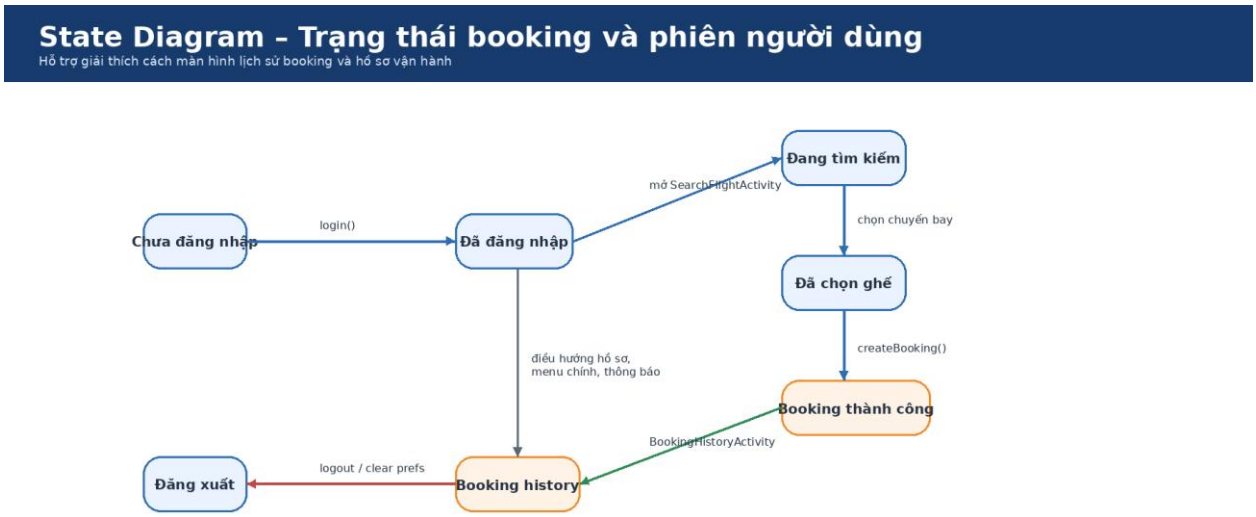
Bản đồ ghế và thông tin hành khách là một nghiệp vụ phức tạp, có nhiều bước kiểm tra như ghế còn trống, đã chọn bao nhiêu ghế, dữ liệu CCCD hợp lệ hay chưa. Nếu dồn toàn bộ logic này về FlightResultsActivity hoặc BookingFormActivity, mã nguồn sẽ trở nên rối và khó bảo trì.

Cũng cần lưu ý rằng dự án đang cho phép nhiều hành khách trong cùng một booking thông qua SelectedSeatInfo và danh sách selectedSeatsList. Đây là một điểm mạnh vì nó phản ánh đúng bài toán thực tế của đặt vé máy bay: một người dùng có thể đặt cho nhiều hành khách trong cùng chuyến bay.

3.9. Thiết kế lịch sử booking, hồ sơ và thông báo

Sau khi booking được tạo, ứng dụng tiếp tục cần những chức năng hậu giao dịch để giữ chân người dùng và nâng trải nghiệm sử dụng. Repo đang có những thành phần rõ ràng cho yêu cầu này: BookingHistoryActivity cho phép xem lịch sử, BookingDetailActivity cho phép xem lại chi tiết, PayActivity được dùng như nhánh thanh toán/đặt lại, Profile phục vụ hiển thị hồ sơ cá nhân, còn NotificationActivity và badge unread ở MainMenuActivity bổ sung khả năng theo dõi thông báo.

Xét về thiết kế hệ thống, đây là một nhóm màn hình quan trọng vì nó biến ứng dụng từ ‘công cụ tìm chuyến bay’ thành ‘ứng dụng quản trị trải nghiệm đặt vé’ trên điện thoại. Người dùng có thể quay lại kiểm tra trạng thái booking, theo dõi tuyến bay, xem thông tin bản thân hoặc nhận thông tin mới từ hệ thống.



Hình 3.14. State Diagram trạng thái booking và phiên người dùng.

State Diagram ở trên giúp giải thích một khía cạnh mà mã nguồn rải rác ở nhiều nơi mới thể hiện đầy đủ: mối quan hệ giữa trạng thái phiên người dùng và trạng thái booking. Khi người dùng đăng xuất, toàn bộ user_prefs bị xóa; khi đăng nhập lại, ứng dụng có thể truy cập lịch sử booking từ server. Trong lịch sử, các trạng thái Confirmed, Completed, Cancelled là thông tin để kích hoạt những nhánh điều hướng khác nhau như xem chi tiết hoặc vào PayActivity.

3.10. Thiết kế lỗi, validate và bảo mật

Qua các Activity đã phân tích, validate đầu vào được triển khai tương đối nhất quán

Về xử lý lỗi server, nhiều Activity đã có các hàm handleErrorResponse() để map mã lỗi HTTP thành thông điệp gần gũi cho người dùng. Chẳng hạn 401 thường được hiểu là hết phiên hoặc sai thông tin xác thực, 404 là không tìm thấy tài nguyên, 409 là xung đột dữ liệu như ghế đã bị người khác đặt trước. Mô thức này giúp UX rõ ràng hơn và cũng cho phép báo cáo mô tả lỗi theo đúng tình huống nghiệp vụ.

Về bảo mật, client không lưu mật khẩu thô sau khi đăng nhập. Tuy nhiên repo vẫn tồn tại các điểm cần nâng cấp như base URL hard-code, dùng HTTP thay vì HTTPS, quản lý phiên bằng SharedPreferences thay vì token hoặc encrypted storage, và chưa

có interceptor tập trung để xử lý lỗi xác thực xuyên suốt. Những điểm này sẽ được đánh giá sâu hơn ở chương 5.

CHƯƠNG 4.
KẾT QUẢ TRIỂN KHAI THEO MÃ NGUỒN

4.1. Cấu trúc package và trách nhiệm từng nhóm lớp

Một lợi thế của dự án là cách đặt tên file khá trực quan, nhờ đó việc mô tả kết quả triển khai bám rất sát source code. Các lớp Activity, interface API và DTO không bị ẩn sau những tên gọi trừu tượng; người đọc có thể đối chiếu tương đối nhanh giữa báo cáo và dự án. Điều này đặc biệt hữu ích khi báo cáo được dùng để thuyết trình hoặc bàn giao cho người khác tiếp tục phát triển.

Bảng 4.1. Danh sách màn hình chính và trách nhiệm xử lý

Lớp / Activity	Chức năng chính	Layout liên quan
MainActivity	Màn hình chào của ứng dụng, dẫn người dùng tới Login.	activity_welcome.xml
Login	Xác thực người dùng, lưu phiên bằng SharedPreferences.	activity_login.xml
Signup	Tạo tài khoản mới, kiểm tra dữ liệu và gọi register API.	activity_signup.xml
ForgotPasswordActivity	Gửi yêu cầu OTP quên mật khẩu.	activity_forgot_password.xml
ResetPasswordActivity	Đặt lại mật khẩu sau khi OTP hợp lệ.	activity_reset_password.xml
MainMenuActivity	Trung tâm điều hướng sau đăng nhập.	activity_main_menu.xml
SearchFlightActivity	Tìm kiếm chuyến bay với dữ liệu sân bay và tiêu chí bay.	activity_search_flight.xml
FlightResultsActivity	Hiển thị danh sách chuyến bay kết quả.	activity_flight_results.xml
ChooseSeatsActivity	Tải seat map, chọn ghế, nhập thông tin hành khách.	activity_choose_seats.xml
BookingFormActivity	Tổng hợp dữ liệu và gửi CreateBookingDto.	activity_booking_form.xml
BookingHistoryActivity	Lấy lịch sử booking, tìm kiếm/lọc, mở chi tiết.	activity_booking_history.xml
Profile	Hiển thị hồ sơ, thống kê và menu cá nhân.	activity_profile.xml

4.2. Kết quả giao diện nhóm xác thực

Nhóm màn hình xác thực là phần đầu tiên người dùng tương tác với ứng dụng, do đó đóng vai trò quyết định ấn tượng ban đầu. Từ mã nguồn có thể thấy nhóm phát triển chọn thiết kế đơn giản, ít thành phần thừa: MainActivity chỉ đóng vai trò giới thiệu và điều hướng, Login tập trung vào thao tác đăng nhập, Signup chứa nhiều trường nhập liệu hơn nhưng vẫn giữ bố cục một cột dễ nhìn trên màn hình điện thoại.

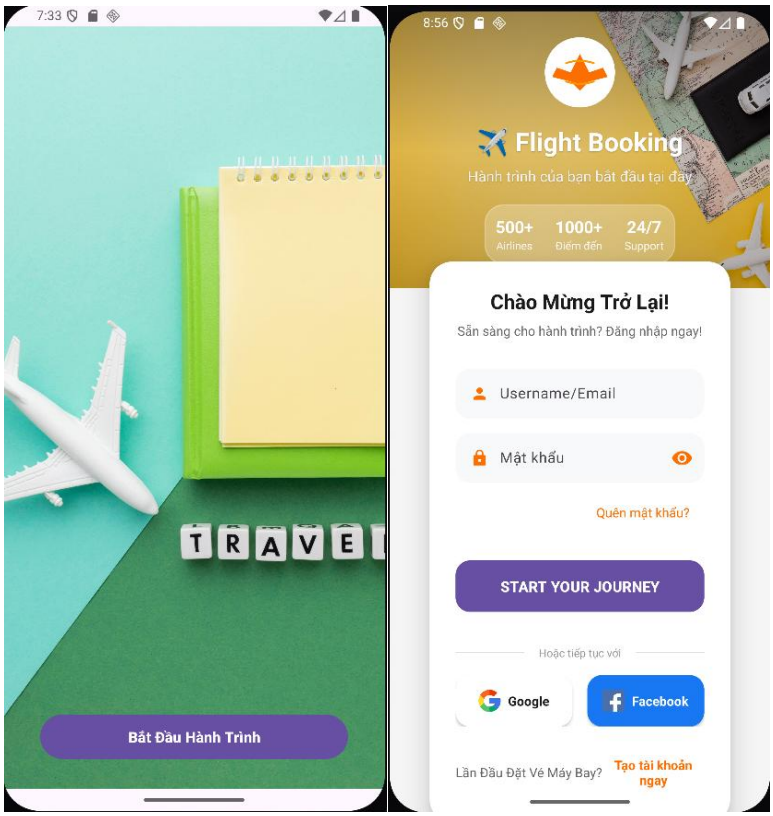
Ở góc độ hiện thực mã nguồn, Login và Signup đều tự binding view, gắn click listener và thực hiện validate trực tiếp trong Activity. Đây là cách làm dễ theo dõi đối với người mới học. Login sau khi nhận UserProfileDto còn gọi saveUserInfo() để đồng

bộ dữ liệu vào user_prefs; Signup cũng có hàm tương tự sau khi đăng ký thành công. Việc lặp lại logic này là một điểm có thể refactor, nhưng cũng cho thấy dự án đã nhận diện rõ nhu cầu lưu phiên nhất quán.

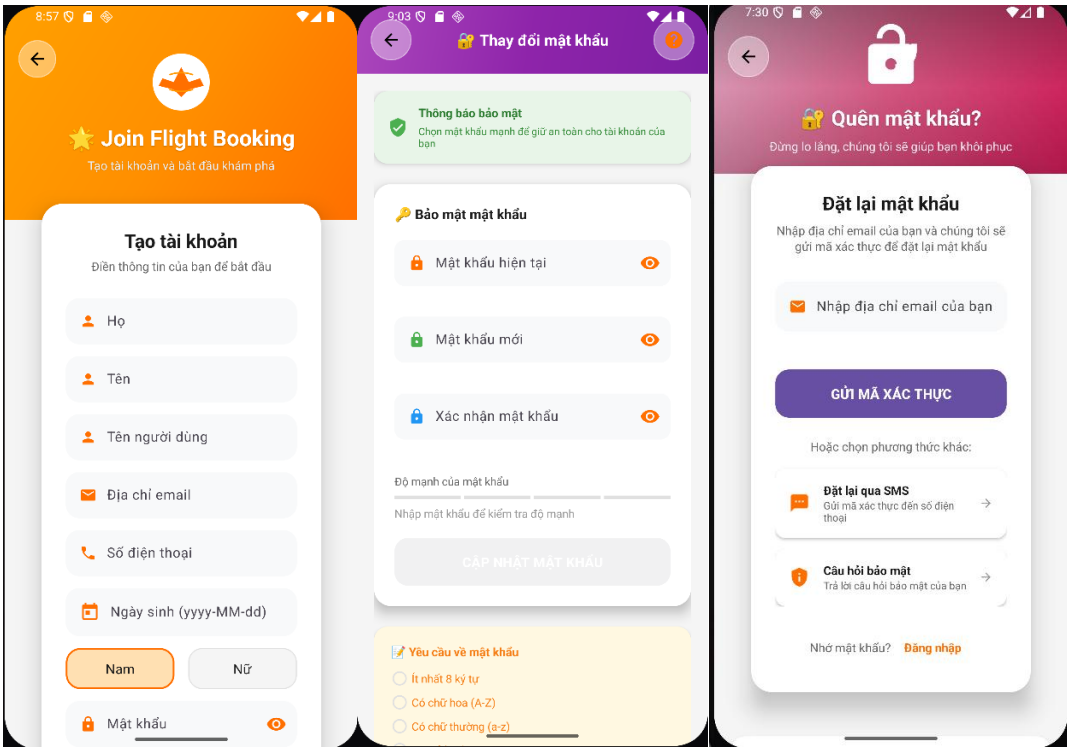
4.2.1. MainActivity và Login

MainActivity hiện thực một màn hình welcome với nút "Get Started" làm điểm điều hướng duy nhất sang Login. Cấu trúc một hành động chính giúp màn hình mở đầu giữ được bố cục gọn, tập trung vào bước chuyển tiếp sang phân hệ xác thực.

Login là màn hình quan trọng nhất của nhóm xác thực. Activity này có các thành phần nhập email/username, mật khẩu, nút đăng nhập, liên kết đăng ký và quên mật khẩu. Phần validate kiểm tra rỗng trước khi gọi API; khi server phản hồi thành công, màn hình lưu thông tin người dùng và mở MainMenuActivity. Khi lỗi xảy ra, Login map mã lỗi HTTP thành thông điệp thân thiện hơn như sai tài khoản/mật khẩu hoặc lỗi server.



Hình 4.1. Giao diện nhóm xác thực người dùng.

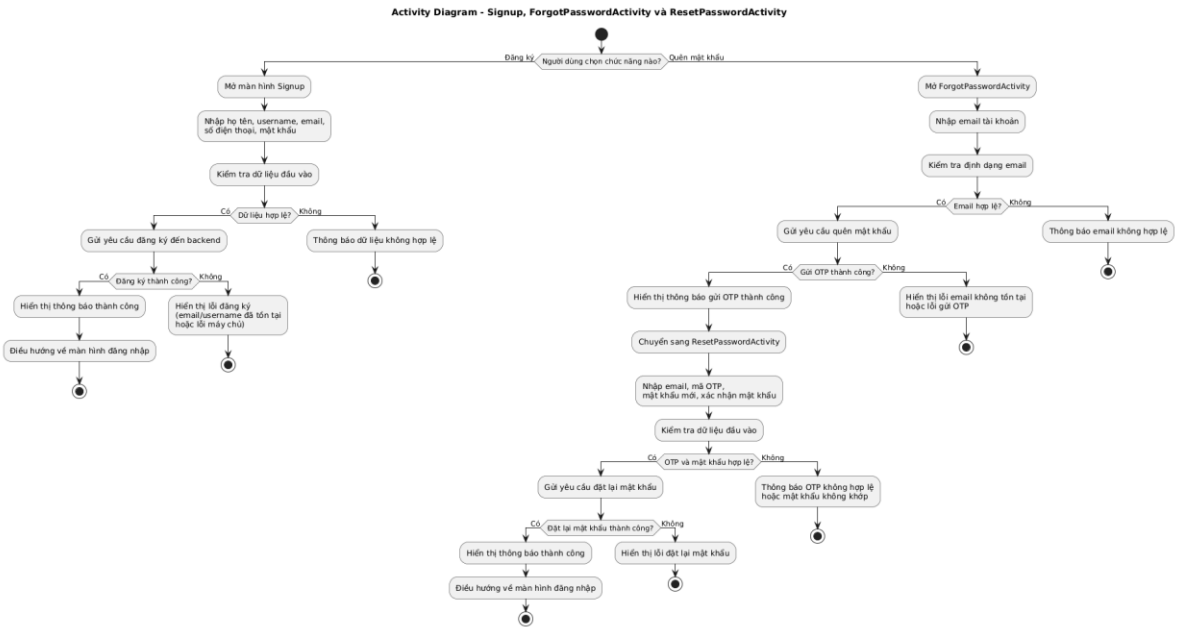


Hình 4.2. Giao diện nhóm đăng kí, quên mật khẩu.

4.2.2. Signup, ForgotPasswordActivity và ResetPasswordActivity

Signup triển khai kiểm tra username, email, phone, ngày sinh, giới tính, điều khoản sử dụng và độ mạnh mật khẩu ngay trên client. Thành phần đáng chú ý của màn hình này là

ForgotPasswordActivity và ResetPasswordActivity tạo thành một chuỗi xử lý liên tiếp.

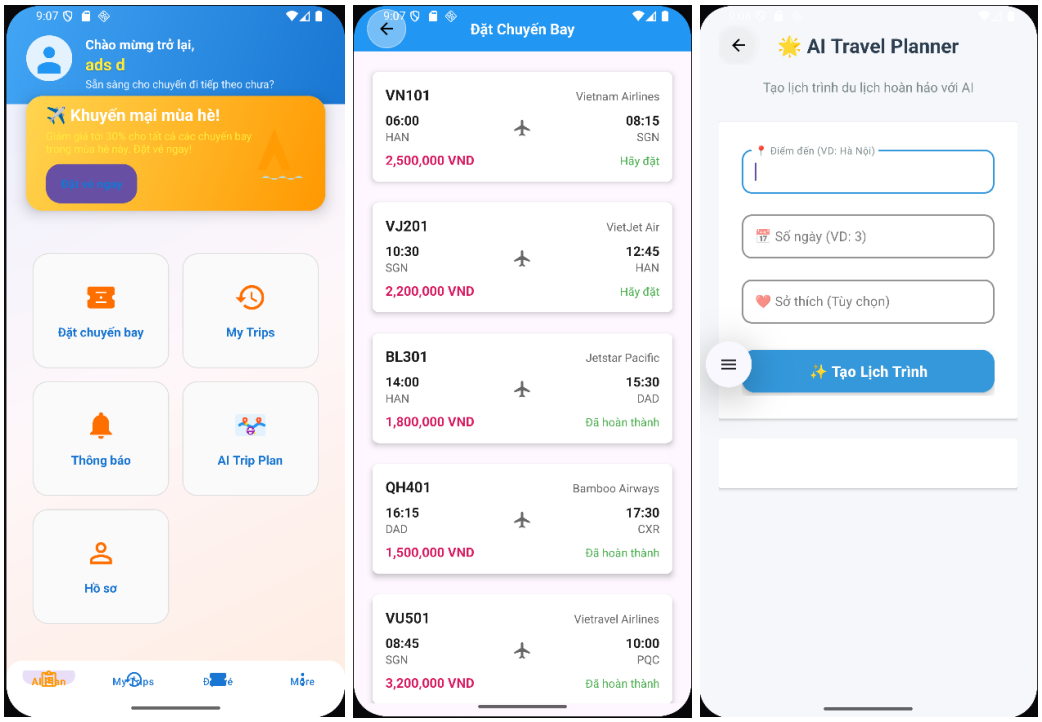


Hình 4.3. Signup, ForgotPasswordActivity và ResetPasswordActivity

4.3. Kết quả giao diện nhóm tìm chuyến bay và AI Planner

Sau khi đăng nhập, MainMenuActivity đóng vai trò trung tâm điều hướng. Ở đây người dùng có thể chuyển sang màn đặt vé, lịch sử, hồ sơ, thông báo hoặc AI Planner. Ngoài ra MainMenuActivity còn đọc user_prefs để hiển thị lời chào cá nhân hóa và gọi notificationApi.getUnreadCount(userId) để cập nhật badge thông báo chưa đọc.

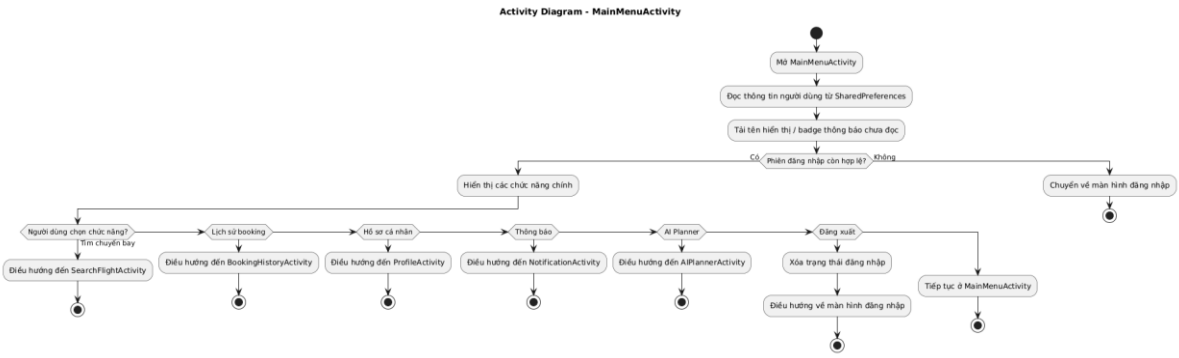
SearchFlightActivity là màn hình trung tâm của phân hệ tìm kiếm vì nó kết hợp dữ liệu động



Hình 4.4. Giao diện màn hình chính và tìm kiếm chuyến bay.

4.3.1. MainMenuActivity

MainMenuActivity tập hợp nhiều menu và click handler, cho thấy nó đóng vai trò trung tâm của trải nghiệm sau đăng nhập. Từ đây người dùng có thể đi vào các cụm chức năng cốt lõi. Một điểm thú vị trong mã nguồn là việc Activity này cũng xử lý logout, hiển thị bottom sheet cho tùy chọn mở rộng và kiểm tra unread notification count. Như vậy, đây không chỉ là màn menu mà còn là nơi điều phối trạng thái chung của người dùng.



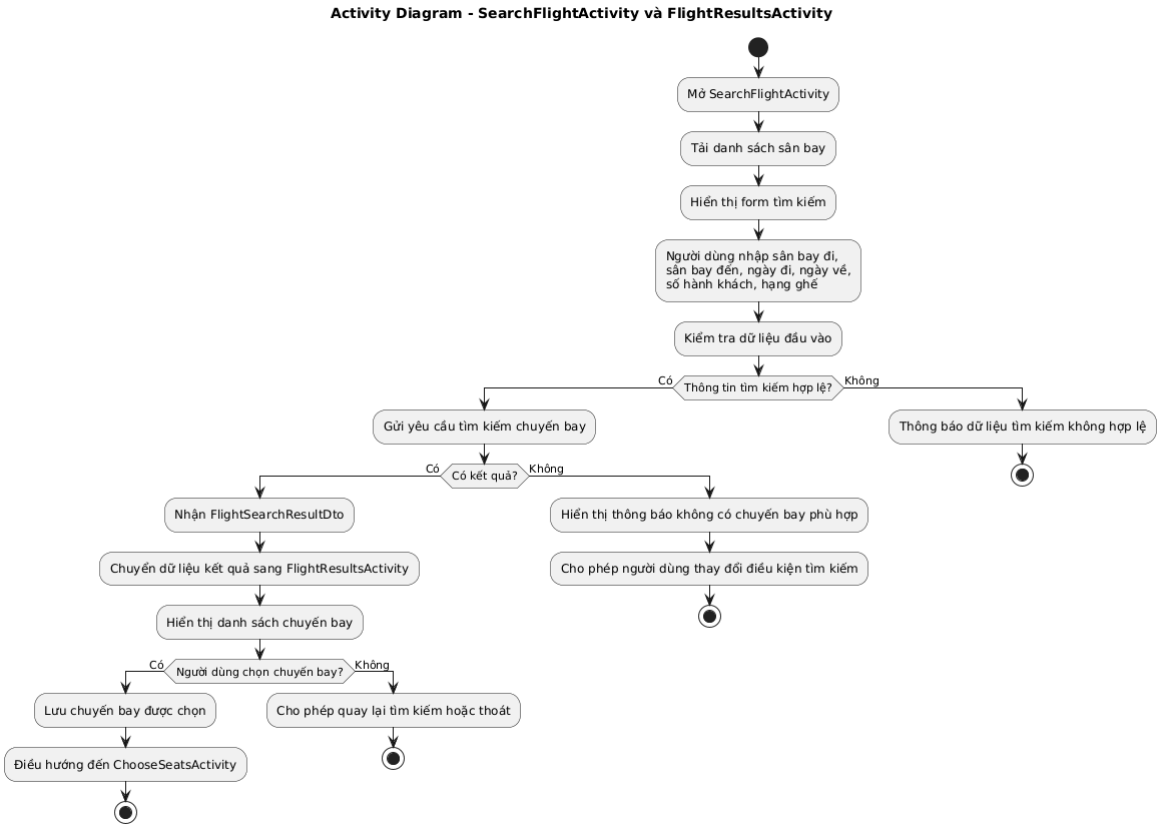
Hình 4.5. MainMenuActivity

4.3.2. SearchFlightActivity và FlightResultsActivity

SearchFlightActivity có cấu trúc tương đối hoàn chỉnh: nạp user name từ SharedPreferences, tải danh sách sân bay từ server, khởi tạo danh sách hạng ghế mẫu,

validate dữ liệu đầu vào và gửi AdvancedFlightSearchDto tới backend. Khi có kết quả, Activity serialize toàn bộ FlightSearchResultDto sang JSON để truyền sang FlightResultsActivity. Cách làm này giúp giảm sự phức tạp khi chuyển object giữa Activity.

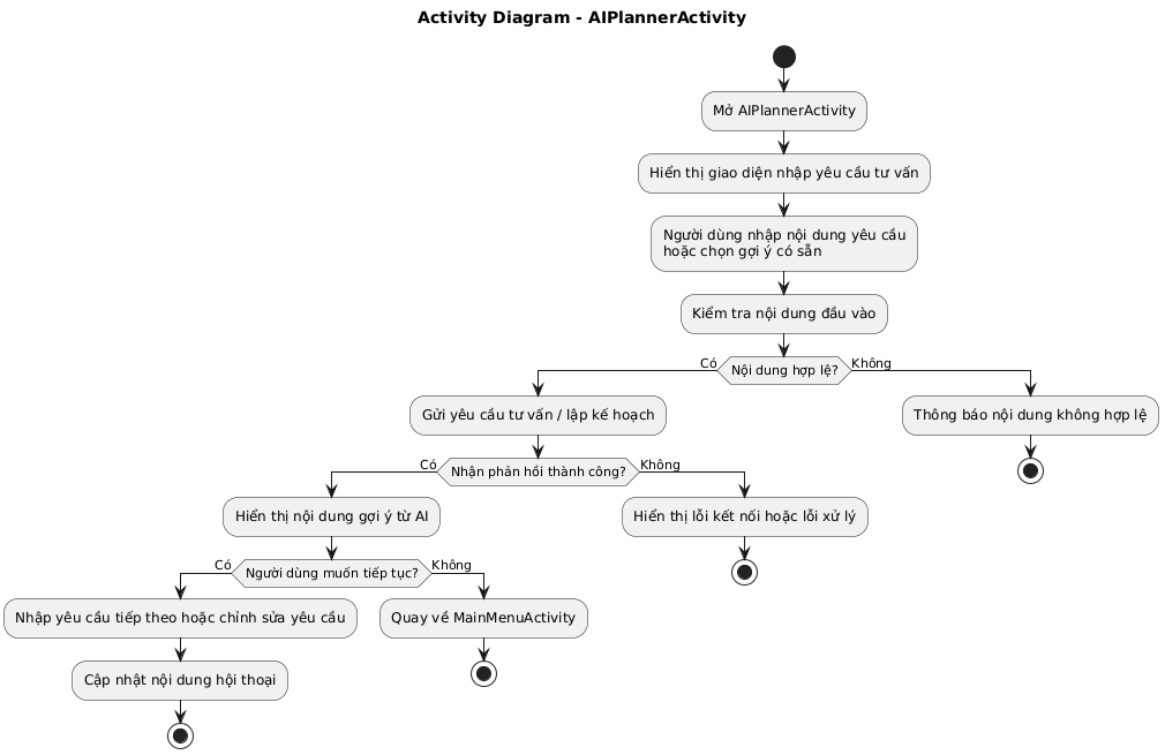
FlightResultsActivity chịu trách nhiệm parse JSON kết quả, hiển thị danh sách chuyến bay và mở ChooseSeatsActivity khi người dùng chọn một chuyến cụ thể. Trong mã nguồn, màn hình này còn có bước kiểm tra isLoggedIn() khi onCreate() và onResume(), thể hiện sự cẩn trọng trong việc bảo vệ luồng booking khỏi người dùng không hợp lệ.



Hình 4.6. SearchFlightActivity và FlightResultsActivity

4.3.3. AIPlannerActivity

Activity này trong danh sách file và menu điều hướng cho thấy ứng dụng đã có định hướng bổ sung một lớp hỗ trợ thông minh. Trong bối cảnh ứng dụng đặt vé máy bay, AI Planner có thể đóng vai trò tư vấn lịch trình, gợi ý thời gian bay hoặc hỗ trợ tổng hợp thông tin chuyến đi cho người dùng.

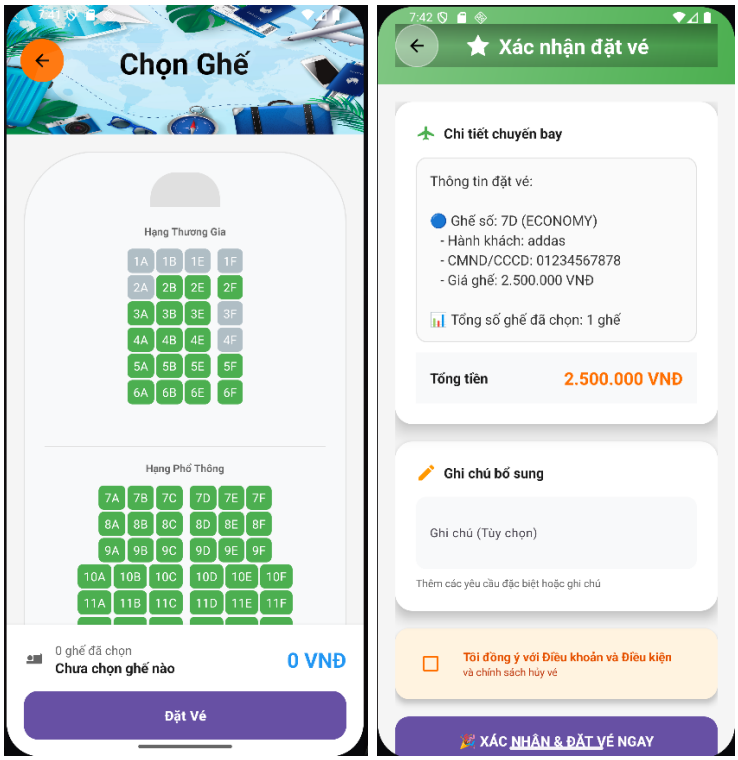


Hình 4.7. **AIPlannerActivity**

4.4. Kết quả giao diện nhóm chọn ghế, tạo booking và thanh toán

Nếu SearchFlightActivity thể hiện năng lực tìm kiếm, thì ChooseSeatsActivity và BookingFormActivity thể hiện năng lực xử lý nghiệp vụ phức tạp. ChooseSeatsActivity không chỉ tải seat map mà còn dựng sơ đồ ghế động bằng code, phân nhóm ghế theo hàng, cho phép chọn/hủy ghế, đồng thời yêu cầu nhập tên hành khách và số giấy tờ cho từng ghế. Đây là nơi lượng logic giao diện khá lớn được đặt trực tiếp trong Activity.

BookingFormActivity tiếp nhận selectedSeatsList, tạo tóm tắt booking, định dạng giá tiền, thu nhận ghi chú và điều khoản, sau đó sinh CreateBookingDto để gửi lên Booking API. Sau khi server trả bookingReference, Activity hiển thị Toast, tạo notification nội bộ và quay về MainMenuActivity. Từ góc nhìn trải nghiệm người dùng, đây là một luồng hoàn chỉnh và đủ thuyết phục để trình bày trong demo đồ án.

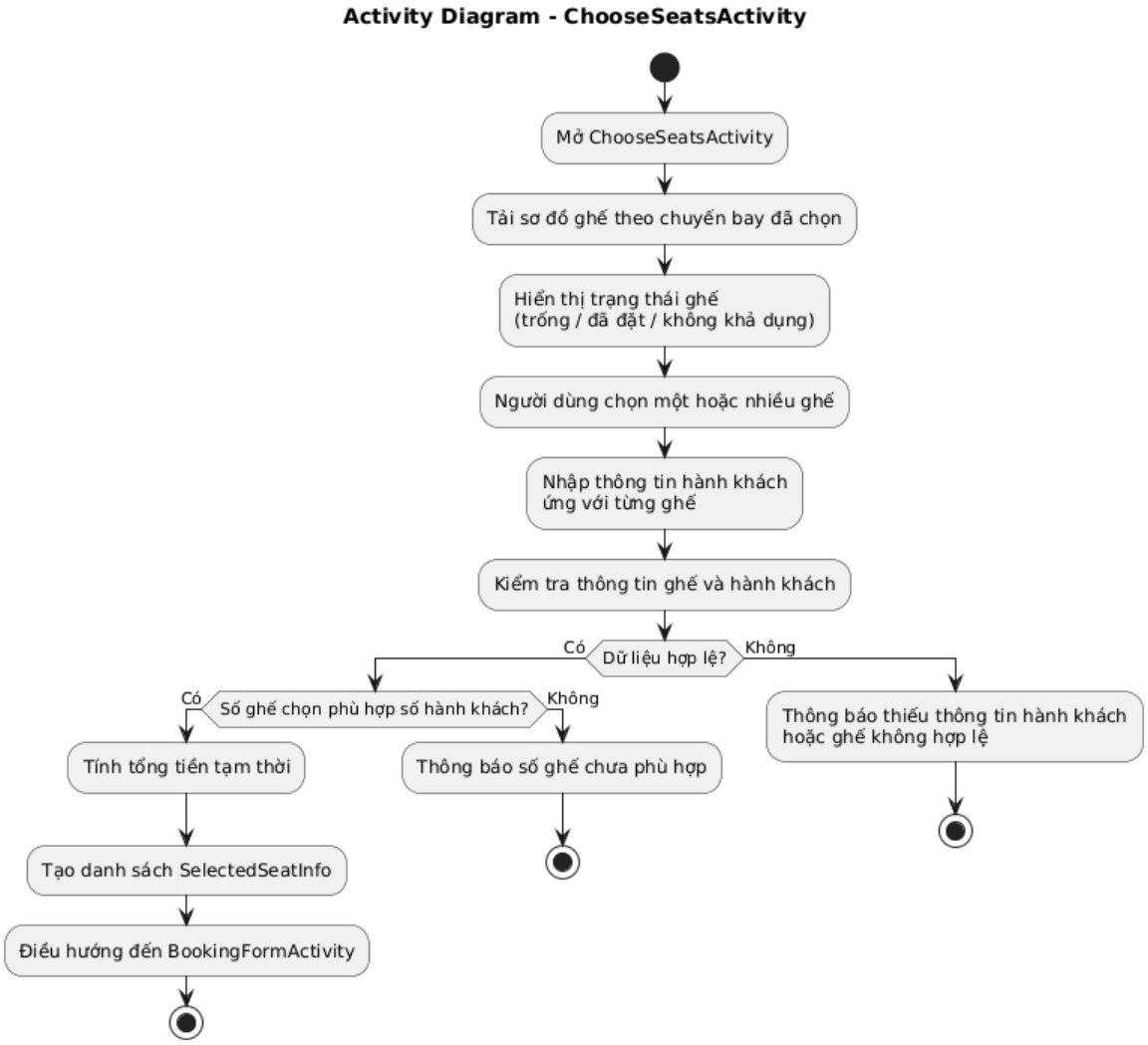


Hình 4.8. Minh họa giao diện chọn ghế, tạo booking và thanh toán.

4.4.1. ChooseSeatsActivity

Đặc điểm nổi bật của ChooseSeatsActivity là khả năng dựng seat map động.

Tuy nhiên, việc toàn bộ logic dựng seat map, hiển thị dialog, kiểm tra dữ liệu hành khách và tính tổng tiền đều nằm trong một Activity cũng là dấu hiệu cho thấy lớp giao diện đang gánh nhiều trách nhiệm. Đây là điều báo cáo ghi nhận ở phần đánh giá kỹ thuật, đồng thời đề xuất hướng refactor sau đồ án.



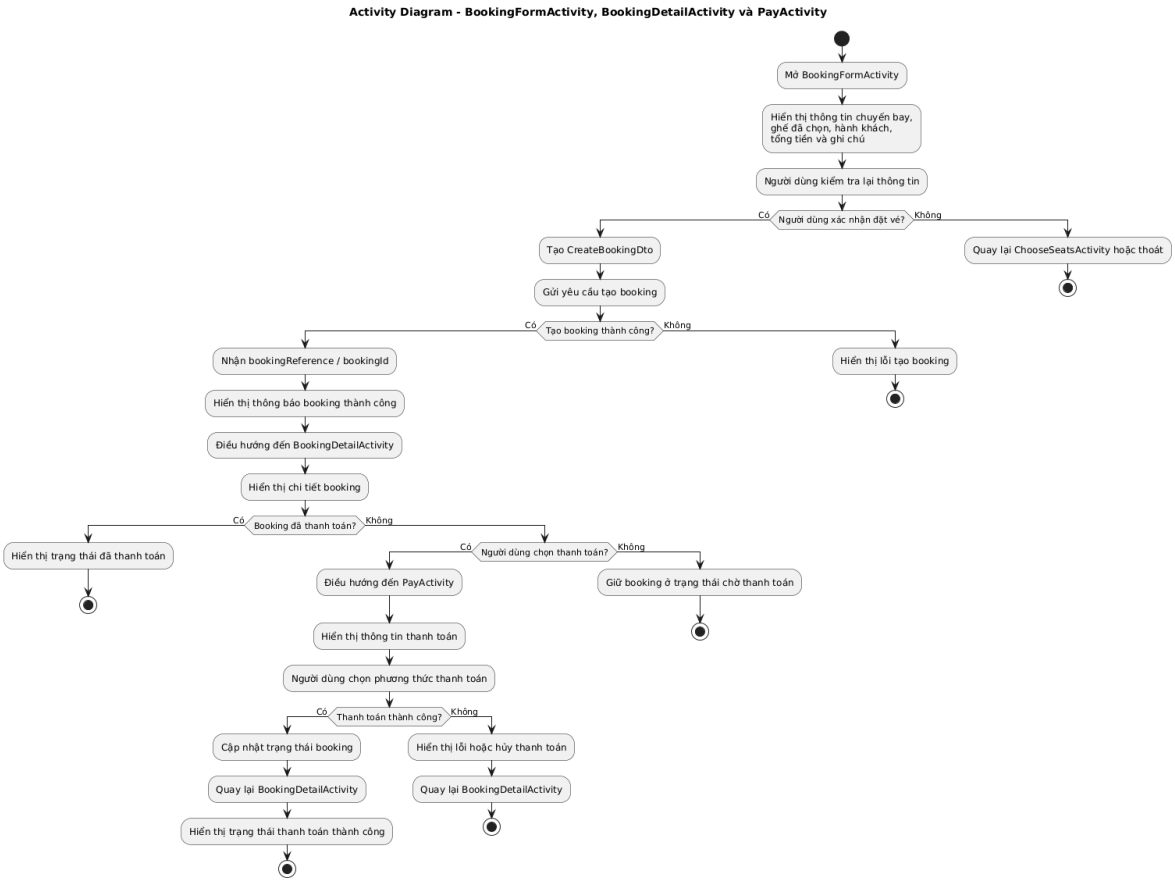
Hình 4.9. ChooseSeatsActivity

4.4.2. BookingFormActivity, BookingDetailActivity và PayActivity

BookingFormActivity được thiết kế như màn hình xác nhận cuối cùng. Điều này rất đúng với kỳ vọng của người dùng: trước khi gửi booking, hệ thống cần cho phép kiểm tra lại danh sách ghế, hành khách, tổng tiền và ghi chú. Nhờ selectedSeatsList chứa đầy đủ thông tin, Activity này có thể dựng tóm tắt chi tiết theo từng ghế.

Về hậu booking, BookingDetailActivity là nơi người dùng có thể xem lại thông tin một booking cụ thể, đặc biệt khi mở từ notification. PayActivity xuất hiện như một nhánh xử lý phát sinh từ BookingHistoryActivity, nhận bookingId, bookingReference, totalAmount, flightNumber và route. Ngay cả khi chi tiết logic

thanh toán chưa được trích xuất đầy đủ, sự tồn tại của luồng này cho thấy nhóm phát triển đã nghĩ tới giai đoạn sau khi booking hình thành.

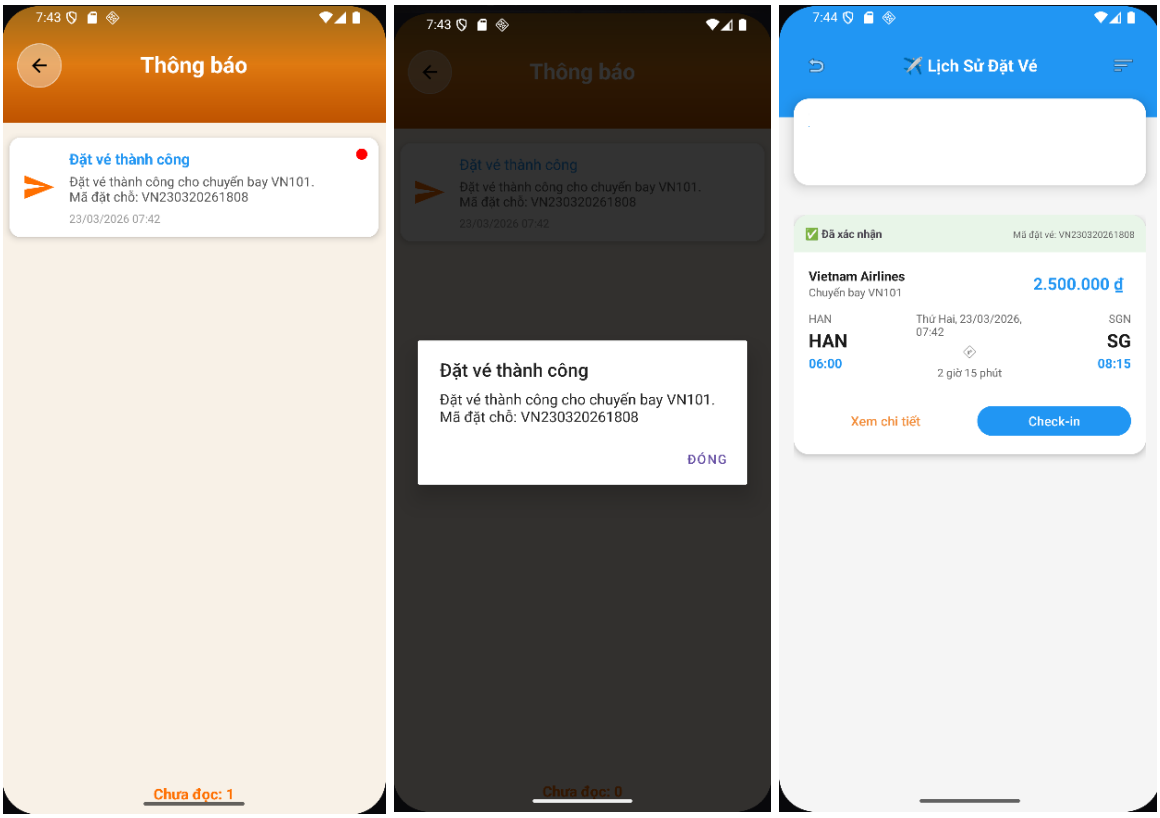


Hình 4.10. BookingFormActivity, BookingDetailActivity và PayActivity

4.5. Kết quả giao diện nhóm lịch sử, hồ sơ và thông báo

Một ứng dụng đặt vé sẽ không hoàn chỉnh nếu người dùng không thể quay lại kiểm tra lịch sử đặt chỗ hoặc cập nhật thông tin bản thân. Ở dự án hiện tại, BookingHistoryActivity và Profile là hai thành phần thể hiện khá rõ nhóm chức năng hậu giao dịch và quản lý tài khoản.

BookingHistoryActivity sử dụng RecyclerView và adapter riêng để hiển thị từng booking với mã đặt chỗ, hãng bay, tuyến đường, trạng thái và các nút hành động. Từ những trạng thái như Confirmed, Completed hoặc Cancelled, ứng dụng quyết định hiển thị nút check-in/đặt lại hoặc ẩn nút tương ứng. Profile thì gọi getProfile(userId), hiển thị tên, email, username, phone, gender, date of birth và tổng số booking. MainMenuActivity bổ sung badge thông báo chưa đọc, trong khi NotificationActivity đóng vai trò nơi người dùng xem nội dung thông báo.

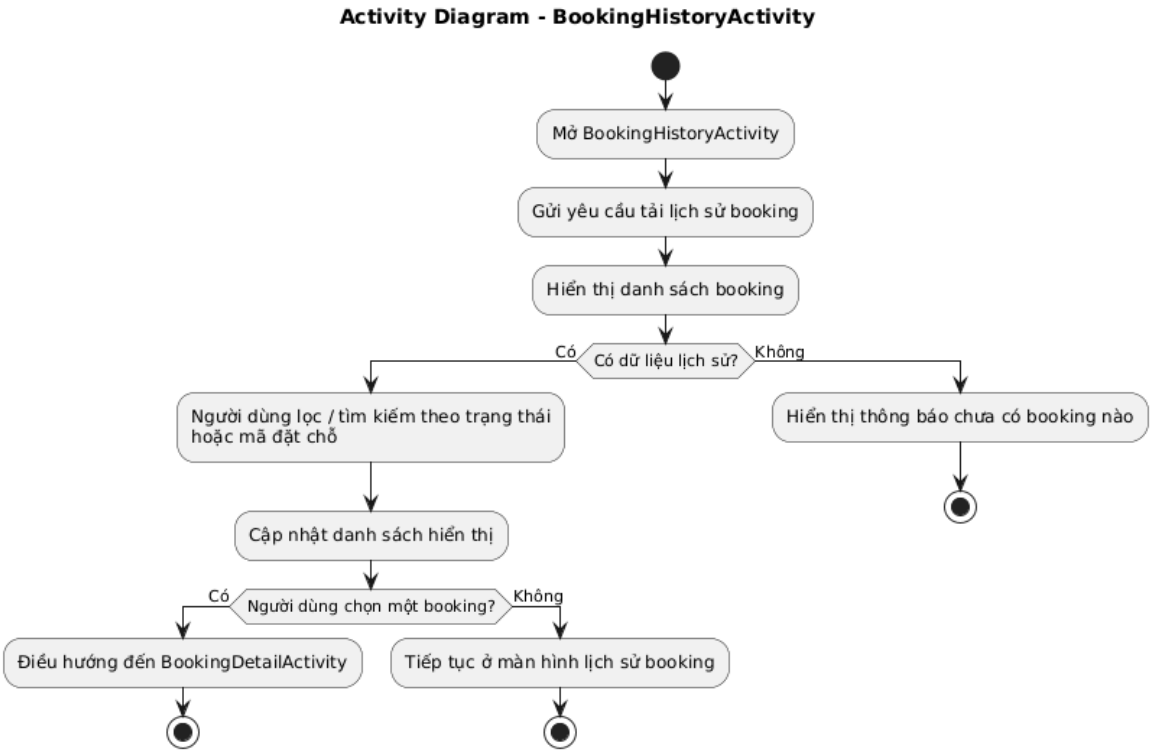


Hình 4.11. Minh họa giao diện hồ sơ, lịch sử và thông báo.

4.5.1. BookingHistoryActivity

BookingHistoryActivity cho thấy cách tổ chức màn hình theo hướng phục vụ đầy đủ nhu cầu tra cứu hậu giao dịch

Dù vậy, quá trình đọc mã nguồn cũng chỉ ra một số rủi ro kỹ thuật ở màn này, chẳng hạn việc updateChipStyles() tham chiếu chipUpcoming và chipCompleted trong khi phần bind view trích xuất được chưa khởi tạo các view tương ứng. Đây là một phát hiện quan trọng vì nó cho thấy vai trò của báo cáo không chỉ là mô tả mặt tích cực, mà còn là tài liệu phát hiện sớm điểm cần sửa trong dự án.

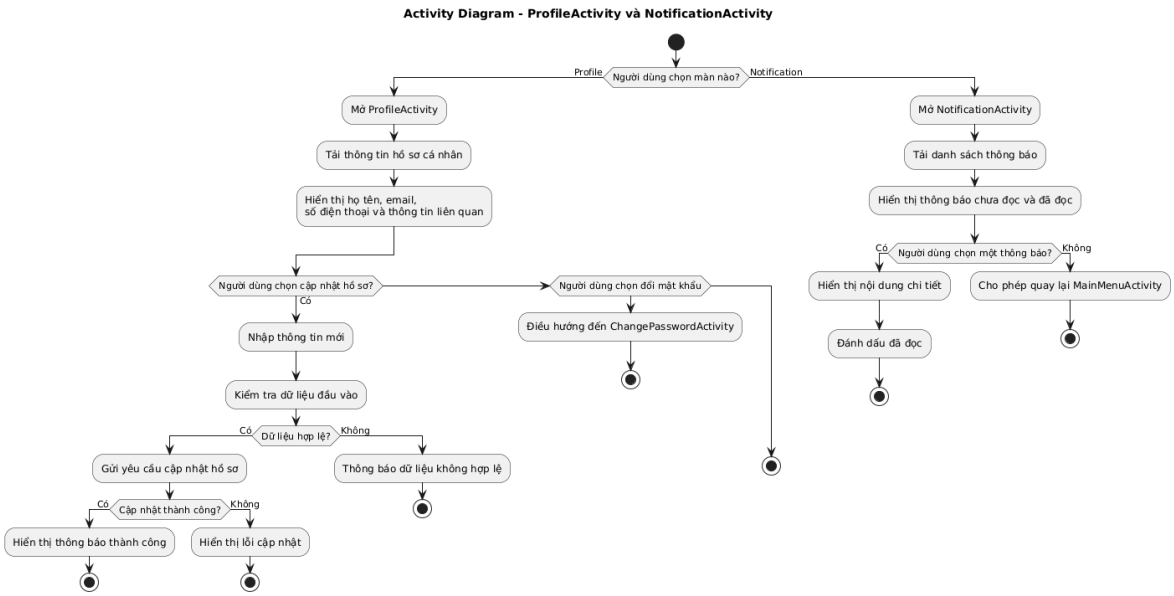


Hình 4.12. BookingHistoryActivity

4.5.2. Profile và NotificationActivity

Profile được thiết kế như màn hình tổng hợp thông tin cá nhân. Một đặc điểm quan trọng của màn hình này là

NotificationActivity và unread badge ở MainMenuActivity góp phần tạo ra trải nghiệm sản phẩm liền mạch hơn. Dù phần mã chi tiết của NotificationActivity chưa được trích xuất trong tài liệu này, chỉ riêng việc badge unread được cập nhật theo userId đã cho thấy repo chú ý tới giao tiếp hai chiều sau khi người dùng đăng nhập và phát sinh booking.



Hình 4.13. Profile và NotificationActivity

4.6. Kết quả triển khai khối API và DTO

Ở tầng API, dự án được tổ chức theo hướng phân tách rõ ràng giữa thành phần cấu hình kết nối, thành phần cung cấp dịch vụ và các nhóm endpoint theo từng chức năng nghiệp vụ. Cụ thể, RetrofitClient đảm nhiệm vai trò khởi tạo và cấu hình đối tượng Retrofit dùng cho toàn bộ quá trình giao tiếp với backend. Trên cơ sở đó, ApiServiceProvider thực hiện chức năng tập trung hóa việc cung cấp các service API, giúp các màn hình và lớp xử lý trong ứng dụng có thể truy cập endpoint theo cách thống nhất. Bên cạnh đó, các interface endpoint được chia theo từng nhóm nghiệp vụ như xác thực người dùng, tìm kiếm chuyến bay, đặt vé, hồ sơ cá nhân và lịch sử giao dịch. Cách tổ chức này góp phần làm rõ trách nhiệm của từng thành phần, giảm mức độ phụ thuộc trực tiếp giữa các lớp và hỗ trợ thuận lợi cho việc mở rộng hoặc bảo trì hệ thống trong các giai đoạn tiếp theo.

Về DTO, hệ thống sử dụng các lớp dữ liệu trung gian để chuẩn hóa quá trình trao đổi dữ liệu giữa ứng dụng Android và backend. Các lớp như LoginDto, UserProfileDto, AdvancedFlightSearchDto, FlightSearchResultDto, SelectedSeatInfo, CreateBookingDto và UserBookingHistoryDto phản ánh việc dự án tổ chức dữ liệu theo hướng tách biệt giữa mô hình truyền nhận dữ liệu và mô hình giao diện. Cách tiếp cận này giúp dữ liệu đầu vào và đầu ra của từng API được biểu diễn rõ ràng, nhất quán

với từng ngữ cảnh nghiệp vụ, đồng thời hỗ trợ kiểm soát tốt hơn quá trình ánh xạ dữ liệu trong ứng dụng.

Xét về mặt chức năng, nhóm DTO phục vụ cho nhiều nghiệp vụ cốt lõi của hệ thống. LoginDto được sử dụng trong quá trình xác thực tài khoản người dùng; UserProfileDto phục vụ hiển thị và cập nhật thông tin hồ sơ cá nhân; AdvancedFlightSearchDto chứa các tham số tìm kiếm chuyến bay nâng cao; FlightSearchResultDto biểu diễn tập kết quả trả về sau khi truy vấn chuyến bay; SelectedSeatInfo lưu trữ thông tin ghế và hành khách tương ứng trong quá trình chọn chỗ; CreateBookingDto đóng gói dữ liệu gửi đến backend khi tạo đơn đặt vé; và UserBookingHistoryDto hỗ trợ hiển thị danh sách lịch sử đặt vé của người dùng. Việc phân chia DTO theo từng nghiệp vụ cho thấy hệ thống đã được triển khai theo hướng có cấu trúc, bảo đảm tính rõ ràng trong luồng xử lý và tính nhất quán trong truyền nhận dữ liệu.

Từ kết quả triển khai có thể thấy khối API và DTO giữ vai trò trung tâm trong việc kết nối giữa giao diện người dùng và hệ thống backend. Sự kết hợp giữa cơ chế gọi API thông qua Retrofit và mô hình DTO chuyên biệt theo từng chức năng đã hình thành nên một kiến trúc trao đổi dữ liệu tương đối hoàn chỉnh, đáp ứng được các yêu cầu chính của ứng dụng đặt vé máy bay trên nền tảng Android.

Bảng 4.3. Ma trận chức năng – Activity – API liên quan

Activity	API / phương thức	Mô tả
Login	AuthApiEndpoint.login	Nhận UserProfileDto, lưu user_prefs.
Signup	AuthApiEndpoint.register	Tạo tài khoản, nhận hồ sơ người dùng.
ForgotPasswordActivity	AuthApiEndpoint.forgotPassword	Gửi OTP đến email.
ResetPasswordActivity	AuthApiEndpoint.resetPassword	Đặt lại mật khẩu.
SearchFlightActivity	AirportApiEndpoint.getAllAirports / AdvancedSearchApiEndpoint.advancedSearch	Tải danh sách sân bay và gửi truy vấn chuyến bay.
ChooseSeatsActivity	FlightApiEndpoint.getSeatMap	Lấy seat map theo flightId.
BookingFormActivity	BookingApiEndpoint.createBooking	Tạo booking, nhận bookingReference.
BookingHistoryActivity	BookingApiEndpoint.getBookingHistory	Lấy lịch sử booking theo trang.
Profile	AuthApiEndpoint.getProfile	Lấy hồ sơ người dùng theo userId.
MainMenuActivity	NotificationApiEndpoint.getUnreadCount	Lấy số thông báo chưa đọc.

4.7. Các luồng nghiệp vụ cốt lõi được hiện thực

Việc xem xét ứng dụng dưới góc độ từng use case riêng lẻ chỉ phản ánh được các chức năng độc lập của hệ thống. Để đánh giá đầy đủ mức độ hoàn thiện của quá trình triển khai, cần phân tích ứng dụng như một tập hợp các luồng nghiệp vụ liên tục, trong đó mỗi màn hình và mỗi thao tác của người dùng được liên kết với nhau theo một trình tự xử lý cụ thể. Trên cơ sở mã nguồn đã xây dựng và các sơ đồ thiết kế đã trình bày, hệ thống có thể được mô tả thông qua ba luồng nghiệp vụ cốt lõi, bao gồm luồng xác thực người dùng, luồng tìm kiếm và đặt vé, cùng với luồng quản lý sau giao dịch.

Luồng xác thực người dùng được tổ chức từ điểm khởi đầu là MainActivity, sau đó điều hướng đến các màn hình phục vụ đăng nhập và đăng ký tài khoản như Login và Signup. Trong trường hợp người dùng không thể truy cập tài khoản, hệ thống hỗ trợ nhánh xử lý khôi phục thông tin xác thực thông qua ForgotPasswordActivity và ResetPasswordActivity. Luồng này đảm nhiệm chức năng kiểm tra thông tin người dùng, thiết lập trạng thái đăng nhập và duy trì điều kiện truy cập vào các chức năng chính của hệ thống. Đây là luồng nền tảng, bảo đảm rằng các nghiệp vụ tiếp theo chỉ được thực hiện sau khi người dùng hoàn tất bước xác thực cần thiết.

Luồng tìm kiếm và đặt vé là luồng nghiệp vụ trung tâm của ứng dụng. Quá trình này bắt đầu tại SearchFlightActivity, nơi người dùng nhập các tiêu chí tìm kiếm chuyến bay. Kết quả truy vấn được chuyển sang FlightResultsActivity để hiển thị danh sách các chuyến bay phù hợp. Sau khi lựa chọn chuyến bay, người dùng tiếp tục thao tác tại ChooseSeatsActivity để chọn ghế ngồi và tại BookingFormActivity để hoàn thiện thông tin đặt chỗ. Khi dữ liệu được gửi đến backend và xử lý thành công, hệ thống tạo booking tương ứng và đồng thời phát sinh thông báo nội bộ liên quan đến giao dịch. Chuỗi xử lý này cho thấy luồng đặt vé đã được triển khai theo hướng liên kết đầy đủ giữa dữ liệu tìm kiếm, lựa chọn của người dùng, thông tin hành khách và phản hồi từ hệ thống.

Luồng quản lý sau giao dịch được triển khai từ MainMenuActivity và mở rộng đến các màn hình phục vụ tra cứu, theo dõi và cập nhật thông tin cá nhân. Cụ thể, người dùng có thể truy cập BookingHistoryActivity để xem danh sách các đơn đặt vé đã phát sinh, tiếp tục mở BookingDetailActivity để xem chi tiết từng giao dịch, đồng thời sử dụng Profile để quản lý thông tin tài khoản và NotificationActivity để theo dõi các

thông báo nội bộ của hệ thống. Luồng này thể hiện phần nghiệp vụ diễn ra sau khi giao dịch được tạo lập, góp phần hoàn thiện vòng đời sử dụng của ứng dụng từ giai đoạn truy cập hệ thống, phát sinh giao dịch cho đến giai đoạn tra cứu và quản lý thông tin liên quan.

CHƯƠNG 5.
KẾT QUẢ TRIỂN KHAI, KIỂM THỬ VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Trên cơ sở phân tích mã nguồn và các màn hình đã được hiện thực, ứng dụng đã xây dựng được chuỗi chức năng chính gồm: đăng ký tài khoản, đăng nhập, khôi phục mật khẩu bằng OTP, tìm kiếm chuyến bay, hiển thị kết quả tìm kiếm, chọn ghế, tạo booking, xem lịch sử đặt chỗ, xem chi tiết booking, quản lý hồ sơ cá nhân và tiếp nhận thông báo trong ứng dụng [8].

Ở mức độ hoàn thiện hiện tại, hệ thống đã mô tả và hiện thực đầy đủ luồng client-side từ khi người dùng mở ứng dụng cho đến khi hoàn tất thao tác tạo booking. Các nội dung như thanh toán hoàn chỉnh, tối ưu kiến trúc phân lớp, tăng cường bảo mật mạng và bổ sung minh chứng kiểm thử thực nghiệm vẫn là các hạng mục tiếp tục hoàn thiện ở giai đoạn kế tiếp.

5.2. Kiểm thử theo kịch bản

Trong phạm vi tài liệu này, kiểm thử được tổ chức theo hướng đối chiếu luồng nghiệp vụ hiện thực trong mã nguồn với các kịch bản sử dụng của người dùng. Phần này tập trung mô tả mục tiêu kiểm thử, nhóm chức năng cần kiểm tra và điều kiện cần thiết để đánh giá tính đầy đủ của chuỗi thao tác trên thiết bị Android [8].

Các kịch bản kiểm thử được phân thành các nhóm: xác thực người dùng, tìm kiếm chuyến bay, chọn ghế và tạo booking, hậu giao dịch và quản lý tài khoản. Cách phân nhóm này giúp việc kiểm tra bám sát các cụm chức năng chính của ứng dụng, đồng thời thuận tiện cho việc chuẩn bị minh chứng khi trình bày hoặc bảo vệ đồ án.

Bảng 5.1. Kế hoạch kiểm thử chức năng

Nhóm kiểm thử	Màn hình / lớp liên quan	Mục tiêu
Nhóm xác thực	Login, Signup, ForgotPasswordActivity, ResetPasswordActivity	Đảm bảo validate đầu vào, xử lý mã lỗi, lưu phiên đúng.
Nhóm tìm chuyến bay	SearchFlightActivity, FlightResultsActivity	Đảm bảo danh sách sân bay, gửi truy vấn, hiển thị kết quả.
Nhóm chọn ghế & booking	ChooseSeatsActivity, BookingFormActivity	Đảm bảo seat map, kiểm tra ghế, tạo booking, gửi notification.
Nhóm hậu giao dịch	BookingHistoryActivity, BookingDetailActivity, PayActivity	Đảm bảo xem lịch sử, mở chi tiết, điều hướng theo trạng thái.

Nhóm kiểm thử	Màn hình / lớp liên quan	Mục tiêu
Nhóm hồ sơ & tiện ích	MainMenuActivity, Profile, NotificationActivity, AIPlannerActivity	Đảm bảo badge thông báo, hồ sơ, điều hướng menu.

5.3. Kiểm thử thực nghiệm và kết quả quan sát

Bảng 5.2 tổng hợp các trường hợp kiểm thử tiêu biểu được đối chiếu từ mã nguồn và luồng thao tác thực tế của người dùng. Khi hoàn thiện bản nộp chính thức, nhóm nên bổ sung ảnh chụp màn hình hoặc video minh chứng cho một số ca quan trọng như đăng nhập thành công, tìm chuyến bay, chọn ghế và tạo booking để chuyển phần này từ mức mô tả thực nghiệm sang mức minh chứng thực nghiệm đầy đủ.

Bảng 5.2. Kết quả test case tiêu biểu

Mã	Kịch bản	Bước / dữ liệu đầu vào	Kết quả mong đợi	Nhận xét
TC01	Đăng nhập thành công	Nhập username/email và mật khẩu hợp lệ	Mở MainMenuActivity, lưu user_prefs	Đạt theo logic mã nguồn
TC02	Đăng nhập thiếu email	Để trống email/username	Hiển thị setError và không gọi API	Đạt
TC03	Đăng nhập thiếu mật khẩu	Để trống mật khẩu	Hiển thị lỗi và requestFocus()	Đạt
TC04	Đăng ký với email sai định dạng	Nhập email không hợp lệ	Không gọi API, báo lỗi tại field email	Đạt
TC05	Đăng ký với mật khẩu yếu	Mật khẩu < 8 ký tự hoặc thiếu chữ số/chữ hoa	Báo lỗi tại field password	Đạt
TC06	Quên mật khẩu – email hợp lệ	Nhập email đúng	Gọi forgotPassword(), mở OTP dialog	Đạt
TC07	Đặt lại mật khẩu – xác nhận không khớp	Nhập 2 mật khẩu khác nhau	Báo lỗi tại confirmPassword	Đạt
TC08	Tải danh sách sân bay	Mở SearchFlightActivity khi mạng bình thường	Danh sách sân bay được bind vào AutoCompleteTextView	Đạt
TC09	Tìm chuyến bay thiếu điểm đi	Không chọn sân bay đi	Toast yêu cầu chọn sân bay khởi hành	Đạt
TC10	Tìm chuyến bay thành công	Nhập đủ tiêu chí hợp lệ	Mở FlightResultsActivity với search_results_json	Đạt
TC11	Mở kết quả khi chưa đăng nhập	is_logged_in = false	Redirect về Login	Đạt
TC12	Tải seat map thành công	Chọn một chuyến bay hợp lệ	Hiển thị sơ đồ ghế, màu ghế khả dụng/không khả dụng	Đạt
TC13	Chọn ghế nhưng CCCD sai	Nhập < 9 số	Toast báo lỗi và không lưu ghế	Đạt
TC14	Xác nhận booking khi chưa chọn ghế	selectedSeatsInfoMap rỗng	Toast yêu cầu chọn ít nhất một ghế	Đạt
TC15	Tạo booking thành công	Có selectedSeatsList hợp lệ	Gọi createBooking(), hiện thông báo thành công	Đạt
TC16	Tạo booking xung đột ghế	Server trả 409	Toast: ghế đã được đặt bởi người khác	Đạt
TC17	Xem lịch sử booking	Mở BookingHistoryActivity khi có user_id hợp lệ	Tải danh sách booking và hiển thị RecyclerView	Đạt
TC18	Xem hồ sơ cá nhân	Mở Profile khi đã đăng nhập	Gọi getProfile(userId) và cập nhật UI	Đạt

Mã	Kịch bản	Bước / dữ liệu đầu vào	Kết quả mong đợi	Nhận xét
TC19	Tải badge thông báo	Mở MainMenuActivity khi có user_id hợp lệ	Gọi getUnreadCount() và hiển thị badge	Đạt
TC20	Đăng xuất	Chọn logout ở MainMenuActivity	Clear SharedPreferences và quay về Login	Đạt

5.4. Phân tích hiện trạng triển khai

Về phạm vi chức năng, dự án đã bao phủ hầu hết các bước chính của quy trình đặt vé máy bay trên thiết bị di động, từ xác thực tài khoản, tìm kiếm chuyến bay, lựa chọn ghế ngồi đến theo dõi lịch sử booking và quản lý hồ sơ người dùng.

Về tổ chức mã nguồn, tên lớp, luồng điều hướng và cách phân nhóm endpoint được xây dựng theo hướng bám sát nghiệp vụ, nhờ đó việc lần theo source code, đối chiếu chức năng và mô tả trong báo cáo được thực hiện tương đối trực tiếp.

Về xử lý dữ liệu đầu vào, nhiều màn hình đã triển khai cơ chế kiểm tra rỗng, định dạng và tính hợp lệ trước khi gửi yêu cầu tới backend. Cơ chế này giúp giảm số lần gọi API không cần thiết và giữ cho phản hồi giao diện nhất quán hơn trong quá trình sử dụng.

Về trải nghiệm sau giao dịch, luồng tạo booking được nối tiếp bởi thông báo nội bộ, màn hình chi tiết booking và lịch sử đặt chỗ. Cách tổ chức này cho phép người dùng quay lại tra cứu thông tin sau khi hoàn tất thao tác đặt vé.

5.4.1. Nội dung kỹ thuật cần tiếp tục hoàn thiện

Bên cạnh các nội dung đã được hiện thực, mã nguồn hiện tại vẫn còn một số hạng mục kỹ thuật cần tiếp tục chuẩn hóa để đáp ứng tốt hơn yêu cầu triển khai thực tế. Các nội dung này chủ yếu liên quan đến cấu hình môi trường, quản lý phiên, tổ chức lớp nghiệp vụ và khả năng bảo trì của ứng dụng.

Bảng 5.3. Nội dung kỹ thuật cần tiếp tục hoàn thiện

Hạn chế	Biểu hiện trong mã nguồn	Tác động
Hard-code base URL và dùng HTTP	RetrofitClient chọn giữa 10.0.2.2 và một IP LAN cố định, chưa thấy HTTPS	Khó triển khai production, rủi ro bảo mật mạng, phụ thuộc môi trường demo.
Quản lý phiên bằng SharedPreferences thường	Không thấy token / refresh token / encrypted storage	Phiên dễ mất đồng bộ với server, bảo mật cục bộ còn hạn chế.
Activity gánh nhiều trách nhiệm	Validate, gọi API, xử lý lỗi, điều hướng và cập nhật UI nằm chung trong một lớp	Khó kiểm thử tự động, khó mở rộng, dễ phình to mã nguồn.

Hạn chế	Biểu hiện trong mã nguồn	Tác động
Lặp lại logic lưu user info	Login và Signup đều có hàm saveUserInfo gần giống nhau	Gia tăng trùng lặp, khó bảo trì khi cấu trúc hồ sơ đổi.
Dữ liệu dummy / hard-code cục bộ	Profile còn gán cứng countriesCount, milesCount; SearchFlightActivity tự tạo danh sách SeatClass mẫu	Có sự chênh giữa dữ liệu thật và dữ liệu mô phỏng.
Rủi ro lỗi null / binding chưa hoàn tất	BookingHistoryActivity sử dụng chipUpcoming, chipCompleted nhưng phần bind view trích xuất được chưa thấy khởi tạo	Có thể phát sinh NPE khi người dùng thao tác bộ lọc.
Thiếu lớp repository / viewmodel	Chưa có tầng trung gian giữa UI và API	Khó tái sử dụng logic và quản lý state hiện đại.

5.5. Hướng phát triển

Hướng phát triển thứ nhất là tái cấu trúc kiến trúc client sang mô hình có phân lớp rõ hơn như MVVM hoặc ít nhất là tách Repository. Khi đó Activity chỉ còn tập trung vào giao diện và điều hướng, còn logic gọi API và xử lý dữ liệu được gom về các lớp tái sử dụng. Đây là bước đi cần thiết nếu dự án muốn mở rộng số lượng màn hình, thêm tính năng thanh toán hoặc đồng bộ đa nền tảng.

Hướng phát triển thứ hai là nâng cấp bảo mật và quản lý môi trường. Base URL nên được đưa về BuildConfig hoặc hệ thống environment config; giao tiếp mạng cần chuyển sang HTTPS; xác thực nên dùng token kèm interceptor để xử lý 401 tập trung. Nếu dữ liệu phiên vẫn phải lưu cục bộ, có thể xem xét EncryptedSharedPreferences hoặc DataStore.

Hướng phát triển thứ ba là tăng chiều sâu nghiệp vụ. Ứng dụng có thể bổ sung mã QR/e-ticket, check-in, chọn thêm hành lý, lưu thẻ thanh toán, đồng bộ lịch trình, hoặc tích hợp cổng thanh toán thật. Nếu AIPlannerActivity được phát triển mạnh hơn, nó có thể tư vấn lịch trình đi lại, gợi ý sân bay phù hợp, hoặc tổng hợp booking thành một kế hoạch chuyến đi hoàn chỉnh.

Hướng phát triển thứ tư là tăng chất lượng kỹ thuật thông qua kiểm thử tự động. Những màn hình như Login, SearchFlightActivity, ChooseSeatsActivity và BookingHistoryActivity đều là ứng viên tốt để viết unit test cho validate hoặc instrumentation test cho điều hướng và hiển thị giao diện.

5.6. Mức độ hoàn thành so với mục tiêu ban đầu

So với mục tiêu của một đồ án ứng dụng mobile, repo hiện tại đã đạt mức hoàn thiện tương đối rõ

Tài liệu báo cáo được biên soạn lại trong phiên bản này cũng hoàn thành mục tiêu thứ hai: chuyển repo thành một báo cáo học thuật đầy đủ với trên 50 trang, gồm các phần mà một báo cáo đồ án nghiêm túc thường cần có như yêu cầu hệ thống, cơ sở lý thuyết, thiết kế, kết quả triển khai, kiểm thử, đánh giá và phụ lục. Vì vậy, xét cả ở góc độ sản phẩm lẫn ở góc độ học thuật, đồ án đã đạt được nền tảng đủ tốt để tiếp tục hoàn thiện cho kỳ nộp chính thức.

CHƯƠNG 6

MÔ-ĐUN AI TRONG HỆ THỐNG

6.1. Vai trò của AI trong hệ thống

Trong cấu trúc hiện tại của ứng dụng, AI được xem như một mô-đun hỗ trợ quyết định, bổ sung cho các chức năng nghiệp vụ cốt lõi như tìm kiếm chuyến bay, lựa chọn ghế và theo dõi lịch sử đặt chỗ. Điểm tích hợp AI thể hiện rõ nhất ở AIPlannerActivity, nơi người dùng có thể nhập yêu cầu ở dạng ngôn ngữ tự nhiên để nhận gợi ý về hành trình, lựa chọn chuyến bay hoặc bước thao tác tiếp theo [8].

Khác với nhóm màn hình booking vốn xử lý theo dữ liệu có cấu trúc, mô-đun AI tiếp nhận dữ liệu bán cấu trúc và dữ liệu ngữ nghĩa, sau đó biến chúng thành phản hồi dễ hiểu cho người dùng. Điều này giúp ứng dụng không chỉ dừng ở việc hiển thị thông tin từ API mà còn có khả năng giải thích, gợi ý và cá nhân hóa trải nghiệm sử dụng ở lớp giao diện [8][10].

6.2. Cách AI hoạt động trong hệ thống

Về nguyên lý, mô-đun AI hoạt động theo chuỗi xử lý gồm năm pha. Pha thứ nhất là thu nhận yêu cầu từ người dùng tại AIPlannerActivity, chẳng hạn như yêu cầu gợi ý lịch trình, chọn thời điểm bay phù hợp hoặc tóm tắt lựa chọn chuyến bay. Pha thứ hai là chuẩn hóa dữ liệu đầu vào, bao gồm làm sạch câu hỏi, nhận diện ý định và tách các tham số quan trọng như điểm đi, điểm đến, ngày bay, số hành khách hoặc nhu cầu đặc biệt. Pha thứ ba là ghép ngữ cảnh từ hệ thống, trong đó mô-đun AI có thể khai thác hồ sơ người dùng, lịch sử booking, tiêu chí tìm kiếm hiện tại và dữ liệu chuyến bay lấy từ backend [8].

Pha thứ tư là suy luận. Tùy theo yêu cầu, hệ thống có thể sử dụng cách xử lý dựa trên luật, mô hình tạo sinh trên đám mây hoặc mô hình AI trên thiết bị. Tài liệu chính thức của Android cho thấy nhà phát triển có thể lựa chọn giữa hướng AI chạy trên thiết bị nhằm tối ưu độ trễ và quyền riêng tư, hoặc hướng mô hình đám mây để xử lý ngữ cảnh lớn và bài toán suy luận phức tạp hơn [10][11]. Pha cuối cùng là hậu xử lý, nơi kết quả được chuẩn hóa thành các câu gợi ý ngắn gọn, ưu tiên những phương án phù hợp với dữ liệu nghiệp vụ của hệ thống trước khi hiển thị cho người dùng [12].

Bảng 6.1. Thành phần của mô-đun AI và vai trò xử lý

Thành phần	Vai trò	Nguồn dữ liệu chính
AIPlannerActivity	Thu nhận yêu cầu, hiển thị phản hồi và duy trì phiên tương tác với người dùng.	Yêu cầu văn bản, trạng thái giao diện, user_prefs
Bộ ghép ngữ cảnh	Chuẩn hóa câu hỏi, thêm tham số hành trình, hồ sơ và dữ liệu nghiệp vụ để tạo prompt.	Hồ sơ người dùng, lịch sử booking, tiêu chí tìm chuyến bay
Dịch vụ AI	Suy luận và sinh gợi ý, giải thích hoặc tóm tắt cho người dùng.	Prompt đã ghép, dữ liệu truy hồi, luật nghiệp vụ
Bộ hậu xử lý	Lọc kết quả, xếp hạng phương án, chuẩn hóa ngôn ngữ hiển thị và kiểm tra an toàn.	Kết quả suy luận, ngưỡng ưu tiên, quy tắc hiển thị

6.3. Phương pháp và xu hướng triển khai AI

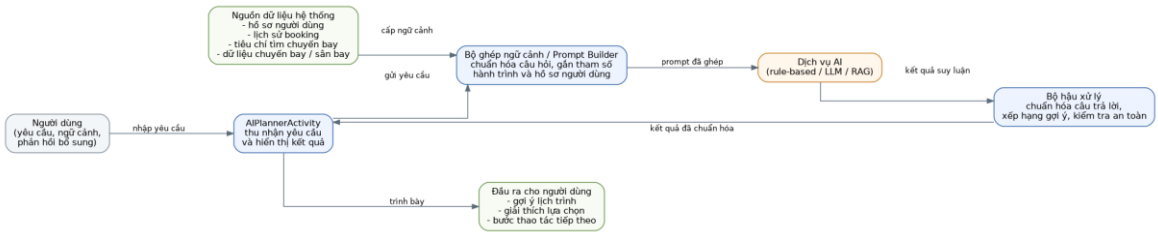
Từ góc độ kỹ thuật, có thể phân biệt ba hướng triển khai AI phù hợp với hệ thống này. Hướng thứ nhất là xử lý dựa trên luật và template. Ở hướng này, hệ thống xây dựng các quy tắc suy diễn đơn giản từ dữ liệu chuyến bay, hồ sơ và lịch sử đặt chỗ để tạo gợi ý mang tính quyết định. Phương pháp này có ưu điểm là dễ kiểm soát, dễ giải thích và phù hợp với những nghiệp vụ có cấu trúc rõ ràng như kiểm tra điều kiện đặt vé, gợi ý bước tiếp theo hoặc cảnh báo dữ liệu thiếu.

Hướng thứ hai là tích hợp mô hình tạo sinh trên đám mây hoặc trên thiết bị. Theo hướng dẫn của Android, AI trên thiết bị có lợi thế về quyền riêng tư, chi phí suy luận thấp và khả năng hoạt động ngoại tuyến, trong khi mô hình trên đám mây phù hợp hơn với các tác vụ cần năng lực suy luận mạnh, xử lý ngữ cảnh dài hoặc đối thoại đa vòng [10][11]. Đối với ứng dụng đặt vé máy bay, cách tiếp cận này thích hợp cho các tình huống như giải thích phương án bay, tóm tắt lựa chọn hiện tại hoặc sinh khuyến nghị bằng ngôn ngữ tự nhiên.

Hướng thứ ba là kiến trúc lai có truy hồi dữ liệu, thường được gọi là Retrieval-Augmented Generation (RAG). Ở kiến trúc này, mô hình tạo sinh không trả lời chỉ dựa trên tham số học sẵn mà còn được bổ sung thông tin truy hồi từ cơ sở dữ liệu, tập tài liệu hoặc API nghiệp vụ. Cách làm này giúp phản hồi bám sát dữ liệu hiện hành của hệ thống và giảm rủi ro sinh nội dung không phù hợp với dữ liệu thực tế [12][13]. Với ứng dụng đặt vé máy bay, RAG là hướng có giá trị vì AI cần căn cứ vào dữ liệu chuyến bay, hồ sơ và lịch sử đặt vé chứ không chỉ dựa vào tri thức chung.

6.4. Sơ đồ luồng dữ liệu của mô-đun AI

Hình 6.1 mô tả luồng dữ liệu của mô-đun AI từ thời điểm người dùng phát sinh yêu cầu cho đến khi kết quả được hiển thị trên giao diện. Trong kiến trúc này, AIPlannerActivity đóng vai trò điểm vào, bộ ghép ngữ cảnh đóng vai trò tạo prompt có kiểm soát, còn dữ liệu nghiệp vụ của hệ thống được đưa vào như nguồn grounding nhằm tăng tính phù hợp của phản hồi [8][12].

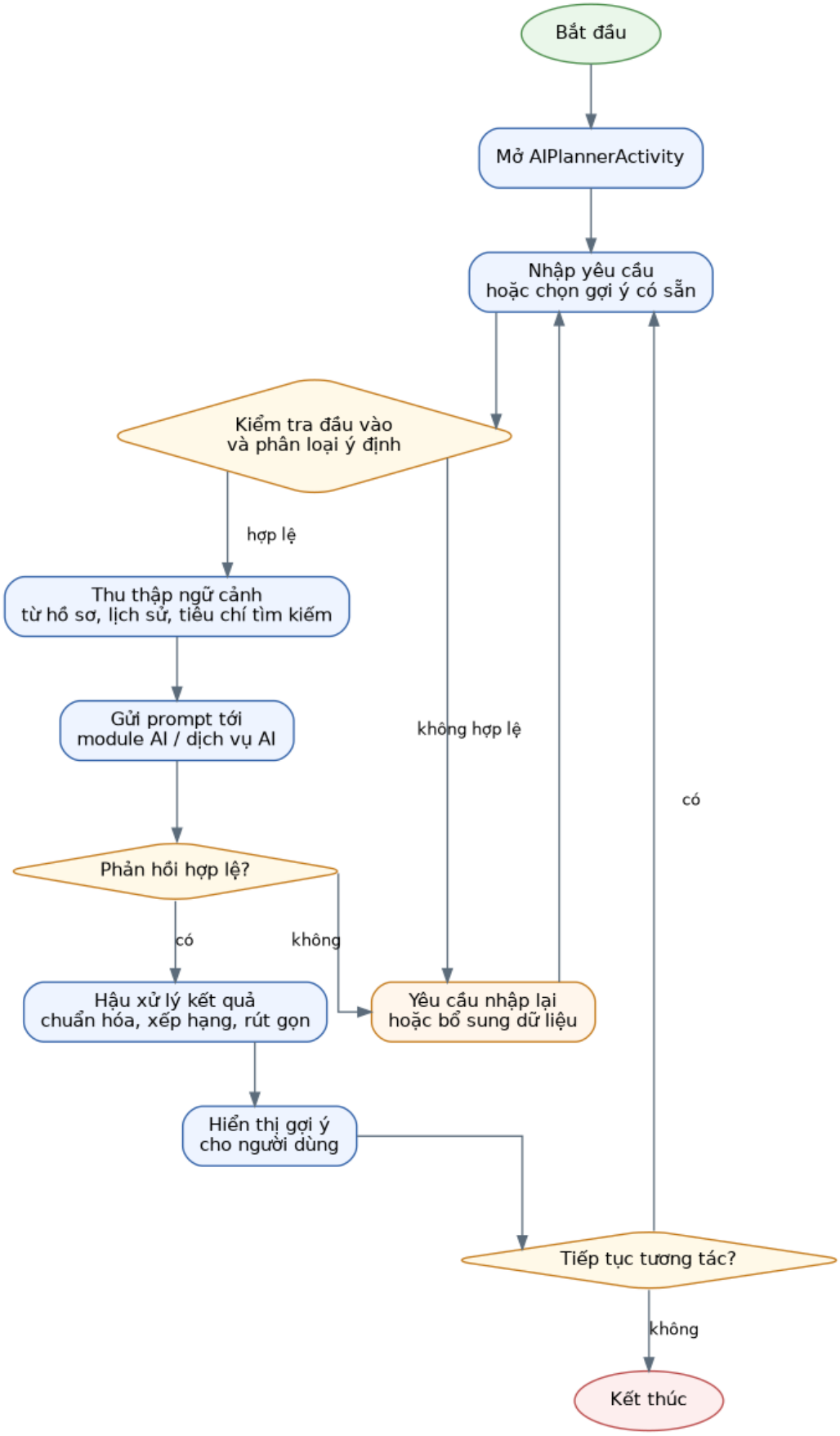


Hình 6.1. Sơ đồ luồng dữ liệu của mô-đun AI trong hệ thống

Luồng dữ liệu trên cho thấy AI không hoạt động tách rời khỏi hệ thống nghiệp vụ. Dữ liệu đầu vào của mô-đun bao gồm yêu cầu ngôn ngữ tự nhiên của người dùng, hồ sơ người dùng, lịch sử booking và ngữ cảnh của phiên tìm kiếm hiện tại. Sau khi được ghép ngữ cảnh, dữ liệu được chuyển sang lớp suy luận AI để sinh phản hồi. Kết quả sau đó được hậu xử lý nhằm loại bỏ nội dung dư thừa, tăng khả năng giải thích và đưa về định dạng dễ hiển thị trong ứng dụng Android.

6.5. Sơ đồ hoạt động của mô-đun AI

Bên cạnh luồng dữ liệu, sơ đồ hoạt động mô tả trình tự thao tác của mô-đun AI ở mức tiến trình. Hình 6.2 cho thấy mô-đun AI bắt đầu bằng việc nhận yêu cầu, kiểm tra hợp lệ của dữ liệu, truy hồi ngữ cảnh cần thiết, gửi yêu cầu suy luận và cuối cùng hiển thị kết quả cho người dùng. Trong trường hợp dữ liệu đầu vào chưa đủ hoặc phản hồi không hợp lệ, hệ thống quay lại bước yêu cầu người dùng bổ sung thông tin để bảo đảm chất lượng đầu ra.



Hình 6.2. Sơ đồ hoạt động của mô-đun AI trong ứng dụng

Từ sơ đồ hoạt động có thể thấy mô-đun AI được tổ chức như một vòng lặp tương tác thay vì một lời gọi một chiều. Người dùng có thể tiếp tục đặt câu hỏi bổ sung, chỉnh sửa yêu cầu hoặc cung cấp thêm dữ liệu để hệ thống sinh gợi ý mới. Cách thiết kế này phù hợp với đặc tính của các tác vụ tư vấn và lập kế hoạch, trong đó nhu cầu của người dùng thường được làm rõ dần qua nhiều bước trao đổi.

6.6. Cách AI xử lý và đưa ra dữ liệu cho người dùng

Xét theo chuỗi xử lý dữ liệu, mô-đun AI có thể được mô tả qua ba lớp: dữ liệu đầu vào, lớp suy luận và dữ liệu đầu ra. Ở lớp đầu vào, hệ thống tiếp nhận yêu cầu văn bản của người dùng, hồ sơ cá nhân cơ bản, dữ liệu lịch sử booking và ngữ cảnh tìm chuyến bay hiện tại. Ở lớp suy luận, yêu cầu được phân tích ý định, truy hồi dữ liệu liên quan, ghép thành prompt, sau đó được chuyển tới bộ máy AI để sinh phản hồi. Ở lớp đầu ra, phản hồi được rút gọn thành gợi ý cụ thể như lựa chọn bước thao tác tiếp theo, khuyến nghị thời gian bay hoặc giải thích tại sao một phương án phù hợp hơn phương án khác [8][10][12].

Bảng 6.2. Chuỗi xử lý dữ liệu của mô-đun AI

Giai đoạn	Dữ liệu sử dụng	Kết quả tạo ra
Đầu vào	Yêu cầu tự nhiên, hồ sơ người dùng, lịch sử booking, tiêu chí tìm chuyến bay.	Tập tham số và ngữ cảnh ban đầu cho phiên AI.
Suy luận	Prompt đã ghép, dữ liệu truy hồi, luật nghiệp vụ và cấu hình mô hình.	Phản hồi trung gian, điểm ưu tiên và gợi ý có giải thích.
Đầu ra	Kết quả hậu xử lý, danh sách gợi ý ưu tiên, câu trả lời đã rút gọn.	Nội dung hiển thị trên AIPlannerActivity hoặc dữ liệu dùng để điều hướng tiếp.

Với cách tổ chức này, AI đóng vai trò là lớp hỗ trợ ra quyết định và diễn giải dữ liệu cho người dùng cuối. Trong giai đoạn hiện tại của đồ án, AIPlannerActivity có thể được xem là hạt nhân giao diện của mô-đun AI; còn về hướng phát triển tiếp theo, mô-đun này có thể được mở rộng theo hướng RAG hoặc AI trên thiết bị để tăng tính cá nhân hóa, độ chính xác và khả năng hoạt động trong môi trường di động [10][11][12].

TÀI LIỆU THAM KHẢO

[1] Android Developers (2026), The activity lifecycle, available at: https://developer.android.com/guide/components/activities/activity-lifecycle, accessed: 22/03/2026.

[2] Square (2026), Declarations | Retrofit, available at: https://square.github.io/retrofit/declarations/, accessed: 22/03/2026.

[3] Google Gson Project (2026), Gson User Guide, available at: https://github.com/google/gson/blob/main/UserGuide.md, accessed: 22/03/2026.

[4] Android Developers (2026), Save simple data with SharedPreferences, available at: https://developer.android.com/training/data-storage/shared-preferences, accessed: 22/03/2026.

[5] Android Developers (2026), Material Design for Android; TextInputLayout và TextInputEditText API reference, available at: https://developer.android.com/develop/ui/views/theming/look-and-feel; https://developer.android.com/reference/com/google/android/material/textfield/TextInputLayout; https://developer.android.com/reference/com/google/android/material/textfield/TextInputEditText, accessed: 22/03/2026.

[6] Android Developers (2026), Create dynamic lists with RecyclerView, available at: https://developer.android.com/develop/ui/views/layout/recyclerview, accessed: 22/03/2026.

[7] Android Developers (2026), Create and manage notification channels, available at: https://developer.android.com/develop/ui/views/notifications/channels, accessed: 22/03/2026.

[8] kienduy13 (2026), DoAn2_UngDungDatVeMayBay_Duy, GitHub repository, available at: https://github.com/kienduy13/DoAn2_UngDungDatVeMayBay_Duy, accessed: 22/03/2026.

[9] Tài liệu hướng dẫn báo cáo đồ án do người dùng cung cấp (DOCX), truy cập trong phiên làm việc ngày 22/03/2026.

[10] Android Developers (2026), Find the right AI/ML solution for your app, available at: https://developer.android.com/ai/generativeai, accessed: 23/03/2026.

[11] Android Developers (2026), Gemini Nano, available at: https://developer.android.com/ai/gemini-nano, accessed: 23/03/2026.

[12] Google Cloud (2026), What is Retrieval-Augmented Generation (RAG)?, available at: https://cloud.google.com/use-cases/retrieval-augmented-generation, accessed: 23/03/2026.

PHỤ LỤC

PL.1. Danh mục file mã nguồn trọng yếu

Bảng PL.1. Danh mục file mã nguồn trọng yếu

File / đường dẫn rút gọn	Vai trò
UI/FlightBooking/app/src/main/java/com/prm/flightbooking/MainActivity.java	Màn chào – điểm vào đầu tiên.
.../Login.java	Đăng nhập và lưu phiên người dùng.
.../Signup.java	Đăng ký tài khoản mới.
.../ForgotPasswordActivity.java	Gửi OTP khôi phục mật khẩu.
.../ResetPasswordActivity.java	Đặt lại mật khẩu.
.../MainMenuActivity.java	Menu trung tâm sau đăng nhập.
.../SearchFlightActivity.java	Nhập tiêu chí tìm kiếm chuyến bay.
.../FlightResultsActivity.java	Hiển thị danh sách chuyến bay.
.../ChooseSeatsActivity.java	Tải seat map, chọn ghế, nhập thông tin hành khách.
.../BookingFormActivity.java	Tạo booking và gửi thông báo.
.../BookingHistoryActivity.java	Lịch sử booking và bộ lọc trạng thái.
.../Profile.java	Hiển thị hồ sơ người dùng.
.../EditProfileActivity.java	Chỉnh sửa hồ sơ.
.../ChangePasswordActivity.java	Đổi mật khẩu khi đang đăng nhập.
.../NotificationActivity.java	Hiển thị thông báo.
.../BookingDetailActivity.java	Chi tiết booking.
.../PayActivity.java	Màn hình thanh toán / xác nhận phát sinh.
.../AIPlannerActivity.java	Màn hình hỗ trợ AI Planner.
.../api/RetrofitClient.java	Khởi tạo Retrofit và chọn base URL.
.../api/ApiServiceProvider.java	Factory tạo các interface API.
.../api/AuthApiEndpoint.java	Contract API nhóm xác thực.
.../api/BookingApiEndpoint.java	Contract API nhóm booking.
.../api/FlightApiEndpoint.java	Contract API nhóm chuyến bay.
.../fragments/OtpVerificationDialogFragment.java	Dialog xác thực OTP.

PL.2. Ma trận truy vết yêu cầu – hiện thực

Bảng PL.2. Ma trận truy vết yêu cầu – hiện thực

Mã	Yêu cầu	Hiện thực trong repo	Mức độ
F1	MainActivity → Login	MainActivity	Có
F2	Đăng nhập người dùng	Login, AuthApiEndpoint.login	Có
F3	Đăng ký tài khoản	Signup, AuthApiEndpoint.register	Có
F4	Quên / đặt lại mật khẩu	ForgotPasswordActivity, OtpVerificationDialogFragment, ResetPasswordActivity	Có
F5	Menu trung tâm	MainMenuActivity	Có
F6	Tải sân bay và tìm chuyến bay	SearchFlightActivity, Airport API, Search API	Có

Mã	Yêu cầu	Hiện thực trong repo	Mức độ
F7	Hiển thị kết quả tìm kiếm	FlightResultsActivity	Có
F8	Chọn ghế	ChooseSeatsActivity, FlightApiEndpoint.getSeatMap	Có
F9	Tạo booking	BookingFormActivity, BookingApiEndpoint.createBooking	Có
F10	Lịch sử booking	BookingHistoryActivity	Có
F11	Chi tiết booking	BookingDetailActivity	Có
F12	Thanh toán / xử lý phát sinh	PayActivity	Một phần
F13	Hiển thị hồ sơ	Profile, AuthApiEndpoint.getProfile	Có
F14	Thông báo chưa đọc	MainMenuActivity, Notification API	Có
F15	AI Planner	AIPlannerActivity	Có file / cần hoàn thiện thêm

PL.3. Hướng dẫn cài đặt, triển khai và cấu hình chương trình

Phụ lục này mô tả các bước cài đặt và cấu hình cần thiết để ứng dụng Android có thể kết nối tới backend và được chạy thử trong môi trường nội bộ. Các bước triển khai nên được thực hiện theo đúng trình tự để hạn chế lỗi phát sinh trong quá trình build và gọi API.

Bảng PL.3. Yêu cầu môi trường cài đặt và cấu hình triển khai

Bước 1: cài đặt Android Studio và bộ Android SDK tương thích với dự án.

Bước 2: mở mã nguồn Android, chờ Gradle sync hoàn tất và xử lý toàn bộ dependency còn thiếu.

Bước 3: khởi động backend REST API trên máy chủ cục bộ hoặc máy chủ trong cùng mạng LAN.

Bước 4: cập nhật base URL trong RetrofitClient theo môi trường chạy thử (10.0.2.2 cho giả lập hoặc IP LAN cho thiết bị thật).

Bước 5: chạy ứng dụng trên emulator hoặc thiết bị thật, kiểm tra lần lượt các luồng đăng nhập, tìm kiếm chuyến bay, chọn ghế và tạo booking.

Bước 6: chuẩn bị một tài khoản demo và dữ liệu chuyến bay phù hợp để bảo đảm phần trình diễn có thể thực hiện liên tục.

Nội dung	Thiết lập cần có	Ghi chú
Mã nguồn Android	Mở dự án trong Android Studio	Kiểm tra Gradle sync trước khi chạy.
SDK / JDK	Cài Android SDK tương thích và JDK 17	Bảo đảm chuỗi build hoạt động ổn định.

Thiết bị chạy thử	Emulator hoặc điện thoại Android	Nếu dùng máy thật, bật USB debugging khi cần.
Backend	Khởi động REST API trước khi mở app	Backend và app phải nhìn thấy nhau qua mạng.
Base URL	Emulator dùng 10.0.2.2; máy thật dùng IP LAN của máy chủ	Cập nhật đúng địa chỉ trong RetrofitClient trước khi build.
Kết nối mạng	Thiết bị và backend cùng mạng nội bộ	Kiểm tra ping hoặc truy cập thử API nếu cần.
Tài khoản demo	Chuẩn bị sẵn tài khoản thử nghiệm hoặc API đăng ký mới	Nên có ít nhất một tài khoản để trình diễn luồng hoàn chỉnh.

PL.4. Hướng dẫn sử dụng chương trình

Mở ứng dụng và chọn Get Started để chuyển sang màn hình đăng nhập.

Đăng nhập bằng tài khoản đã có hoặc tạo tài khoản mới bằng màn hình Signup.

Nếu quên mật khẩu, sử dụng luồng Forgot Password, nhập OTP và đặt lại mật khẩu.

Từ MainMenuActivity, mở chức năng tìm chuyến bay, chọn điểm đi, điểm đến, ngày bay, số hành khách và hạng ghế.

Chọn chuyến bay phù hợp từ màn hình kết quả; tiếp tục mở sơ đồ ghế và khai báo thông tin hành khách theo từng ghế đã chọn.

Xác nhận thông tin tại BookingFormActivity để tạo booking và nhận mã đặt chỗ.

Mở BookingHistoryActivity để tra cứu lịch sử đặt vé, hoặc Profile để xem và cập nhật thông tin cá nhân.

Quy trình sử dụng ứng dụng được tổ chức theo trình tự thao tác của người dùng cuối. Việc mô tả tuần tự từng bước giúp báo cáo dễ đối chiếu với phần cài đặt và thuận tiện cho việc hướng dẫn demo khi bảo vệ đồ án.

PL.5. Vị trí đề xuất chèn ảnh chụp màn hình thực tế

Đối với bản in chính thức, nên ưu tiên ảnh chụp thực tế thay cho hình mô phỏng ở các màn hình đại diện cho các cụm chức năng chính. Bảng dưới đây tổng hợp các vị trí chèn ảnh phù hợp để phần mô tả giao diện và phần minh chứng thực nghiệm được liên kết trực tiếp.

Bảng PL.4. Vị trí đề xuất bổ sung ảnh minh chứng thực tế

Màn hình / nhóm màn hình	Ảnh minh chứng nên chụp	Vị trí nên đặt trong báo cáo

MainActivity, Login	Màn welcome, form đăng nhập, thông báo lỗi/success	Mục 4.2.1
Signup, Forgot Password, Reset Password	Tạo tài khoản, nhập OTP, đặt lại mật khẩu	Mục 4.2.2
SearchFlightActivity, FlightResultsActivity	Form tìm kiếm và danh sách chuyến bay	Mục 4.3.2
ChooseSeatsActivity	Sơ đồ ghế và trạng thái ghế đã chọn	Mục 4.4.1
BookingFormActivity, BookingDetailActivity, PayActivity	Thông tin booking và mã đặt chỗ	Mục 4.4.2
BookingHistoryActivity	Lịch sử đặt vé, bộ lọc trạng thái	Mục 4.5.1
Profile, NotificationActivity	Thông tin người dùng và thông báo	Mục 4.5.2

PL.6. Gợi ý kịch bản demo khi bảo vệ đồ án

Bước 1: mở MainActivity và giải thích đây là điểm vào duy nhất của ứng dụng.

Bước 2: trình bày Login/Signup, nhấn mạnh validate đầu vào và lưu user_prefs sau khi đăng nhập thành công.

Bước 3: vào MainMenuActivity, giới thiệu các menu chính và badge thông báo chưa đọc.

Bước 4: mở SearchFlightActivity, chọn sân bay, ngày bay, số hành khách và hạng ghế.

Bước 5: vào FlightResultsActivity, chọn một chuyến bay, nhấn mạnh dữ liệu được truyền bằng search_results_json.

Bước 6: mở ChooseSeatsActivity, chọn ghế và nhập thông tin hành khách cho từng ghế.

Bước 7: vào BookingFormActivity, xác nhận booking và minh họa notification nội bộ.

Bước 8: quay lại BookingHistoryActivity, mở chi tiết booking và trình bày các trạng thái Confirmed / Completed / Cancelled.

Bước 9: vào Profile để cho thấy app có cụm quản lý tài khoản, không chỉ là form booking.

Bước 10: kết thúc bằng việc tổng hợp kết quả đạt được, các nội dung kỹ thuật cần tiếp tục hoàn thiện và hướng phát triển của hệ thống.

PL.7. Khuyến nghị chỉnh sửa trước khi nộp bản chính thức

Nếu đề tài được thực hiện theo nhóm, nên bổ sung thêm một bảng phân công công việc, trong đó nêu rõ thành viên phụ trách giao diện, API, kiểm thử và biên soạn báo cáo. Trường hợp đề tài được thực hiện cá nhân thì không cần thêm nội dung này.

- Điền đầy đủ thông tin sinh viên, MSSV, lớp, giảng viên hướng dẫn ở trang bìa và trang thông tin chung.
- Nếu có ảnh chụp màn hình thật từ ứng dụng đang chạy, có thể thay một phần wireframe minh họa bằng screenshot thật để tăng tính thuyết phục.
- Mở file bằng Microsoft Word và rà soát lại mục lục, khoảng cách đoạn, page break cuối cùng trước khi in.
- Nếu trường yêu cầu danh sách hình và danh sách bảng có số trang, có thể dùng Word để cập nhật lại sau khi khóa nội dung cuối cùng.
- Khi bảo vệ, nên chuẩn bị thêm video demo hoặc APK cài đặt để minh chứng sự tương ứng giữa báo cáo và sản phẩm.

PL.8. Bảng khóa SharedPreferences và nơi sử dụng

Bảng PL.5. Khóa SharedPreferences và nơi sử dụng

Khóa	Nơi sử dụng	Vai trò
user_id	Login, Signup (ghi); MainMenuActivity, SearchFlightActivity, ChooseSeatsActivity, BookingFormActivity, BookingHistoryActivity, Profile (đọc)	Khóa định danh người dùng cho toàn bộ phiên.
username	Login, Signup (ghi); MainMenuActivity (đọc)	Tên đăng nhập rút gọn.
user_name	Login, Signup (ghi); MainMenuActivity, SearchFlightActivity (đọc)	Tên hiển thị của người dùng.
user_email	Login, Signup (ghi)	Email lưu cục bộ phục vụ hiển thị/tái sử dụng.
user_phone	Login, Signup (ghi)	Số điện thoại người dùng.
date_of_birth	Login, Signup (ghi)	Ngày sinh.
gender	Login, Signup (ghi)	Giới tính.
total_bookings	Login, Signup (ghi); Profile (đọc gián tiếp hoặc gọi lại API)	Tổng số booking.
is_logged_in	Login, Signup (ghi); nhiều màn hình hậu giao dịch (đọc)	Cờ xác định trạng thái đăng nhập.
flightId	ChooseSeatsActivity (ghi); BookingFormActivity (đọc)	Mã chuyến bay tạm thời trong luồng booking.

PL.9. Bảng mô tả DTO được dùng trực tiếp trong luồng nghiệp vụ

Bảng PL.6. DTO được dùng trực tiếp trong luồng nghiệp vụ

DTO	Màn hình sử dụng	Thuộc tính tiêu biểu	Mục đích
LoginDto	Login	usernameOrEmail, password	Đóng gói dữ liệu đăng nhập.
RegisterUserDto	Signup	username, email, password, fullName, phone, dateOfBirth, gender	Đóng gói dữ liệu đăng ký.
UserProfileDto	Login, Signup, Profile	userId, username, fullName, email, phone, dateOfBirth, gender, totalBookings	Đại diện hồ sơ người dùng trên client.
ForgotPasswordRequestDto	ForgotPasswordActivity	email	Yêu cầu gửi OTP.
ResetPasswordDto	ResetPasswordActivity	email, otpCode, newPassword	Yêu cầu đặt lại mật khẩu.
AdvancedFlightSearchDto	SearchFlightActivity	departureAirportCode, arrivalAirportCode, departureDate, returnDate, passengers, seatClass	Yêu cầu tìm chuyến bay.
FlightSearchResultDto	SearchFlightActivity, FlightResultsActivity	outboundFlights, returnFlights	Kết quả tìm kiếm chuyến bay.
SeatDto / SeatMapDto	ChooseSeatsActivity	seatId, seatNumber, seatRow, seatColumn, seatClassName, totalPrice, availability	Dữ liệu bản đồ ghế.
SelectedSeatInfo	ChooseSeatsActivity, BookingFormActivity	seatId, seatNumber, seatClassName, totalPrice, passengerName, passengerIdNumber	Thông tin ghế đã chọn và hành khách.
BookingSeatDto	BookingFormActivity	seatId, passengerName, passengerIdNumber	Chi tiết từng ghế gửi lên booking API.
CreateBookingDto	BookingFormActivity	userId, flightId, seats, notes	Yêu cầu tạo booking.
BookingResponseDto	BookingFormActivity	bookingReference, bookingId	Phản hồi tạo booking thành công.
UserBookingHistoryDto	BookingHistoryActivity	bookingId, bookingReference, bookingStatus, airlineName, flightNumber, route, totalAmount, departureTime, arrivalTime	Dữ liệu lịch sử booking hiển thị cho người dùng.